Vellore Institute of Technology, Vellore

Computer Networks Lab (BCSE308P)

When Does Adapting Concurrency and Range Size Help?

A Controlled Study of Parallel HTTP Range Downloads

Team Handbook

Everything in the project, for the viva and the presentation

Name	Registration number
Sourajyoti Basak	24BCE0774
Shwetank Shrijan	24BDS0141
Ayush Konar	24BDS0107

Prepared 26 October 2026 · Viva: Thursday 29 October 2026

Every number in this handbook comes from `reports/paper_draft.md` (checked by `tools/claims_check.py`, 220 of 220 numbers) or from the named study folder under `testdata/`.

Contents

How to Use This Handbook **1**

1 The Project in One Page **3**

1.1 The question . . . 3
1.2 The answer in ten findings . . . 3
1.3 The logic of the project . . . 4
1.4 What we built . . . 5
1.5 What is novel, in one paragraph . . . 5
1.6 The takeaway . . . 6

2 Background You Need **7**

2.1 HTTP range requests . . . 7
2.2 Segmented (parallel) downloading . . . 7
2.2.1 Why more connections can help . . . 8
2.2.2 Why more connections can hurt . . . 8
2.2.3 Why range size matters . . . 8
2.2.4 Straggler splitting . . . 8
2.3 The tools we compared against . . . 9
2.4 Network emulation and trace replay . . . 9
2.5 Reinforcement learning in five minutes . . . 9
2.5.1 The setting . . . 9
2.5.2 Q-learning and DQN . . . 10
2.5.3 Double DQN . . . 10
2.5.4 Contextual bandits and LinUCB . . . 10
2.6 Oracles and headroom (project-specific idea) . . . 10
2.7 Related work in one table . . . 11

3 What We Built: The System **12**

3.1 Architecture . . . 12
3.2 The download engine . . . 14
3.2.1 Design rules (the invariants) . . . 15
3.2.2 The worker algorithm . . . 16
3.2.3 Straggler splitting (added after Review 2's audit) . . . 16
3.2.4 The request deadline . . . 17
3.3 The control layer . . . 17
3.3.1 The policy interface . . . 17
3.3.2 Rule-based controllers . . . 18
3.3.3 Learned controllers . . . 18
3.4 The test platform . . . 19
3.4.1 The paced test server . . . 19
3.4.2 The soft concurrency cap . . . 20

3.4.3 Network emulation, trace replay, live runner . . . 21
3.5 The simulators . . . 22
3.6 The evidence pipeline . . . 22
3.7 Testing . . . 22

4 Method and Statistics **24**
4.1 Run design: arms, blocks, pairs . . . 24
4.2 The time ratio and why it is geometric . . . 24
4.3 Confidence intervals . . . 25
4.3.1 Percentile bootstrap (primary) . . . 25
4.3.2 Paired t-interval (sensitivity check) . . . 25
4.3.3 Verdicts . . . 25
4.3.4 Workload-level intervals . . . 25
4.4 Headroom gates: measure the opportunity before building the learner . . . 26
4.5 Frozen protocols and amendments . . . 26
4.6 Independent review before freezing . . . 27
4.7 The claims checker . . . 27
4.8 Metrics . . . 27
4.9 Validity threats we controlled for . . . 27

5 Timeline, Reviews and Work Division **29**
5.1 From proposal to final results . . . 29
5.2 What each review covered . . . 30
5.2.1 Review 1: the proposal . . . 30
5.2.2 Review 2: progress (submitted 24 Oct) . . . 30
5.2.3 Review 3: final report (submitted 24 Oct) . . . 30
5.2.4 After the reviews (25 Oct): all planned experiments complete . . . 30
5.3 Objectives and deviations . . . 31
5.4 Work division . . . 32
5.4.1 Sourajyoti Basak (24BCE0774): architecture, core engine, server and network experiments . . . 33
5.4.2 Shwetank Shrijan (24BDS0141): components, monitoring, automation, baselines and heuristics . . . 33
5.4.3 Ayush Konar (24BDS0107): LinUCB, RL design, models and learned-policy evaluation 34

6 Results I: Fixed Settings Against aria2 and Other Tools **35**
6.1 Early aria2 comparisons and the granularity confound . . . 35
6.2 The D2 benchmark: 62.4 % less time (Confirmed) . . . 35
6.2.1 The mechanism . . . 36
6.2.2 Controls . . . 36
6.3 Pacing-schedule sensitivity (screen) . . . 37
6.4 Stock NGINX check (screen) . . . 37
6.5 Other segmented downloaders: axel and lftp (screen) . . . 38
6.6 Delay and loss sweep (screen) . . . 39
6.7 Goodput with the fixed engine . . . 39

7 Results II: Searching for an Adaptation Opportunity **40**
7.1 Rule-based adaptation on D2 (Track 1) . . . 40
7.2 Mechanism screens . . . 41
7.3 Recorded cellular traces: the oracle gate (Confirmed; Failed gate) . . . 41
7.4 Simulated headroom that was not real (L2–L4) . . . 42

8 Results III: The Capped Server, Adaptive Control and the Engine **44**
8.1 The live mechanism gate (Failed gate, 5 of 6 checks passed) . . . 44
8.2 Workload selection . . . 44
8.2.1 Two workloads (P1) . . . 44
8.2.2 Six-workload family W1–W6 (Descriptive) . . . 44
8.3 Within-download adaptation (P2): 23.4 % less time (Confirmed on the original engine) . . . 45
8.4 The engine ablation: why aria2 led under the cap (Confirmed) . . . 46
8.5 Online concurrency tuner (prior-work re-implementation) . . . 47
8.6 Adaptation speed . . . 48
8.7 Why does 16×32 MiB win under the cap? . . . 48

9 Results IV: Every Learned Policy **49**
9.1 LinUCB . . . 49
9.2 DQN versions v1–v6 . . . 49
9.3 Frozen v6 on the capped server (screen) . . . 50
9.4 Cap-trained DQN v1 (`rl_cap_train`) . . . 50
9.5 Joint concurrency and range-size agent (`rl_joint`) . . . 51
9.6 DQN v2: broader training, held-out scenarios inside the ranges (`rl_cap_train_v2`) . . . 51
9.7 v2 against aria2 and large ranges (cross-study, not paired) . . . 53
9.8 Model-based controller attempt (Failed gate) . . . 53

10 Results V: Real Networks and Real Servers **55**
10.1 Live-internet pilot (`live_pilot_l1`, Descriptive) . . . 55
10.2 Adaptive arms on real mirrors (`live_mirror_check`, Descriptive) . . . 56
10.2.1 What happened . . . 56
10.2.2 Why . . . 57

11 Synthesis: What It All Means **59**
11.1 Why reinforcement learning did not win . . . 59
11.2 What we claim and what we do not . . . 59
11.3 What is novel in this project . . . 60
11.4 Practical lessons . . . 62
11.5 Future work . . . 62
11.6 Publication status . . . 62

12 Limitations and Threats to Validity **63**
12.1 Incidents we disclose . . . 64

13 Doubt Clearing: Viva Questions and Model Answers **65**
13.1 The basics . . . 65
13.2 Method and statistics . . . 65
13.3 The aria2 result . . . 66

13.4 The search for an opportunity . . . 67
13.5 The capped server and the engine . . . 67
13.6 Reinforcement learning . . . 68
13.7 Real networks . . . 70
13.8 Novelty and value . . . 70
13.9 Design choices examiners like to probe . . . 71
13.10 Hostile or tricky questions . . . 72

14 Presentation and Demo Guide **73**
14.1 Suggested slide flow (about 15 minutes) . . . 73
14.2 Demo script (about 2.5 minutes plus talking) . . . 73
14.3 Five sentences everyone should be able to say . . . 75

A Glossary of Terms and Internal Labels **76**

B Numbers Cheat Sheet **78**

C Inventory of Every Study Folder **80**

D Repository Map and How to Reproduce **83**
D.1 Top-level layout . . . 83
D.2 `internal/` packages . . . 83
D.3 `cmd/` programs . . . 83
D.4 Reproducing . . . 84

E References **85**

List of Figures

1.1 The chain of studies. Blue: instrument and first benchmark; green: the constructed opportunity; red: the engine finding; purple: learned controllers; orange: real servers. 5

3.1 System architecture. One Downloader coordinates five parts; only the Fetcher talks to the network; policies act only through the controllers. 12
3.2 UML class diagram. The Downloader owns its parts; every policy implements the same interface. 15
3.3 UML state diagram of one byte range in the RangeTracker. 16
3.4 Worker loop (`internal/downloader/worker_pool.go`). 16
3.5 Extra bytes re-downloaded under the original 30 s whole-request timeout (capped server, 32 MiB ranges). 17
3.6 Reference schedule D2: 640 KiB/s, a dip to 160 KiB/s from 2 s to 4 s, then 2,500 KiB/s. A request that starts in the first 4 s keeps its slow rate. 19
3.7 The soft cap: a new request's rate and the implied total, against the number of active requests. 20
3.8 Schematic of the arrival-time rule under CAP8. 21

4.1 Ratios are averaged on the log scale because "half the time" and "twice the time" should cancel. 24

5.1 Project timeline. Dates from `docs/PROJECT_CONTEXT.md` and the git log (which begins 2026-10-11); the Review 1 date is not recorded in the repository. 29

6.1 Discovery grid: completion time for all 40 (N,S) settings on D2. 35
6.2 Data received over time in one paired run, reconstructed from per-request server logs. . . . 36
6.3 Mean completion time under the six schedules. 37
6.4 Paired ratios ours/aria2: D2 benchmark, six sweep schedules, two NGINX configurations. Sweep and NGINX cells are screens ($n = 5$). 38

7.1 Oracle headroom per trace against the 10 % bar. 42

8.1 Selector ratios per workload; the shaded band is the workload-level 95 % interval. 45
8.2 P2 completion times (10 blocks). 46
8.3 Completion time before and after the engine fix, and aria2. 46
8.4 Adaptive controllers against the best fixed setting on the corrected engine. 47
8.5 Online tuner against fixed settings and aria2. 48

9.1 Joint N + range-size agent against the best fixed setting per scenario (arrival-rule server, 10 blocks). 51
9.2 The v2 study: v1, v2 and the drop rule against the best fixed 1 MiB arm. 52
9.3 CAP8 completion times from the joint and v2 studies. 53
9.4 Selected adaptive and learned arms (primary CAP8-type comparisons) against their comparison fixed setting. Held-out and no-cap results are in the tables above. 54

10.1 Live pilot: how much slower one global fixed setting and aria2 were than the per-condition best. 56
10.2 Real mirrors: both adaptive arms above the 1.05 no-harm line on all three. 57

14.1 Terminal capture of step 1. 74
14.2 Terminal capture: the test suite passing. 75

List of Tables

2.1 External downloaders and how they segment . . . 9
2.2 Prior work we cite and how we used it . . . 11

3.1 Technology stack . . . 13
3.2 Components and responsibilities . . . 14
3.3 The 13 DQN state features of v6 and the cap-trained agents (`python/rl/state_v6.py`) 19
3.4 Server scenarios used throughout . . . 21
3.5 Verification commands . . . 22

5.1 Dated milestones . . . 29
5.2 Review 1 objectives and outcomes (updated with the post-review studies) . . . 31
5.3 Deviations from the Review 1 plan and why . . . 32
5.4 Team members and contributions (from Review 2) . . . 32

6.1 Pacing sweep, ours (16×4 MiB) / aria2 . . . 37
6.2 Ours against axel, lftp and aria2 (ratios; 95 % bootstrap) . . . 38
6.3 Goodput in MB/s (geometric mean of 10 runs; ours on the fixed engine) . . . 39

7.1 Screens that looked for a state-dependent optimum (all closed before policy work) . . . 41
7.2 Oracle headroom per trace (`testdata/final_oracle_analysis`) . . . 42

8.1 Selector across W1–W6 . . . 45
8.2 Online tuner (ratios tuner/other) . . . 47
8.3 Adaptation speed (median seconds, range over runs) . . . 48

9.1 DQN candidates (Double DQN, trained in simulation, run live in Go) . . . 49
9.2 Frozen DQN v6 on the capped and uncapped server . . . 50
9.3 v2 study results (8 blocks; ratios against the better fixed 1 MiB arm) . . . 52

10.1 Mirror check slot M2 (ratios against the better fixed arm, 16×1 MiB on all three; 95 % bootstrap) . . . 57

11.1 Claim map (from the paper; “label” is the evidence strength) . . . 59
11.1 Claim map (from the paper; “label” is the evidence strength) . . . 60

14.1 Suggested slides, figures and presenter (by area of the work division) . . . 73

How to Use This Handbook

This handbook is written for the three of us to prepare for the viva and the presentation. It tries to contain *everything*: what the project asked, what we built, how we measured, every study and its numbers, what we claim and what we do not, and the questions an examiner is likely to ask. It is deliberately long. If you are using an AI assistant to study, give it this PDF and ask it to quiz you chapter by chapter.

Reading paths

- **One hour before the viva:** Chapter 1 (the project in one page), Chapter 11 (why RL did not win, and what we claim), Appendix B (numbers cheat sheet), and skim Chapter 13 (questions).
- **To understand the project properly:** read Chapters 2–4 first (background, system, method), then the five results chapters (6–10) in order. The results chapters follow the story of the project.
- **To present:** Chapter 14 has a suggested slide flow, the demo script, and who presents what.

Conventions

- N is the number of parallel connections (requests in flight). S is the range size, the number of bytes asked for in one request.
- A **time ratio** R is the completion time of the first option divided by the second's, averaged geometrically over paired blocks. $R < 1$ means the first option was *faster*. $R = 0.845$ means it took 84.5 % of the time, i.e. 15.5 % less time.
- Intervals in square brackets, such as $[0.840, 0.850]$, are 95 % percentile bootstrap intervals over blocks unless stated otherwise.
- "Faster", "slower" and "not distinguishable" are verdicts: faster if the whole interval is below 1, slower if it is above 1, not distinguishable if it contains 1.
- **Evidence labels** (used in the paper and here): *Confirmed* = reproduced from raw runs with the stated n; *Descriptive* = a point estimate without an inferential claim; *Screen* = a small run (often 3–5 blocks) to check a direction; *Failed gate* = a criterion fixed in advance that was not met; *Not supported* / *Not established* = a claim we do *not* make.
- Scenario names: **HEALTHY** (no cap), **CAP0** (soft concurrency cap from the start), **CAP8** (cap switching on at 8 s), **D2** (the reference paced workload). They are explained in Chapter 3.

The one sentence to remember

Fixed configurations cannot adapt; on the conditions we tested, they mostly did not need to. Adaptation pays only when using more concurrency has a cost that changes over time, and even there, engine details (timeouts, straggler splitting) mattered more than the controller.

Where things are

- The paper draft: `reports/paper_draft.md`. The submitted reviews: `reports/Review2_Project_Review.pdf`, `reports/Review3_Final_Report.pdf`.
- Viva material: `reports/thursday/` (one-page summary, Q&A, demo script).
- The full living project log (about 10,000 lines): `docs/PROJECT_CONTEXT.md`.
- Each study has its own folder under `testdata/`; the later ones have `protocol.md`, a manifest, raw run records and `analysis/`.
- This handbook's source: `reports/handbook/` (`make_figs.py` draws the new figures from the committed analysis files).

1. The Project in One Page

1.1 The question

Segmented download managers such as aria2 and IDM fetch one file as many byte ranges over parallel HTTP connections. Two settings decide how they do it: N, the number of connections, and S, the range size. The usual story is: *networks change, fixed settings cannot adapt, so an adaptive or learned controller should win.* Our project asked a narrower, testable question:

Research question

When is there actually something for adaptation to exploit? That is: under which network and server conditions does changing N and S *during* a download beat a well-chosen fixed setting, and can a learned (reinforcement-learning) controller find that benefit?

The project started (Review 1) as "build an RL-based adaptive download manager". Early results showed that well-chosen fixed settings were almost always close to optimal, so the central question became *when adaptation pays*, and every learned controller had to beat the best fixed setting, not just a weak baseline.

1.2 The answer in ten findings

1. **A fixed setting beat aria2 by 62.4 %, and adaptation was not the reason.** On one paced workload (D2), fixed 16×4 MiB had $R = 0.3756$ against aria2 (30 of 30 pairs). Much of the gap comes from range granularity interacting with the server's request-start rate rule (it shrinks to 0.862 with matched granularity); a residual is unexplained. Ours was also faster than axel and lftp (0.345–0.375), in all 9 cells of a 3×3 delay–loss grid, under 5 of 6 pacing schedules (28 of 30 pairs; parity at a constant slow rate) and against stock NGINX (10 of 10 pairs).

2. **Variability is not opportunity.** Recorded cellular traces swing about 5–27$\times$ in capacity (p90/p10 over the Mahimahi set), yet a clairvoyant oracle that switches N optimally gains only 0.41 % on average; the one-sided 95 % lower bound was -0.26 % against a predeclared 10 % bar (*Failed gate*). Under the frozen counterfactual model, this bounds *any* concurrency-switching policy on these traces.

3. **Adaptation helps when concurrency has a cost that changes over time.** On a server we built, where per-request rate falls with concurrency and the cap switches on mid-download, a one-threshold causal controller beat the best of three fixed settings by 23.4 % (58.7 s vs 76.7 s, 10 of 10 blocks); aria2 was still faster (53.6 s).

4. **A workload selector matched the per-workload best on average** across six predeclared workloads ($R = 0.999$ [0.969, 1.029]); an advantage over one family-wide setting was *not established*.

5. **Engine before policy.** A 230-run ablation traced aria2's lead under the cap to two engine details in our downloader: a 30 s whole-request timeout and no straggler splitting. With both fixed, our best fixed setting matched aria2 with the cap from the start (0.995) and came within 1.6 % with the cap

from 8 s (1.016), and beat the causal controller (1.071) and the selector (1.229). A FastBioDL-style online tuner was slower than the better of two fixed 1 MiB settings (1.212, 1.220).

6. **Every early learned policy lost.** LinUCB locked onto one arm; the September DQN candidates v2, v5 and v6 took 1.757×, 2.322× and 1.456× as long as aria2 (in head-to-heads that are not comparable with each other; this early "v2" is not the capped-server v2 of finding 9); a model-based controller failed its feasibility gate (0 of 6 criteria); one environment design showed 17.6 % headroom in simulation that did not reproduce live.
7. **A DQN retrained for the capped server (v1) beat the fixed setting with its own range size where it was trained** (0.907 [0.893, 0.921] vs 16×1 MiB), stayed slower than 16×32 MiB and aria2 (1.298, 1.310), and was faster in 0 of 3 held-out scenarios outside its training ranges.
8. **A joint agent that also set the range size did not beat the best fixed setting** (1.471 under CAP8, slower in 4 of 4 scenarios), although a hand-written rule had shown large headroom (0.352) in the simulator. A control server that re-checks rates during each response removed most of the large-range advantage (50.9 s became 67.4 s).
9. **Broader training made the agent generalize partially (v2).** Faster than the best fixed 1 MiB setting in 3 of 4 held-out scenarios inside its training ranges (0.882, 0.819, 0.952), slower in 1 (1.007); faster than v1 in all 6; about 5 % slower with no cap; beaten by a simple drop rule in that rule's home scenarios. One seed, selected on validation.
10. **On real public mirrors, adapting cost time.** In a frozen no-harm check on three kernel mirrors (60 of 60 valid), both v2 (1.215–1.654) and the drop rule (1.063–1.332) were slower than the best fixed setting: no harm on 0 of 3 mirrors for both. Separately, a live pilot found one fixed setting (8×1 MiB) on average 3.7 % [1.0, 4.8] slower than each condition's best (within 5 % in 11 of 16 conditions; at most 15.3 %).

1.3 The logic of the project

Figure 1.1 shows how the studies connect. Each arrow is a question the previous result raised.

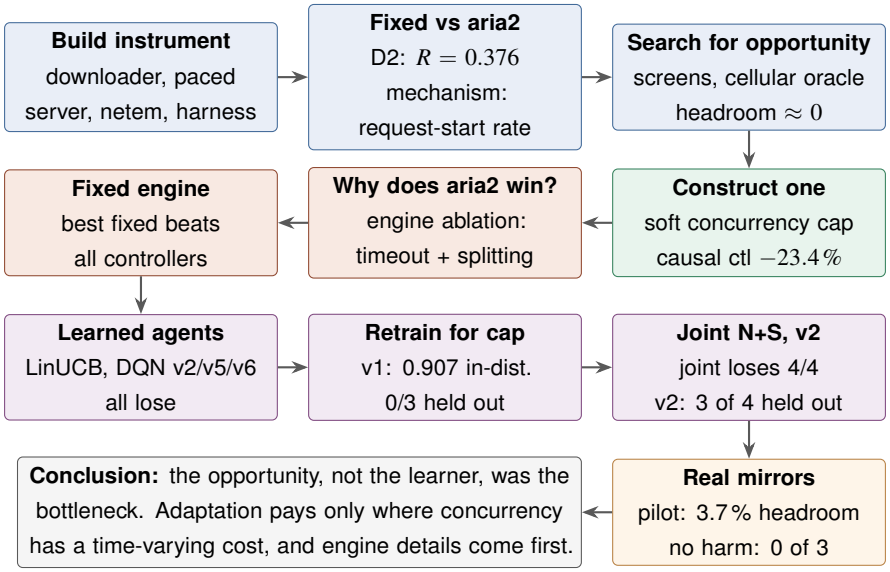


Figure 1.1: The chain of studies. Blue: instrument and first benchmark; green: the constructed opportunity; red: the engine finding; purple: learned controllers; orange: real servers.

1.4 What we built

- **Go downloader** (module `adaptivedl`): range tracker, fetcher, assembler, worker pool, N and S controllers, and one policy interface that fixed rules, LinUCB and DQN agents all plug into.
- **Controlled server**: per-response pacing, wall-clock rate schedules, fault injection, a soft concurrency cap, and (added later) an opt-in live-rate mode.
- **Harness**: `tc netem` delay and loss, Mahimahi replay of recorded cellular traces, aria2, axel, lftp, curl and NGINX baselines, SHA-256 verification of every output, and a live-pilot runner for public mirrors.
- **Learners**: LinUCB (Go, Gonum), Double DQN trained in PyTorch on simulators, run live through Go inference.
- **Method**: frozen protocols with dated amendments, oracle headroom gates computed *before* training any learner, blind simulator predictions, and a claims checker that recomputes all 220 numbers in the paper from raw runs.

1.5 What is novel, in one paragraph

We do not propose a new tuning algorithm. The novelty is in measuring *when* adaptation can help at all, with oracle headroom bounds computed before training any learner; in explaining a large gap against aria2 mechanistically (request timing, not adaptation); in showing that two engine details mattered more than any controller; in exposing the request-start rate rule behind large-range wins, with a live-rate control; and in an honest, predeclared inventory of learned controllers with held-out tests and a real-server no-harm check. Full statement: Section 11.3.

1.6 The takeaway

Takeaway (the summary's, extended with the post-review studies)

Fixed configurations cannot adapt. On the conditions we tested, they mostly did not need to. Adaptation pays when concurrency has a cost that changes over time, and even there engine details (timeouts, straggler splitting) mattered more than the controller. A learned agent can win only on the server model it was trained for, and only against fixed settings that share its range size; with broader training it generalized partially, within its training envelope; on real mirrors, adapting to noise cost time.

Be careful how you say this

Never say "RL does not work for downloading" or "our downloader is better than aria2". Say: "on the conditions we tested, the opportunity was small; where we built one, a simple rule captured it; learned agents won only against fixed settings at the same range size on the server they were trained for." Never say the work is "pre-registered" in the formal sense; say "predeclared in logged, hashed protocols" (Chapter 4).

2. Background You Need

This chapter explains the networking and learning ideas the rest of the handbook assumes. Most of it is textbook material; the parts specific to our project are marked.

2.1 HTTP range requests

HTTP lets a client ask for *part* of a resource with a `Range` header (RFC 9110, Section 14). The server answers `206 Partial Content` with a `Content-Range` header saying which bytes it sent:

```
GET /pub/linux/kernel/v3.x/linux-3.0.tar.xz HTTP/1.1
Host: cdn.kernel.org
Range: bytes=0-1048575

HTTP/1.1 206 Partial Content
Content-Range: bytes 0-1048575/63798008
Content-Length: 1048576
```

HTTP ranges are *inclusive*: `bytes=0-1048575` is exactly 1 MiB. A server that ignores `Range` replies `200 OK` with the whole file; a correct client must treat that as an error when it asked for a range. Our CLI, when not given the file size, probes support first with `Range: bytes=0-0` and requires a `206`.

Half-open vs inclusive ranges (project-specific)

Inside the downloader, ranges are stored *half-open*, $[s, e)$: bytes from s up to but not including e, length $e - s$. Conversion to HTTP happens at exactly one place:

$$[s, e) \longmapsto \texttt{Range: bytes=}s\texttt{-}(e-1). \tag{2.1}$$

For example $[0, 4194304)$ becomes `bytes=0-4194303`. Keeping one conversion point removes off-by-one errors, a classic bug source in downloaders.

2.2 Segmented (parallel) downloading

A segmented downloader splits a file of size F into ranges and fetches up to N of them at once over separate TCP connections. Two settings matter:

- N, the **concurrency**: how many requests are in flight.
- S, the **range size** (“chunk size”): bytes per request. A file of size F needs $\lceil F/S \rceil$ requests.

2.2.1 Why more connections can help

A single TCP flow's steady-state throughput is limited by its congestion window and the round-trip time. A well-known approximation (Mathis et al.) for a loss-limited flow is

$$T_{\text{flow}} \approx \frac{\text{MSS}}{\text{RTT}} \cdot \frac{\sqrt{3/2}}{\sqrt{p}}, \tag{2.2}$$

where MSS is the segment size and p the loss rate. With N flows on a path whose bottleneck is not yet full, aggregate throughput grows roughly as $N \cdot T_{\text{flow}}$ until it hits the bottleneck capacity. Servers that limit *each connection* (for example NGINX `limit_rate`) make this even more direct: N connections get N times the per-connection rate. This is why download managers open many connections.

2.2.2 Why more connections can hurt

- Once the bottleneck is full, extra connections only add overhead and queueing.
- Some servers share a fixed budget per client, or penalise many parallel connections. Our *soft concurrency cap* (Chapter 3) models this.
- Fairness: N connections take a larger share of a shared link from other users. We measured speed, not fairness.

2.2.3 Why range size matters

- **Small** S: many requests, each paying a request overhead (about one RTT before the first byte); fine-grained load balancing; quick reaction to changing conditions.
- **Large** S: few requests, low overhead, but a long tail at the end (the last big range finishes alone), and a "commitment": whatever rate a request starts with, it may keep for a long time. This last point is central to our results (the *request-start rate rule*, Chapter 3).

2.2.4 Straggler splitting

When nothing is left to allocate, an idle connection can take over half of the remaining bytes of a slow in-flight range. aria2 does this; our engine picks the in-flight range with the most unreceived bytes. Our engine originally did not, and learning that this was the main reason aria2 led under the cap was one of the project's key findings (Chapter 8).

2.3 The tools we compared against

Table 2.1: External downloaders and how they segment

Tool	How it works, and how we ran it
aria2 1.37.0	Open-source, widely used. Run with `-x16 -s16 -k1M` (up to 16 connections per server, split into 16 pieces, minimum piece 1 MiB). For a 512 MiB file it issues one unranged probe plus 15 ranges of about 32 MiB, all near the start; it splits the slowest piece when a connection is idle. Flags chosen once from a discovery sweep of its split settings (1/2/4/8/16) and never re-tuned.
axel	Splits the file into N equal parts at the start; run at 16 connections.
lftp `pget`	Same up-front split; never re-splits a slow part. Run at 16 connections.
curl	Single connection; used as a single-stream reference in the live pilot.
wget2	Planned, then removed: it fetches one file over a single connection however configured (confirmed in its source; upstream issue #195).
IDM	Not evaluated: closed source, Windows-only, cannot be scripted for hundreds of randomised runs in our Linux testbed.
NGINX 1.24.0	Not a downloader: a stock *server* with `limit_rate` and `limit_rate_after`, used to check our results on a server we did not write.

2.4 Network emulation and trace replay

- **`tc netem`** is the Linux network emulator. We apply it to the loopback interface: in most studies `delay 40ms loss 0.4%`, which applies in *both* directions, so the round trip is 80 ms (measured 80.1 ms). Early drafts wrongly said "40 ms RTT"; it is 40 ms each way.
- **Mahimahi** (`mm-link`) replays recorded packet-delivery traces, so a download sees the capacity a real cellular network had over time. We used five driving traces recorded with the Saturator tool by Winstein et al. (Sprout, NSDI 2013) and distributed with Mahimahi: Verizon EVDO, Verizon LTE, AT&T LTE, T-Mobile LTE, T-Mobile UMTS. This is *replay*, not a live mobile network.

2.5 Reinforcement learning in five minutes

2.5.1 The setting

An agent observes a **state** s_t, picks an **action** a_t, gets a **reward** r_t and a new state s_{t+1}. The goal is a **policy** $\pi(a \mid s)$ maximising the discounted return $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$ with discount $\gamma \in [0, 1)$. This is a Markov decision process (MDP).

In our agents: the state is 13 measurements of the download (Chapter 3), the actions are {none, $+1$ connection, -1 connection} (and, for the joint agent, ± 4 MiB range size), a decision is taken after each completed request, and the reward is $r_t = -\Delta t$, the negative time elapsed since the last decision. With $\gamma = 1$ the episode return is exactly minus the completion time; with $\gamma = 0.99$ (the cap-trained agents, one step per completed MiB) it approximates it. Either way, maximising reward means finishing sooner.

2.5.2 Q-learning and DQN

The action-value $Q^{\pi}(s,a)$ is the expected return from taking a in s and following π. The optimal Q^{*} satisfies the Bellman equation

$$Q^{*}(s,a) = \mathbb{E}\left[r + \gamma \max_{a'} Q^{*}(s',a')\right]. \tag{2.3}$$

A Deep Q-Network (DQN; Mnih et al., 2015) approximates Q with a neural network Q_θ and trains it on transitions (s,a,r,s') sampled from a **replay buffer**, minimising (shown here as a squared error; our implementation uses the Huber loss with gradient clipping at 10)

$$\mathscr{L}(\theta) = \mathbb{E}\left[\left(y - Q_\theta(s,a)\right)^2\right], \qquad y = r + \gamma \max_{a'} Q_{\theta^-}(s',a'), \tag{2.4}$$

where θ^- is a periodically copied **target network**. Actions are chosen ε-greedily: random with probability ε (decayed from 1.0 to 0.05), otherwise $\arg\max_a Q_\theta(s,a)$.

2.5.3 Double DQN

Plain DQN overestimates values because the same network both selects and evaluates the max. Double DQN (van Hasselt et al., 2016) decouples them:

$$y^{\mathrm{DDQN}} = r + \gamma Q_{\theta^-}\Big(s',\ \arg\max_{a'} Q_\theta(s',a')\Big). \tag{2.5}$$

All our DQN agents are Double DQN with a multilayer perceptron of two hidden layers of 64 (ReLU). The cap-trained agents (v1 and v2 of the capped server) use 3 outputs, $\gamma = 0.99$, learning rate 5×10^{-4}, replay 100,000, batch 64, ε from 1.0 to 0.05 over 150,000 steps, target sync every 1,000 steps, training starting at 2,000 steps. The joint agent has 5 outputs, $\gamma = 1.0$ and rewards scaled by 1/10. The early candidates (to v6) used the library defaults: $\gamma = 1.0$, learning rate 10^{-3}, replay 20,000, ε decay over 5,000 steps, target sync every 200, training start 500. Training runs in Python/PyTorch on a simulator; the trained weights are exported as JSON and run inside Go for live decisions.

2.5.4 Contextual bandits and LinUCB

A contextual bandit is RL without long-term consequences: each decision's reward is treated as immediate. LinUCB (Li et al., 2010) models each action's expected reward as linear in the context x, $\mathbb{E}[r \mid x,a] = x^{\top}\theta_a$, keeps $A_a = I + \sum xx^{\top}$ and $b_a = \sum r\,x$, and picks

$$a_t = \arg\max_a \left(x_t^{\top}\hat{\theta}_a + \alpha\sqrt{x_t^{\top}A_a^{-1}x_t}\right), \qquad \hat{\theta}_a = A_a^{-1}b_a. \tag{2.6}$$

The second term is an exploration bonus. In our project LinUCB had 5 actions (none, $\pm N$, $\pm S$) and 10 normalised features, and *locked onto a single action* after warm-up (Chapter 9). This is why we moved to Double DQN, which handles credit over time.

2.6 Oracles and headroom (project-specific idea)

An **oracle** is a hypothetical controller that knows the future and always picks the best setting at each moment (subject to the same switching costs). It is not a policy anyone can run. Its value is that it gives an *upper*

bound: no real adaptive policy can beat it. We define the **headroom** as the oracle's relative gain over the best fixed setting:

$$H = \frac{T_{\text{best fixed}} - T_{\text{oracle}}}{T_{\text{best fixed}}}. \tag{2.7}$$

If H is near zero, *no* learner can help, however good. Computing H *before* training any learner is one of the main methodological ideas of the project (a *headroom gate*, Chapter 4).

"Isn't an oracle unfair to adaptive methods?"

It is the opposite: it is generous to them. The oracle is the best *any* adaptive method could possibly do. If even the oracle gains under 1 %, adaptation has nothing to offer in that environment.

2.7 Related work in one table

Table 2.2: Prior work we cite and how we used it

Work	Used?	How, or why not
Balaprakash et al. 2016 (direct search for GridFTP streams)	Context	Online tuning of parallel streams on production links.
Falcon (Arifuzzaman et al., TPDS 2023)	Context	Utility T/K^N with online search for science networks.
FastBioDL (Swargo et al., 2025, arXiv)	Re-implemented	Gradient-style online concurrency tuning for genomic downloads; we wrote our own version and tested it (Chapter 8).
Liu et al. 2018 (smart data transfer node)	Reviews only	Cited in Review 1 and Review 3 as the idea behind our DQN agents, but the paper's reference log could not verify it, so the paper does not cite it.
Swargo & Arifuzzaman 2025 (heuristics + PPO)	No	Cited in the reviews; not verified in the paper's log, so not cited there. One learner (DQN) was carried through; a PPO hybrid is future work.
MDTP (Abdollah et al., 2025)	No	Multi-source chunk allocation; we are single-source.
BOLA (Spiteri et al., 2016)	Analogy	Video bitrate decisions without bandwidth prediction.
DeLoad (Liu et al., 2025)	No	Short-video preloading; a different objective.
aria2, IDM	aria2 baseline; IDM not	Dynamic segmentation (straggler splitting) is what closed aria2's lead.

Our position: a mechanism-first, controlled evaluation with explicit failed gates and a practical baseline comparison, not a new tuning algorithm.

3. What We Built: The System

The system is the *instrument* for the study. It has four parts: the downloader, the control layer, the test platform, and the evidence pipeline.

3.1 Architecture

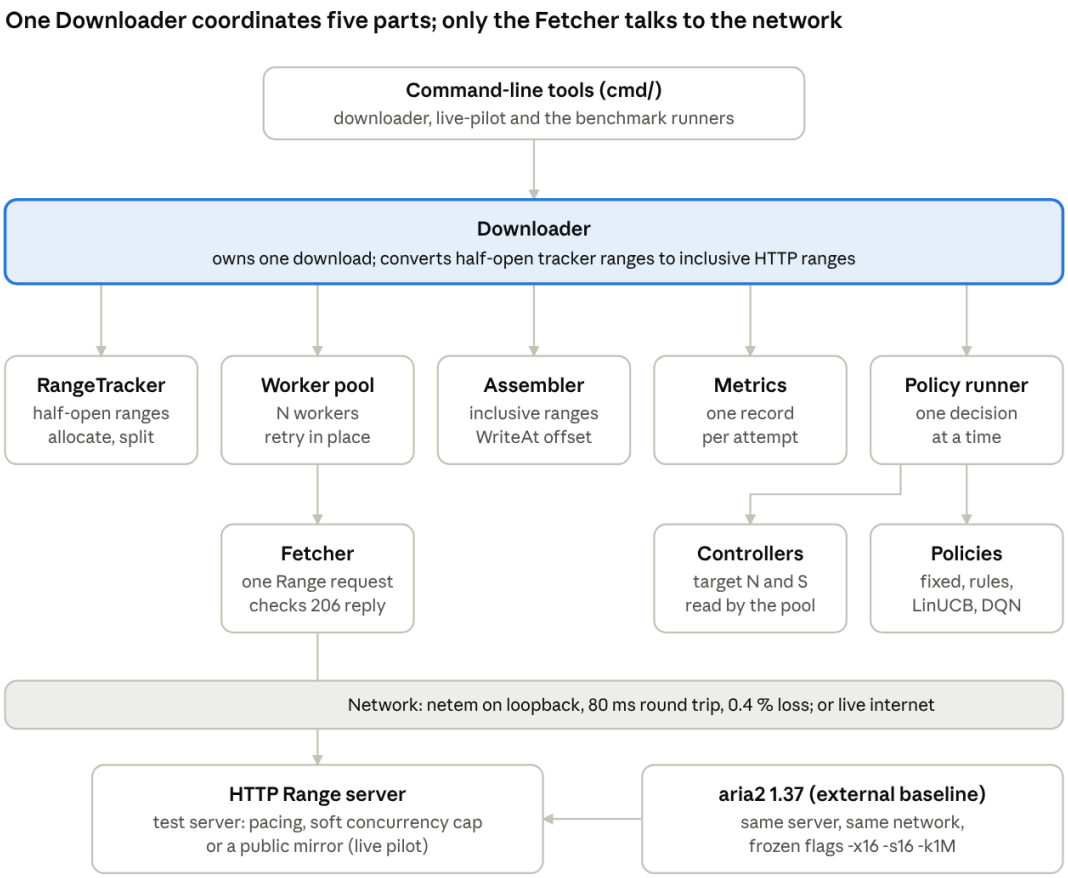


Figure 3.1: System architecture. One Downloader coordinates five parts; only the Fetcher talks to the network; policies act only through the controllers.

- **Interface layer.** Command-line programs in `cmd/` parse flags and build a `downloader.Config`. The user-facing CLI (`cmd/downloader`) and every experiment program use the same engine packages.
- **Engine layer.** The Downloader and its parts (Table 3.2). Each part has one job and its own tests.
- **Control layer.** Controllers hold the current N and S targets within bounds; policies decide changes. A policy only returns an action; it never touches the controllers directly.
- **Environment layer.** The HTTP server and the network: in experiments, our local test server plus `tc netem` or Mahimahi; in the live pilot, public mirrors.

Table 3.1: Technology stack

Part	Technology	Purpose
Downloader and test server	Go 1.27.1 (`net/http`, HTTP/1.1)	Concurrency with goroutines
Linear algebra (LinUCB)	Gonum v0.17.0	The only external Go dependency (zero transitive dependencies)
Network conditions	Linux `tc netem`	Delay and packet loss on loopback
Recorded cellular traces	Mahimahi (`mm-link`)	Replays real mobile traces
External baselines	aria2 1.37.0, axel, lftp, curl, NGINX 1.24.0	Comparison points
Training and analysis	Python 3.14, PyTorch 2.9 (CPU), NumPy	DQN training, simulators, statistics
Host	AMD Ryzen 5 5500U laptop (6 cores, 12 threads); Ubuntu 24.04 container on Linux 7.x	Client and server on the same machine

3.2 The download engine

Table 3.2: Components and responsibilities

Package	Responsibility
`rangetracker`	Splits the file into pending, in-flight and completed half-open ranges; keeps pending ranges sorted and merged; one lock makes it safe for many workers; supports splitting an in-flight range.
`fetcher`	Sends one HTTP/1.1 Range request; rejects anything that is not a `206` of the expected length, or whose `Content-Range` (when present) does not match; dedicated transport so each connection carries one request at a time; a shrinkable end for splitting.
`assembler`	Writes each finished range with `WriteAt` at its own offset, so ranges may finish in any order; the file is sized to its final length up front.
`downloader`	Worker pool, serialized policy runner, retries with unchanged bounds, opt-in straggler splitting, opt-in removal of the request deadline.
`concurrency`, `chunksize`	Bounded target controllers for N and S.
`policy`	The `Policy` interface; fixed, throughput, chunk-size, joint and online-tuner policies.
`rl`, `rl/dqn`	State/action/reward contracts, LinUCB, DQN inference in Go (e.g. `NewLiveConcurrencyPolicyV6`).
`metrics`	One record per attempt: bytes, duration, time to first byte, status; aggregates.
`experiment`	Test server (pacing, cap, faults, live-rate mode), netem control, trace replay, runners.

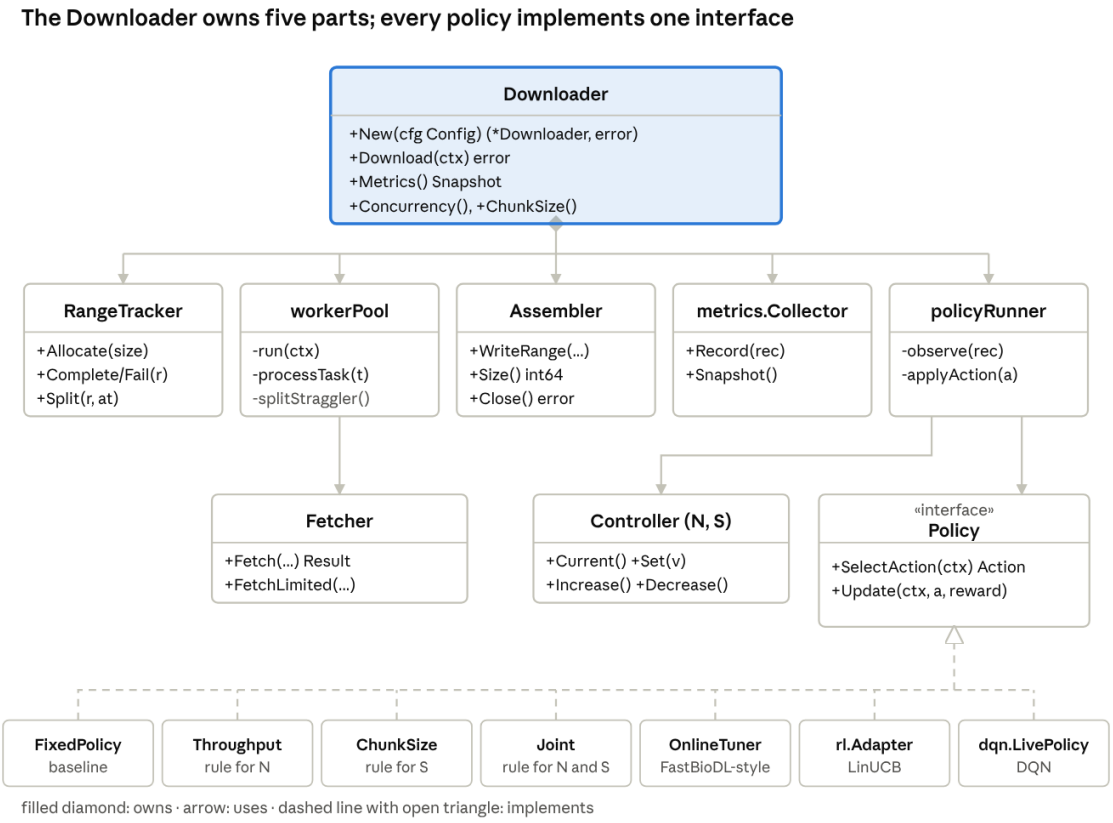


Figure 3.2: UML class diagram. The Downloader owns its parts; every policy implements the same interface.

3.2.1 Design rules (the invariants)

1. **One request fetches one contiguous byte range.**
2. **A range is never resized while in flight**, with one opt-in exception: a straggler split, which turns one range into two.
3. **A failed range is retried with the same bounds**, up to `MaxRetries`, so offsets and accounting stay stable.
4. ***N* and *S* are controlled separately.** Raising *N* starts more workers. Lowering *N* never cancels a running request: extra workers retire after their current range. Changing *S* affects only ranges allocated later.
5. **One policy decision at a time.** The policy runner holds a lock, even when many requests finish together.

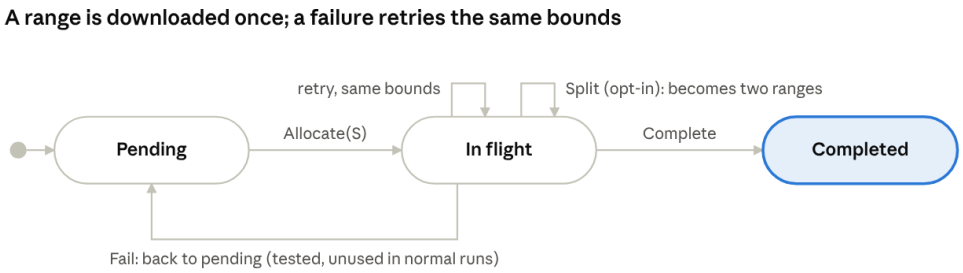


Figure 3.3: UML state diagram of one byte range in the RangeTracker.

3.2.2 The worker algorithm

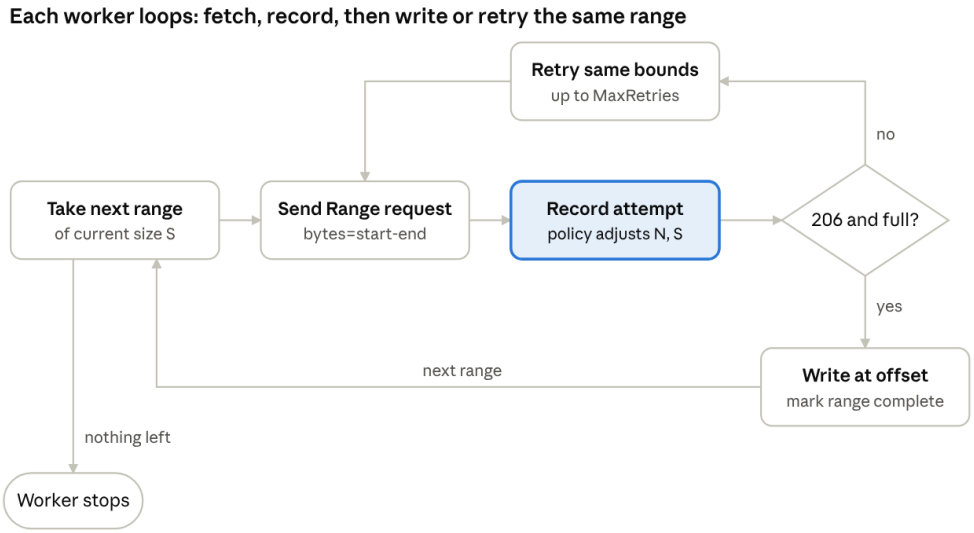


Figure 3.4: Worker loop (`internal/downloader/worker_pool.go`).

1. Read the current S and allocate the next range $[s, s+S)$. If nothing is left: with splitting on, try to split a straggler; otherwise stop.
2. Send `GET` with `Range:` `bytes=`s-$(s+S-1)$.
3. Record one metrics entry and pass it to the policy runner, which may change N or S.
4. On a full-length `206`, write at offset s and mark the range complete; otherwise retry the same range.

3.2.3 Straggler splitting (added after Review 2's audit)

When the tracker has nothing left to allocate, an idle worker picks the in-flight range with the most unreceived bytes. If b bytes of $[s, e)$ have arrived, the unreceived remainder is $r = e - (s+b)$. If $r \geq 2\,\text{MiB}$ (so each half is at least 1 MiB and worth a new request), the original request's end is moved to

$$m = s + b + \lfloor r/2 \rfloor, \tag{3.1}$$

and the idle worker fetches $[m, e)$ itself. Bytes already received stay with the original request. The client discards nothing it received; the server logs the cut-off tail as an aborted request, which is not waste. Splitting

is *off by default* (`SplitStragglers`), so every earlier result is unchanged.

3.2.4 The request deadline

The original engine put a 30 s deadline on each whole request. On the capped server, a 32 MiB range can take longer than 30 s while still making progress, so the deadline aborted it and it restarted from zero: up to 112 % of the file was downloaded again. The option `NoRequestTimeout` removes the deadline. A stalled connection would then wait until the run limit; a stall-based timeout is future work. The CLI still uses 30 s.

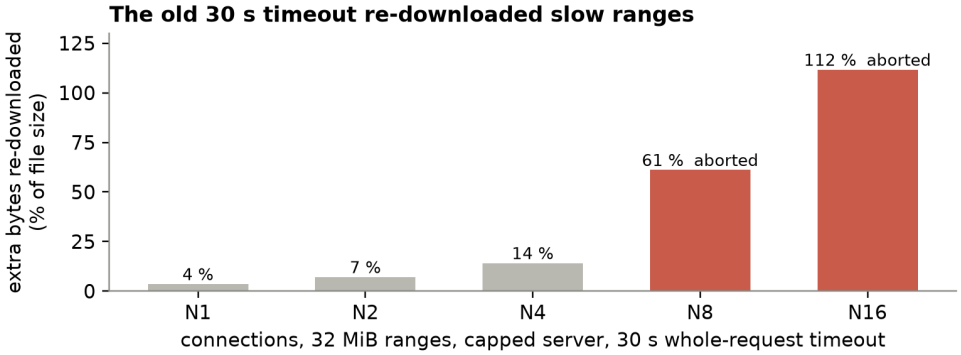


Figure 3.5: Extra bytes re-downloaded under the original 30 s whole-request timeout (capped server, 32 MiB ranges).

3.3 The control layer

3.3.1 The policy interface

Every controller, from a fixed setting to a DQN agent, implements one Go interface:

```
type Policy interface {
    // sees N, S, recent throughput, latency and request counts
    SelectAction(ctx Context) Action
    Update(ctx Context, action Action, reward float64)
}
// Actions: none, +/- concurrency, +/- chunk size (an action may carry a
   step count)
```

The `Context` holds: current N and S; windowed *per-request* throughput (bytes over request durations for the last window of attempts, roughly independent of N); windowed *aggregate* throughput (bytes of the last window of attempts over their wall-clock span, sensitive to N); last request duration; mean time to first byte; and success, failure, retry and attempt counters.

Per-request vs aggregate throughput

Per-request throughput tells you how fast *one* request goes: it drops when a cap starts, so it is what the drop rule watches. Aggregate throughput tells you how fast the *whole download* goes. Over a window of 4 attempts the aggregate figure barely changes above $N = 4$; that mismatch is one reason the frozen v6 agent could hardly "see" extra connections, and why the cap-trained agents use a window of 16.

3.3.2 Rule-based controllers

- **Fixed:** constant (N, S).
- **Throughput rule:** compares recent per-request throughput T_{now} (over a window of recent attempts: 4 in Track 1; the engine falls back to 8 if none is set) with the value at the previous decision:

$$\Delta = \frac{T_{\text{now}} - T_{\text{prev}}}{T_{\text{prev}}}, \qquad \text{action} = \begin{cases} \text{increase } N \text{ (or } S\text{)} & \Delta \geq 0.10 \\ \text{decrease } N \text{ (or } S\text{)} & \Delta \leq -0.10 \\ \text{none} & \text{otherwise.} \end{cases} \tag{3.2}$$

  The joint version alternates between N and S.
- **Causal drop controller ("drop rule").** Tracks the running maximum of the median of the last k per-request throughputs; the first time the median falls below half that maximum, it switches *once* to a preset target and stays. On the capped-server studies the target was $(4, 4\,\text{MiB})$ joint, $(4, 1\,\text{MiB})$ N-only or $(16, 4\,\text{MiB})$ S-only; the later N-only drop rule uses $k = 3$, ratio 0.5, capped $N = 4$ ("C=8-tuned", because 4 suits a cap of 8 MiB/s).
- **Workload selector.** Starts at 16×1 MiB, watches per-request throughput for the first 20 samples, classifies the server as capped or not by a predeclared threshold, then moves once to that class's predeclared target.
- **Online tuner (FastBioDL-style re-implementation).** Every $\Delta t = 2$ s it measures aggregate throughput T (bytes completed in the window over Δt) and scores $U = T/K^N$ with $K = 1$, so $U = T$. If U is at least 5 % worse than the previous window it reverses direction and halves its step; otherwise it keeps going. Starts at $N = 4$ with step 4, within $[1, 16]$; S untouched.

3.3.3 Learned controllers

- **LinUCB**: 5 actions (none, $\pm N$, $\pm S$); 10 normalised features (N, S, per-request and aggregate throughput, request duration, bytes, success/failure/retry/attempt counts); reward combining aggregate throughput and its change minus penalties for duration, retries, failures and actions.
- **Double DQN** (v1–v6, then the cap-trained v1, joint, v2): every version has a 13-feature state; Table 3.3 is the v6 / cap-trained layout (v1–v4 and the joint agent use `state.py`, with range-size features instead of the last two; v5 has its own layout). v2 and v3 acted on both N and S; from v4 on, N only, with S fixed at 1 MiB (v2 fragmented ranges into 32–128 KiB pieces; v3's simulator rewarded large ranges the real server did not). The joint agent of the later study added ± 4 MiB range actions (1–33 MiB).

Table 3.3: The 13 DQN state features of v6 and the cap-trained agents (`python/rl/state_v6.py`)

#	Feature	Meaning
1	`concurrency_norm`	current N, normalised
2	`remaining_bytes_frac`	remaining bytes / total
3	`aggregate_throughput_norm`	current aggregate throughput / scale
4	`aggregate_throughput_ewma_norm`	EWMA of aggregate throughput
5	`aggregate_throughput_slope`	change since previous observation
6	`aggregate_throughput_volatility`	EWMA of $\lvert\Delta\rvert$
7	`request_duration_norm`	request duration / scale
8	`request_duration_slope`	change in duration
9	`recent_failure_rate`	failures / attempts in the window
10	`recent_retry_rate`	retries / attempts in the window
11	`since_concurrency_change_norm`	time since the last effective N change
12	`recent_completed_norm`	completed requests in the window / window size
13	`throughput_improvement`	signed relative throughput change, in $[-1, 1]$

3.4 The test platform

3.4.1 The paced test server

Our own Go server serves a 512 MiB object with deterministic content (verifiable byte for byte), honours Range requests, and paces every response. Three rules define it:

1. **Request-start rate rule:** each response is sent at the server's rate *at the moment the request arrives*, and keeps that rate until it finishes.
2. A schedule changes the rate at fixed wall-clock times; a change affects only requests that start afterwards.
3. Without the cap, rates are per response with no shared limit, so more connections add bandwidth.

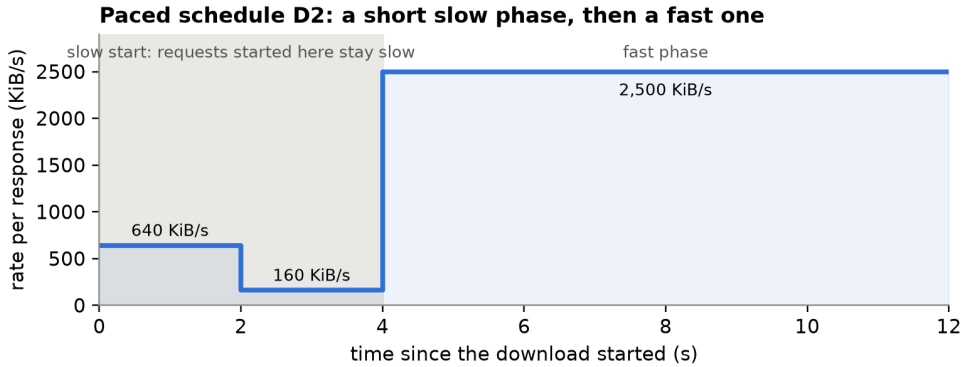


Figure 3.6: Reference schedule D2: 640 KiB/s, a dip to 160 KiB/s from 2 s to 4 s, then 2,500 KiB/s. A request that starts in the first 4 s keeps its slow rate.

3.4.2 The soft concurrency cap

To create a case where the best setting changes, we added a cap. When a request arrives, let k be the number of active requests *including* the new one; the new request gets

$$\text{rate} = \min\left(\text{base},\ \frac{C}{k\,(1+\gamma(k-1))}\right), \qquad C = 8\ \text{MiB/s},\ \gamma = 0.05. \tag{3.3}$$

If all k requests ran at that rate the total would be $C/(1+\gamma(k-1))$, which *falls* as k grows: about 7.3 MiB/s at $k=3$ but 4.6 MiB/s at $k=16$ (Figure 3.7). It imitates a server that shares a budget across a client's connections and penalises many parallel ones. C and γ were chosen during design from an exploratory screen of 27 (schedule, cap) pairs, of which exactly one passed the selection rule (the later six-workload family was chosen from 72 analytic candidates); this is a constructed environment and we always say so.

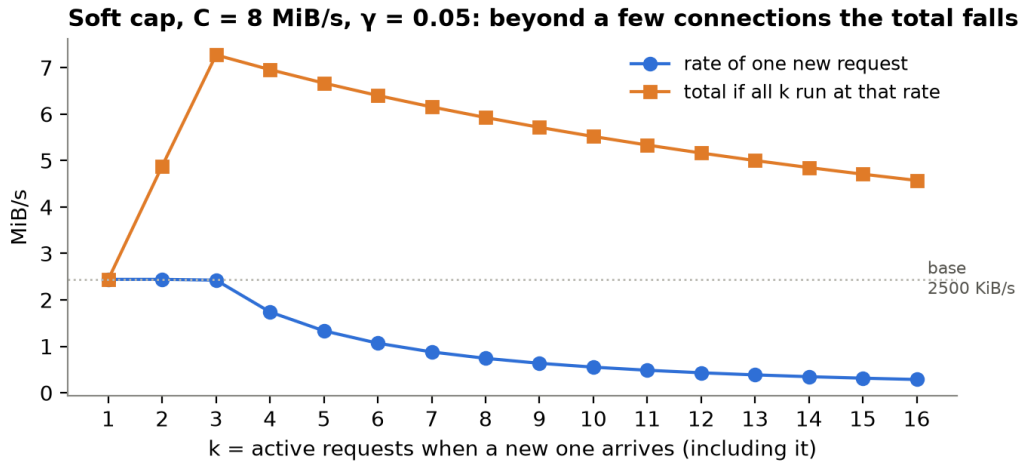


Figure 3.7: The soft cap: a new request's rate and the implied total, against the number of active requests.

The request-start rule drives two headline results

Because a request keeps the rate it had on arrival: (1) on D2, aria2's large pieces all start during the slow phase and stay slow, while our small ranges keep starting after the speed-up (the 62.4 % result); (2) under CAP8, 16×32 MiB starts all its ranges *before* the cap and keeps the fast rate for their whole length, which a 1 MiB agent cannot copy (Figure 3.8). The **live-rate** server mode, added for the joint study, re-evaluates each response's rate during the response from the current k. On it, fixed 16×32 MiB under the cap slowed from 50.9 s to 67.4 s.

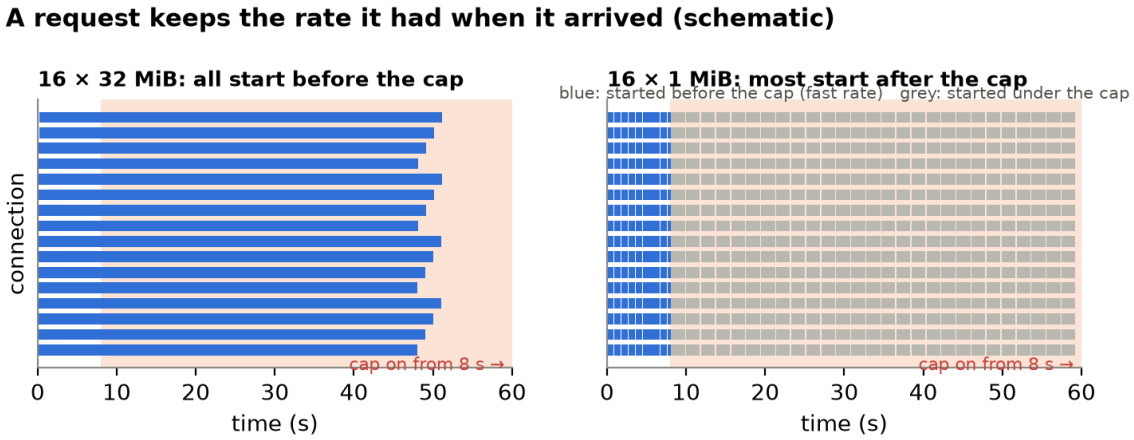


Figure 3.8: Schematic of the arrival-time rule under CAP8.

Table 3.4: Server scenarios used throughout

Scenario	Server behaviour
D2 (paced)	640 KiB/s; 160 KiB/s from 2 s; 2,500 KiB/s from 4 s; per response
HEALTHY / "no cap" / t1	640 KiB/s rising to 2,500 KiB/s from 1 s; no cap
CAP0	HEALTHY plus the soft cap from the start
CAP8	HEALTHY with the cap switching on at 8 s
Held-out (v1 study)	cap $C = 16$ MiB/s; 80 ms each way; the three-phase d2 schedule
Held-out (v2 study)	H_C12 ($C = 12$ from 6 s); H_RTT110 (110 ms each way, cap from 12 s); H_DIP (cap from 6 s + a dip 30–40 s); H_C20 ($C = 20$ from 4 s, 20 ms each way)
Pacing sweep S0–S5	Six variants of D2 (Table 6.1)
Stock NGINX N1, N2	`limit_rate 640k`; `limit_rate_after 1m` + `limit_rate 640k`

The server also injects faults (error statuses, failing ranges, delayed headers), used in failure-handling tests.

3.4.3 Network emulation, trace replay, live runner

- **netem**: applied and removed by the harness; each run records whether shaping actually applied (silent netem failures were an early engineering problem); runs where it did not are excluded and reported.
- **Mahimahi**: `mm-link` replays five cellular traces; `cmd/mm-replay` and `cmd/oracle` drive it.
- **Live runner** (`cmd/live-pilot`): reads a manifest of servers, files and checksums; runs fixed cells, aria2, curl and (later) policy arms (`dqn6`, `drop`) in seeded random order; enforces etiquette (identifying User-Agent, ≤ 16 connections per server, 5 s gap between runs, abort a server after any `429`/`503`); writes one JSONL record per run with a 100 ms throughput timeline.

3.5 The simulators

DQN agents are trained in simulation (thousands of episodes would take months live). The simulators are event-driven models of the download:

- `sim_env_v5`, `sim_env_v6`: earlier simulators. v6 had *no* arrival-time cap, and its aggregate-throughput observable did not match the live runner's, which is why the frozen v6 agent did badly on the capped server.
- `sim_env_cap.py`: the capped-server simulator. It copies the live runner's measurements (window 16), models the cap at request arrival, and draws per-request server excess and client overhead from live request logs (completion times were *not* used for calibration). For v2 it interpolates calibration across four delays (20, 40, 80, 120 ms) and four rate classes.
- Each simulator was checked *blind*: predictions for fixed settings were written down before the live runs (Chapter 4).

3.6 The evidence pipeline

- Every run writes a JSON record (fields vary by harness): typically `run_id`, system, config, `completion_ms`, output SHA-256 and `content_verified`, and per-request logs; the main benchmark records also hold coverage checks and `netem_status`.
- The later studies (from the pacing sweep on) have a `protocol.md` frozen before any run, a manifest with the run list and seed, raw run records and an `analysis/` folder; earlier folders vary in layout.
- `tools/claims_check.py` recomputes every number in the paper from the raw records (220 of 220 pass) and scans the paper for unverified numeric tokens (0). Its self-test (9 of 9) shows it rejects deliberately wrong numbers.

3.7 Testing

The Review 2 count was 549 automated Go test functions in 29 packages; more were added since (live-pilot policy arms, live-rate server, cap/joint DQN parity tests). No test needs internet access: each starts its own local server. Tests needing root, `tc` or Mahimahi skip themselves when unavailable.

Table 3.5: Verification commands

Command	Result
`go build ./..., go vet ./..., gofmt -l`	clean
`go test ./...`	all pass
`go test -race ./...`	no data races
`go test -count=5 ./...`	pass (one load-sensitive trace test failed once under heavy load, passed 11 reruns)
`pytest python/tests`	pass
`python3 tools/claims_check.py`	220 of 220 numbers; 0 unverified tokens

Parity tests: a Go test checks that Go inference of each exported DQN model matches Python inference on fixtures (`internal/rl/dqn/cap_candidate_parity_test.go`); it must pass before any live run of that model.

4. Method and Statistics

The project's credibility rests on its method as much as its results. This chapter explains how every comparison was designed, measured and judged, with the formulas.

4.1 Run design: arms, blocks, pairs

- An **arm** is one option in an experiment: a fixed setting, a controller, a learned agent, or an external tool.
- A **block** holds one run of each arm, in a random order fixed by a seed (for example `20261101` in the mirror check). In the loopback studies every run uses a fresh server, a fresh output file and a fresh netem setup; the live-internet studies use public servers and no netem.
- Arms are compared **paired by block**: the ratio is computed within each block, so slow drift of the machine or the network affects both arms in that block equally.
- A run is **valid** only if it exits cleanly, reports no error, and its output matches the source byte for byte (SHA-256). **Invalid runs are kept and reported**, never silently re-run or dropped.

4.2 The time ratio and why it is geometric

For blocks $i = 1, \ldots, n$ with completion times t_i^A and t_i^B, the time ratio of A to B is

$$R = \exp\left(\frac{1}{n}\sum_{i=1}^{n} \ln \frac{t_i^A}{t_i^B}\right) = \left(\prod_{i=1}^{n} \frac{t_i^A}{t_i^B}\right)^{1/n}. \tag{4.1}$$

$R < 1$ means A was faster. The percentage reduction in time is $(1 - R) \times 100\,\%$; for example $R = 0.3756$ means 62.4 % less time.

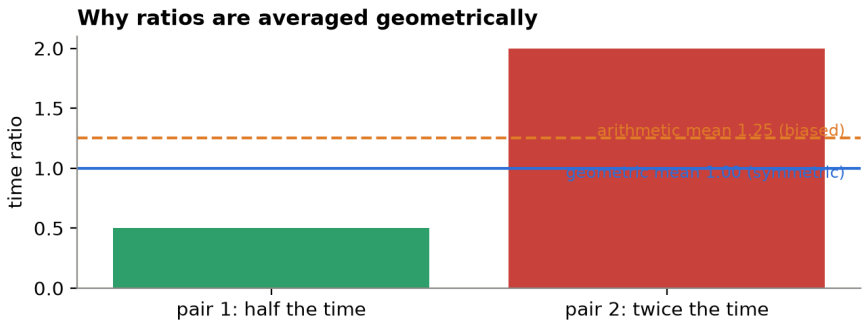


Figure 4.1: Ratios are averaged on the log scale because "half the time" and "twice the time" should cancel.

"Why not just compare mean times?"

Times differ by scenario (18 s here, 115 s there), so an arithmetic mean is dominated by the slow scenarios. Ratios are scale-free, and the geometric mean treats 0.5 and 2 symmetrically: $\sqrt{0.5 \times 2} = 1$, whereas the arithmetic mean 1.25 would wrongly suggest A is slower. Taking logs also makes the

per-block values roughly symmetric, which the t-interval needs.

4.3 Confidence intervals

4.3.1 Percentile bootstrap (primary)

Let $d_i = \ln(t_i^A / t_i^B)$. Draw n blocks with replacement from $\{d_1, \ldots, d_n\}$, compute their mean $\bar{d}^*$, repeat $B = 10{,}000$ times (the seed is fixed per study: 1 in the v2 study and the mirror check, 20261024 in the engine ablation, 20261025 in the tuner study), and take the 2.5th and 97.5th percentiles of $\exp(\bar{d}^*)$:

$$\mathrm{CI}_{95} = \left[\exp\left(q_{0.025}(\bar{d}^*)\right),\ \exp\left(q_{0.975}(\bar{d}^*)\right) \right]. \tag{4.2}$$

The bootstrap makes no normality assumption. (A few older headline intervals, e.g. the D2 benchmark's [0.3726, 0.3787], are t-intervals recomputed by the claims checker.)

4.3.2 Paired t-interval (sensitivity check)

With $\bar{d}$ the mean and s_d the standard deviation of the d_i,

$$\mathrm{CI}_{95}^{t} = \exp\left(\bar{d} \pm t_{0.975,\,n-1} \frac{s_d}{\sqrt{n}} \right). \tag{4.3}$$

In the v2 study and the mirror check, both intervals were computed and **the less favourable verdict was used**. Example: H_C20 had bootstrap $[1.000, 1.014]$ ("slower") and t-interval $[0.998, 1.016]$ ("not distinguishable"); the reported verdict is "slower".

"Your intervals are tiny (0.840 to 0.850). Is that suspicious?"

No, but it must be read correctly. The emulated server is nearly deterministic, so block-to-block noise is small. The intervals quantify *testbed* noise, not how results would vary across real servers or networks. We say this in the paper. Real-network intervals (mirror check) are wide: $[0.873, 2.849]$.

4.3.3 Verdicts

$$\text{verdict} = \begin{cases} \text{faster} & \text{upper bound} < 1, \\ \text{slower} & \text{lower bound} > 1, \\ \text{not distinguishable} & \text{otherwise.} \end{cases} \tag{4.4}$$

"Not distinguishable" is not "equal": it is not an equivalence test. The selector's 0.999 $[0.969, 1.029]$ means we cannot tell it apart from the per-workload best, not that it is proven equal.

4.3.4 Workload-level intervals

When a claim is about a *family* of workloads (the six-workload selector study), the unit of generalisation is the workload, not the block. We then also compute a t-interval over the six workload means (df $= 5$). It is much wider and is the primary interval for family-level claims: the selector's 0.770 against a single family-wide setting had a workload-level interval of $[0.455, 1.304]$, so that advantage is *not established*.

4.4 Headroom gates: measure the opportunity before building the learner

Before training a learner in a new environment, we compute the oracle headroom H (Chapter 2) and compare it with a bar fixed in advance. For the cellular traces, one trace is one observation ($n = 5$, df $= 4$), and the gate requires the one-sided 95 % lower bound

$$\mathrm{LCB}_{95} = \bar{H} - t_{0.95,4}\,\frac{s_H}{\sqrt{5}} \;\geq\; 0.10. \tag{4.5}$$

With $\bar{H} = 0.00406$, $s_H = 0.00700$, $t_{0.95,4} = 2.132$: $\mathrm{LCB}_{95} = 0.00406 - 2.132 \times 0.00700/\sqrt{5} = -0.0026$. **Gate failed.**

For the v2 study, each held-out scenario had to show at least 5 % *simulator* headroom between the best fixed N and a $16 \to n$ oracle, or be dropped (not replaced): a held-out scenario where nothing can be gained cannot test generalization. All four passed, at 9.9–17.4 %.

Why headroom gates matter

A learner that "fails" where $H \approx 0$ tells you nothing about the learner. A gate separates "the environment offers nothing" from "the learner could not find it". This is how we can say the cellular result holds for *any* concurrency-switching policy under the frozen trace model, not just ours.

4.5 Frozen protocols and amendments

Every study after the early era follows the same discipline:

1. **Protocol first.** A `protocol.md` states the question, arms, scenarios, blocks, seeds, validity rules, the analysis, the decision rules and the exact sentences that will be reported for each outcome. It is committed (hashed) *before* any run.
2. **Amendments, never edits.** Changes go at the end, dated (A1, A2, . . .), with the reason and whether any data existed when they were written. Example: the mirror check's A2 aborted slot M1 after a client-side outage and added a *link gate* before slot M2.
3. **Blind predictions.** Simulator predictions for live arms are written down before the live runs ("parity" checks), so the simulator's accuracy is tested, not assumed.
4. **Validation-only selection.** Among trained seeds, the candidate is chosen on validation scenarios only, never on test or held-out scenarios. Seed variance is disclosed (v2: the other four seeds were mostly at or above the best fixed time on held-out scenarios).
5. **Stop rules.** Each protocol says when to stop: e.g. if the simulator fails its blind check, no live comparison is run.
6. **Failed gates are reported as failed**, and the criterion is never relaxed after seeing results (the live mechanism gate's S-margin was 3.74 % against a 5 % bar and stayed failed).

"Pre-registered"?

Protocols were written to logged documents with recorded SHA-256 hashes before their runs, but they were not deposited with an independent timestamping service; the repository history begins with

one import commit (2026-10-11); and one manifest (W1–W6) was amended twice after design output had been seen, with the first version overwritten. So say **"predeclared in logged protocols"**, not "pre-registered" (`docs/PREREGISTRATION_NOTE.md`).

4.6 Independent review before freezing

For the later studies an independent adversarial review (a separate AI reviewer instructed to attack the design) was run on the draft protocol. For v2 the protocol records that the draft was revised once before freezing, on held-out headroom, the drop rule's tuning (now labelled "C=8-tuned", with its simulator predictions recorded), wording coverage, the number of blocks (8, with the t-interval check) and a missing stop rule.

4.7 The claims checker

`tools/claims_check.py` recomputes every number in the paper from the raw run files:

- 220 of 220 checked numbers pass; the paper scan finds 0 numeric tokens without a source.
- Each number has a *text anchor* (the sentence it appears in), so a coincidental match elsewhere cannot hide an edit (this was found and fixed when a self-test showed 0.716 passing by coincidence).
- The self-test (9 of 9) feeds in deliberately wrong numbers and checks the checker rejects them.
- It caught real errors in our own frozen scripts: a fixed t value of 2.045 used for every n (correct only for $n = 30$), and a normal value 1.96 in one analysis. The paper uses corrected values; frozen artifacts were left untouched.

4.8 Metrics

- **Completion time**: wall clock from start to verified file. Primary metric.
- **Goodput**: file size / completion time (MB/s, 10^6 bytes).
- **Transfer efficiency**: useful bytes / all bytes received. 100 % on the fixed engine in every run; as low as 47 % (112 % extra bytes) in one aborted run of the earlier capped grid ($N = 16$, 32 MiB ranges, original engine).
- **Reliability**: failed attempts, retries, wasted bytes, `429`/`503` counts.
- **Adaptation speed**: time from an event (cap onset) to the controller's first move, from decision logs.

4.9 Validity threats we controlled for

- **Order effects**: seeded random order within each block.
- **Machine load**: timing-sensitive runs were not meant to share the machine with heavy jobs. One exception is disclosed: v2 Stage A (the live blind check) ran with a host load average of 12–19 from outside the container (amendment A2); the load was removed before training and the Stage C/D live runs.

- **Silent netem failure**: recorded per run.
- **Leftover shaping**: harness removes netem; `tc qdisc show dev lo` must show `noqueue`.
- **Content errors**: SHA-256 of every output.
- **Analysis bugs**: the claims checker and its self-test.

5. Timeline, Reviews and Work Division

5.1 From proposal to final results

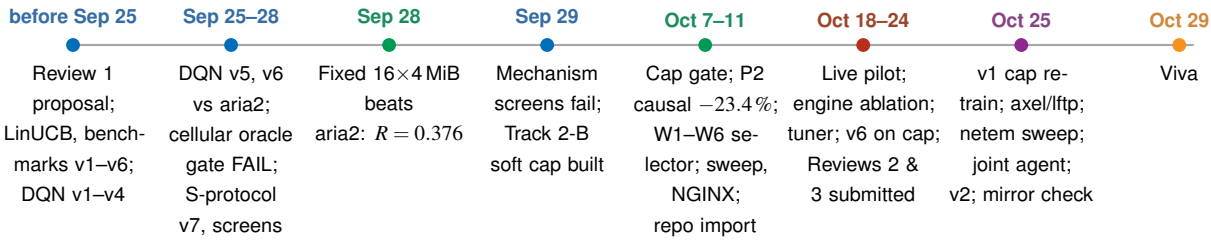


Figure 5.1: Project timeline. Dates from `docs/PROJECT_CONTEXT.md` and the git log (which begins 2026-10-11); the Review 1 date is not recorded in the repository.

Table 5.1: Dated milestones

Date (2026)	Milestone
not recorded	Review 1 proposal ("Assessment 8", a seven-week plan): adaptive HTTP download manager with joint N/S adaptation using RL. Early LinUCB calibration and benchmarks v1–v6; DQN candidates v1–v4.
Sep 25	Mobile traces retrieved; seed date for the v5/v6 evaluations; DQN v5 and v6 head-to-heads, long-duration diagnostic, environment designs L2–L4, live probes T1/T2, mobile chain M1–M3.
Sep 26–27	Mahimahi replay diagnostics; Stage 0 epoch pilot; oracle preregistrations v2–v4; tau (switch-cost) campaign.
Sep 28	Cellular oracle gate **fails** (LCB −0.26 %); S-protocol v7 frozen and Stage 0S feasibility GO; S and N mechanism screens fail; **fixed** 16×4 **MiB beats aria2 by 62.4 %** (30 pairs).
Sep 29	Phase-pacing, queueing and correlated-loss screens fail; Track 1 (heuristic joint N+S on D2: 2.32× slower than fixed); Track 2 pre-screen stopped; Track 2-B soft cap implemented; team status note.
Oct 7	Track 2-B live fixed-grid gate: fails one of six checks (S-margin 3.74 % vs 5 %).
Oct 8–9	P1 selector and P2 within-download controller ($R = 0.766$, −23.4 %); S-only ablation; W1–W6 design and live comparison (selector 0.999).
Oct 11	Repository import commit; verification audit; pacing sweep and NGINX check; claims checker 113 of 113.
Oct 12	`paper-rc1`; model-based controller attempt stopped at its gate (0 of 6).
Oct 18–20	Live-internet pilot on four public mirrors (protocol frozen Oct 18).
Oct 24	Pilot analysed; study audit finds the engine flaws; split ablation (230 runs); online tuner (64); frozen v6 on the capped server (50); **Review 2 and Review 3 submitted**.
Oct 25	Cap-trained DQN v1 (Stages A–D); axel/lftp baselines (80); delay×loss sweep (108); joint N+S agent (150 + 40); DQN v2 (24 + 8 + 64 + 160); real-mirror check (M1 aborted, M2 60 of 60); paper and checker to 220 of 220.
Oct 29	Viva (Thursday).

5.2 What each review covered

5.2.1 Review 1: the proposal

Title: *Adaptive HTTP File Download Manager with Joint Concurrency and Chunk-Size Adaptation using Reinforcement Learning*. Its own words matter in the viva: it positioned the project "as an engineering and experimental contribution rather than a claim of inventing adaptive segmented downloading", and promised to *investigate* "whether jointly selecting (N, S) provides a measurable advantage". It explicitly said: "The evaluation does not depend exclusively on successful RL convergence".

Proposed components: Network Monitor, Context Builder, Adaptive Decision Engine/Policy, Concurrency Manager, Range Manager, Request Scheduler, HTTP Fetcher + Writer, Test Server/Network Emulator. Seven-week plan: (1) core stabilization and test environment; (2) network monitor and policy foundation; (3) runtime adaptive control; (4) heuristics and LinUCB; (5) RL integration; (6) experiments; (7) analysis, paper, demo. Expected comparisons: fixed vs heuristic; N-only vs S-only vs joint; LinUCB vs RL; aria2 and IDM "where reproducible experimental access is available"; stable vs variable bandwidth, latency/loss, throttling, background traffic, combined conditions.

5.2.2 Review 2: progress (submitted 24 Oct)

System architecture, detailed design, algorithm, UML and DFDs, modules, prototype, the first benchmark (62.4 % vs aria2), the pacing sweep and NGINX check, 549 test functions, and the individual-contribution table.

5.2.3 Review 3: final report (submitted 24 Oct)

Complete system, straggler splitting, online tuner, live demo, test results, goodput and reliability, the engine ablation, "does adaptation help", adaptation speed, the tuner, other searches, real networks (live pilot, cellular traces), learned policies incl. the frozen v6 on the capped server, limitations, comparison with aria2, objectives and deviations, innovation, future work, and the publication plan (IEEE Access).

5.2.4 After the reviews (25 Oct): all planned experiments complete

Review 3's "future enhancement" list had seven items. The first three were done, plus two extra studies; the rest (broader live study, HTTP/2 and multiple sources, splitting by default with a stall-based timeout, PPO and IDM) remain future work:

1. **An agent trained for the capped server** → cap-trained DQN v1 (`rl_cap_train`), then broader-trained v2 (`rl_cap_train_v2`).
2. **More baselines in the same testbed** → axel and lftp (`baselines_ext`); wget2 shown unable to download one file in parallel.
3. **Joint learning on harder environments** → joint N + range-size agent with a live-rate control server (`rl_joint`).
4. **Extra: delay and loss sweep** → 3×3 grid (`netem_sweep`).
5. **Extra: adaptive agents on real servers** → no-harm check on three mirrors (`live_mirror_check`).

5.3 Objectives and deviations

Table 5.2: Review 1 objectives and outcomes (updated with the post-review studies)

Objective (Review 1)	**Status**	**Outcome**
Correct segmented downloader, one range per request	Achieved	Byte-identical output; probe and SHA-256 check in the CLI
Controlled test server with throttling and faults	Achieved	Pacing, soft cap, live-rate mode, fault injection
Network monitor over rolling windows	Achieved	Metrics + rolling-window state: per-request and aggregate throughput, TTFB, failures, retries
Policy interface: fixed, heuristic, LinUCB, RL	Achieved	One interface runs all of them
Joint adaptation of N and S	Achieved	Rule-based, causal, LinUCB, DQN v2/v3, and the joint N+S agent; none beat the best fixed setting
Evaluate under stable, variable, lossy, throttled, background-traffic conditions	Achieved	Rate schedules; netem (incl. a 3×3 delay–loss grid); cellular traces; live internet; capped servers; `iperf3` background traffic
Compare on time, goodput, reliability, adaptation speed	Achieved	All four reported
N-only, S-only, joint ablation	Achieved	CAP8: N-only 0.808, joint 0.766, S-only 1.070 of fixed
LinUCB vs RL	Partly	Both evaluated, not in one predeclared head-to-head
aria2 and IDM	aria2 yes; IDM no	aria2 throughout; axel, lftp added; IDM not reproducible on Linux

Table 5.3: Deviations from the Review 1 plan and why

Deviation	Reason
Framing: from "build an RL download manager" to "when does adaptation help?"	Early results showed well-chosen fixed settings near-optimal; the system, scope and policies stayed as planned.
Final DQN learns N only ($S = 1$ MiB)	v2 fragmented ranges into 32–128 KiB pieces; v3's simulator rewarded large ranges the server did not. (The later joint agent revisited S and lost.)
No PPO or hybrid agent	After LinUCB locked onto one action, Double DQN was carried through to live evaluation; PPO is future work.
Decisions after each completed request, not every fixed interval T	The live runner decides per completed attempt; the simulator was aligned after a v5 mismatch.
In-flight ranges were not split (until the audit)	Kept for correctness; the ablation showed this was the main reason aria2 led under the cap, so splitting was added as an opt-in.
No resume from disk	Outside the evaluated scope.
Cobra, slog, gRPC, `x/time/rate` not used	The standard library sufficed; DQN runs inside Go from exported weights, so no RPC; the server needed its own rate model.
FastBioDL evaluated as a re-implementation	Its idea was re-implemented and tested under a frozen protocol.

5.4 Work division

The work division below is the "Individual Contribution" table from the submitted Review 2 report, which states that "responsibilities follow the division of work set out in the Review 1 proposal". (The Review 1 proposal file in the repository has no per-member table, so Review 2's table is the record.) Each member's subsection then lists the parts of this handbook that fall in that area, as a study guide: *these are the topics an examiner is most likely to ask that member about*.

Table 5.4: Team members and contributions (from Review 2)

Member	Reg. no.	Contribution
Sourajyoti Basak	24BCE0774	Downloader architecture and core implementation; adaptive concurrency and range management; test server and Linux network experiments; system integration and validation
Shwetank Shrijan	24BDS0141	Downloader development and testing of assigned components; monitoring and experiment automation; baseline and heuristic evaluation
Ayush Konar	24BDS0107	LinUCB; RL state, action and reward design; Python/PyTorch models; integration of the learned policy with the Go downloader; ML/RL evaluation

5.4.1 Sourajyoti Basak (24BCE0774): architecture, core engine, server and network experiments

Stated work: downloader architecture and core implementation; adaptive concurrency and range management; test server and Linux network experiments; system integration and validation.

Be ready to explain:

- The architecture and every engine component (Table 3.2); the five invariants; half-open ranges and the single conversion point; retries with unchanged bounds; why lowering N never cancels a request.
- The worker loop, the policy runner's lock, and the controllers.
- Straggler splitting ($m = s + b + \lfloor r/2 \rfloor$, the 2 MiB minimum) and the request deadline; the engine ablation and why it changed the conclusion (Chapter 8).
- The paced server's three rules, the request-start rate rule, the soft cap formula (3.3), the live-rate mode, fault injection.
- netem (40 ms each way = 80 ms RTT, 0.4 % loss), the delay–loss sweep, Mahimahi replay, the live-pilot runner and mirror etiquette.
- The D2 mechanism (why 112 of our 128 requests start after the speed-up), the granularity control (0.862), the constant-rate controls.
- Validation: SHA-256 checks, the test suite, race detector, the claims checker.
- The demo steps 1 and 2 (Chapter 14).

5.4.2 Shwetank Shrijan (24BDS0141): components, monitoring, automation, baselines and heuristics

Stated work: downloader development and testing of assigned components; monitoring and experiment automation; baseline and heuristic evaluation.

Be ready to explain:

- Metrics and monitoring: per-attempt records, per-request vs aggregate throughput, TTFB, rolling windows, the run record fields (Chapter 3).
- Test types and representative test cases; failure-handling tests (`200` instead of `206`, short bodies, wrong `Content-Range`).
- Experiment automation: arms, blocks, seeded order, fresh state per run, validity rules, netem status recording, the harness programs.
- Baselines: aria2 flags and how they were chosen; axel, lftp, curl, NGINX; why not wget2 and IDM (Chapter 6).
- Heuristic controllers: the throughput rule, the causal drop rule, the workload selector, the online tuner; their results and adaptation speed (Chapter 8).
- The statistics: geometric mean, bootstrap, t-interval, verdicts, workload-level intervals (Chapter 4).
- The live-pilot and mirror-check results and how the link gate worked (Chapter 10).
- Demo step 3: tests and the claims checker.

5.4.3 Ayush Konar (24BDS0107): LinUCB, RL design, models and learned-policy evaluation

Stated work: LinUCB; RL state, action and reward design; Python/PyTorch models; integration of the learned policy with the Go downloader; ML/RL evaluation.

Be ready to explain:

- LinUCB: features, actions, reward, the UCB formula, and why it locked onto one action (Chapter 9).
- DQN and Double DQN (equations in Chapter 2); the 13 state features; actions; reward $r_t = -\Delta t$; hyperparameters; network $13 \to 64 \to 64 \to 3$.
- The DQN version history v1–v6 and why each changed; why S was removed from the action at v4.
- The simulators, what the v6 simulator got wrong, calibration from request logs, blind parity checks, Go inference and parity tests.
- The cap-trained v1, the joint agent, and v2: training distributions, seed selection on validation only, held-out scenario design and headroom, results, the drop-rule comparison (Chapter 9).
- The oracle and headroom gates, and the argument "the opportunity, not the learner, was the bottleneck" (Chapter 11).
- The model-based attempt and the L4 simulator-only headroom.

Viva tip

Everyone should be able to give the one-page story (Chapter 1) and the novelty statement (Section 11.3). If a question goes to another member's area, answer the headline and say "*[name] can give the detail*". That shows teamwork and is better than guessing.

6. Results I: Fixed Settings Against aria2 and Other Tools

6.1 Early aria2 comparisons and the granularity confound

The first aria2 runs (`benchmark_aria2`, aria2 at 4 connections on the 4 MiB LinUCB-era scenarios, 50 of 50 valid) were confounded by request granularity: aria2 made 1 unranged request plus 3 ranges of 1 MiB with at most 4 in flight, which is not how our benchmark modes split the file (PROJECT_CONTEXT §22.16–22.17). A granularity-matched internal control (our engine fixed at 4×1 MiB, `cmd/granularity-study`) was needed before any comparison meant anything. This taught the lesson that shaped the rest of the project: *when comparing downloaders, match or explicitly vary granularity*. aria2 was later recalibrated on a validation set to "config F" (16 connections, split 16, min split 1 MiB), used in every later study.

6.2 The D2 benchmark: 62.4 % less time (Confirmed)

Workload D2: 512 MiB; per-response pacing 640 KiB/s, 160 KiB/s from 2 s, 2,500 KiB/s from 4 s; netem 40 ms each way + 0.4 % loss. Both systems receive the same object.

Selection (discovery): all 40 (N,S) combinations were run once (the top three at $N = 16$ three times) (Figure 6.1). More connections always helped; at every N a 4 MiB range was fastest or tied; we stopped at $N = 16$ to match aria2. aria2's full split frontier (1/2/4/8/16) was also run.

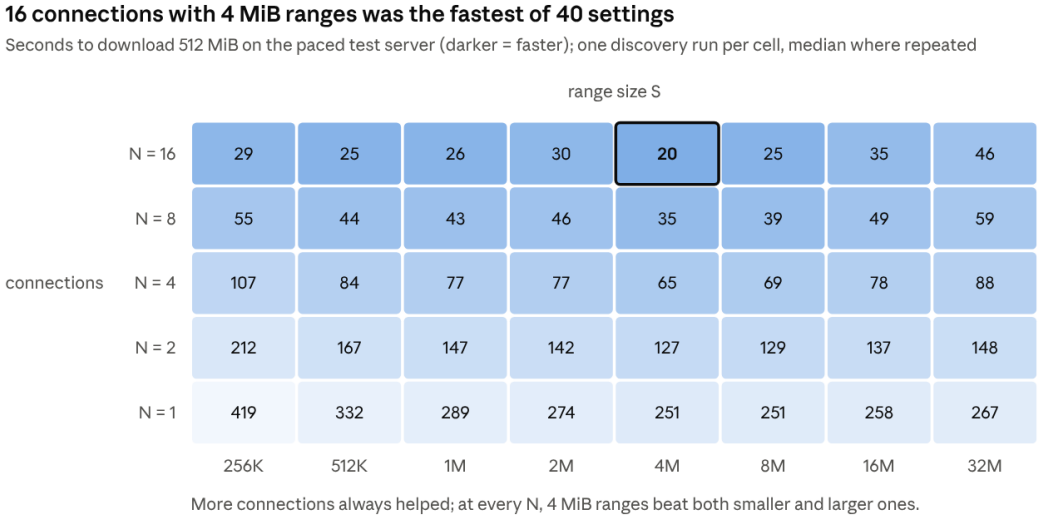


Figure 6.1: Discovery grid: completion time for all 40 (N,S) settings on D2.

Confirmation (30 fresh pairs): ours (16×4 MiB) 20.12 s vs aria2 53.56 s (means); geometric ratio **0.3756**, 95 % CI $[0.3726, 0.3787]$; 30 of 30 wins; all outputs content-verified.

6.2.1 The mechanism

From per-request server logs:

- aria2 issues 16 requests (one unranged probe plus 15×32 MiB ranges), **all starting before 2 s**, so all are held at 640 KiB/s for their whole length.
- Ours issues 128×4 MiB ranges: 16 start before 2 s, and **112 start after 4 s**, in the fast phase (3,840 ranged requests over 30 runs: 480 early, 3,360 late).
- aria2 cannot be configured to finer initial pieces: with `-x16`, `-split` of 16, 64 or 256 gave the same range-size distribution in a local probe.

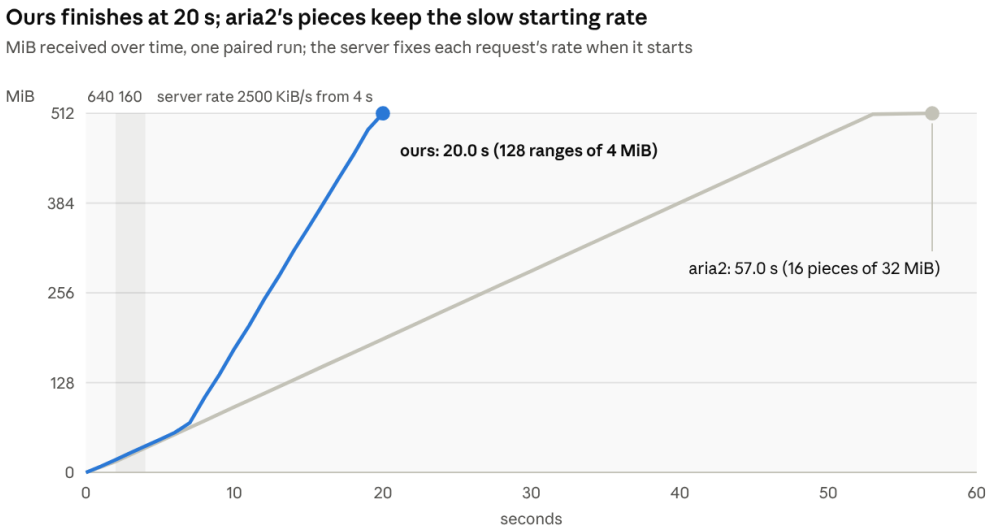


Figure 6.2: Data received over time in one paired run, reconstructed from per-request server logs.

6.2.2 Controls

- **Granularity-matched control:** our engine with aria2-like 32 MiB pieces gave $R = 0.862\,[0.834, 0.890]$ (10 pairs). Most of the gap is granularity, but ours stays 14 % faster.
- **Constant 2.5 MB/s schedule:** removes most of the 4 MiB vs 32 MiB difference inside our engine ($2.3\times \rightarrow 1.1\times$), yet ours is still faster than aria2 (15.3 s vs 24.8 s).
- So the phase-locked mechanism explains the *size* of the D2 gap, not all of it. The residual is **unexplained**; a candidate is the finishing tail of 32 MiB pieces, untested.

Be careful how you say this

This is a *fixed-configuration* result. Neither system adapts during the transfer. Do not present it as "adaptation beats aria2". And it is one workload family on our own server, so it is a property of this pacing mechanism, not a general statement about aria2.

6.3 Pacing-schedule sensitivity (screen)

Same workload, systems and netem; only the server schedule varies; 5 pairs per schedule; 60 of 60 valid; protocol frozen with seed 20261011.

Table 6.1: Pacing sweep, ours (16×4 MiB) / aria2

Schedule (KiB/s; switch times)	Ours (s)	aria2 (s)	Ratio	[min, max]	Ours faster
S0 D2 `640,160@2,2500@4`	20.1	53.3	0.378	[0.372, 0.389]	5/5
S1 no dip `640,2500@2`	20.1	48.9	0.425	[0.353, 0.751]	5/5
S2 long dip `640,160@2,2500@8`	44.1	54.7	0.806	[0.783, 0.827]	5/5
S3 partial `640,160@2,1280@4`	31.6	54.3	0.583	[0.559, 0.608]	5/5
S4 constant fast `2500`	15.4	27.1	0.568	[0.523, 0.618]	5/5
S5 constant slow `640`	53.8	53.9	0.998	[0.987, 1.008]	3/5

Both predeclared expectations held: S0 reproduces D2 (0.378 vs 0.3756) and S5 (one slow rate, 16 connections each) is at parity. The advantage is largest when the slow phase is short (S0, S1), shrinks when it is long (S2), and persists under constant fast pacing (S4) where no phase-locking exists; the S4 mechanism is a hypothesis.

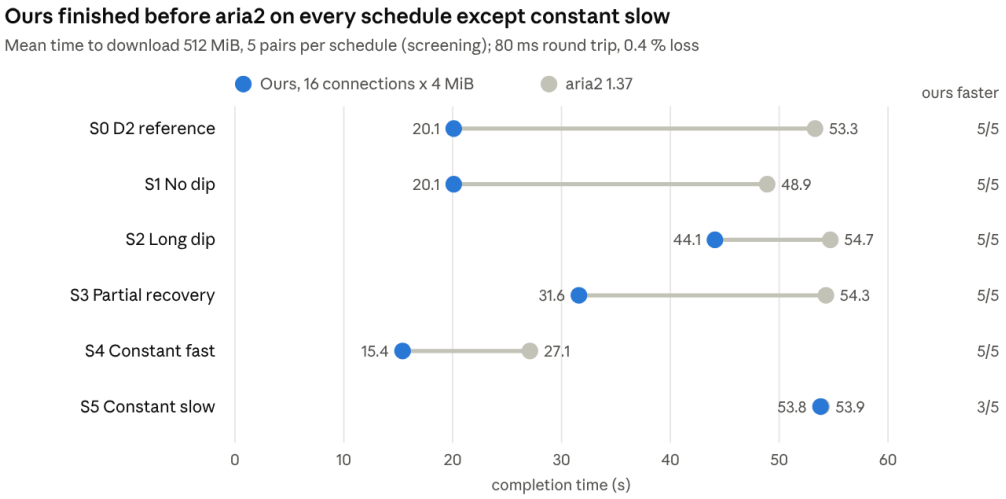


Figure 6.3: Mean completion time under the six schedules.

6.4 Stock NGINX check (screen)

NGINX 1.24.0, same 512 MiB object (deterministic AES-CTR stream), 5 pairs per configuration, 20 of 20 valid. It limits each response independently: no shared cap, no connection limit.

- N1 `limit_rate 640k`: ours 50.3 s, aria2 58.3 s, $R = 0.863$, 5/5.
- N2 `limit_rate_after 1m; limit_rate 640k`: 33.7 s vs 56.4 s, $R = 0.597$, 5/5.

The predeclared expectation for N1 was parity (as in S5). **It was not met**: ours was faster in all five pairs, for reasons not tested (NGINX pacing granularity, aria2's request handling, connection behaviour). It supports

only the direction of the comparison on a stock server.

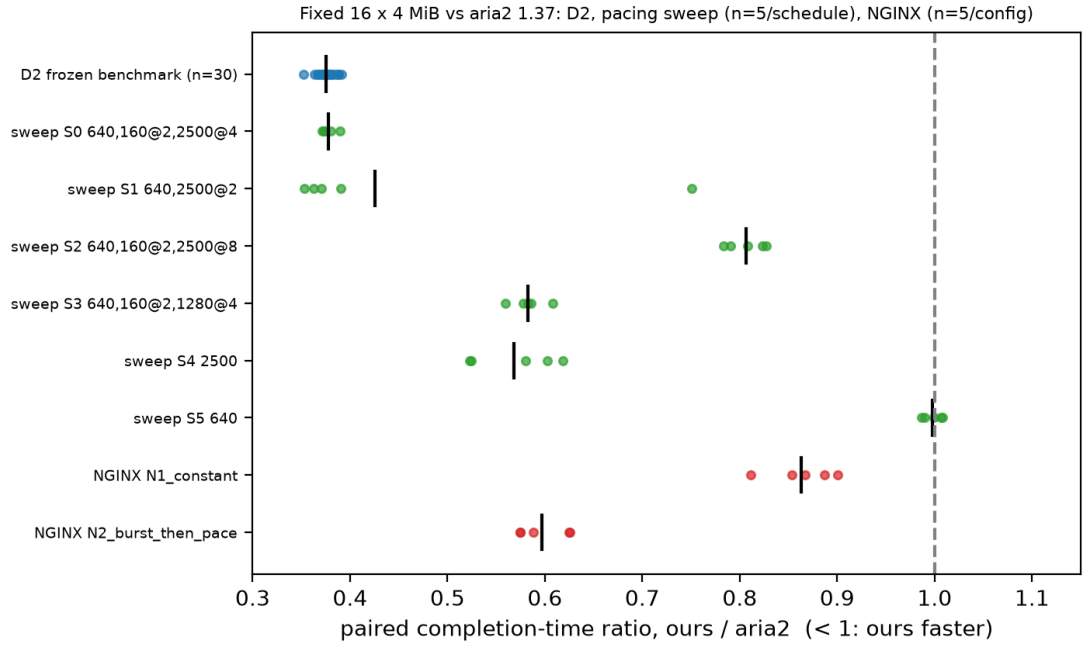


Figure 6.4: Paired ratios ours/aria2: D2 benchmark, six sweep schedules, two NGINX configurations. Sweep and NGINX cells are screens ($n = 5$).

6.5 Other segmented downloaders: axel and lftp (screen)

Frozen protocol `baselines_ext`; all tools at 16 connections; HEAD requests answered at once (no tool pays for a paced probe); 5 blocks per scenario; 80 of 80 valid. wget2 removed by amendment A2 (single connection per file).

Table 6.2: Ours against axel, lftp and aria2 (ratios; 95 % bootstrap)

Scenario	Ours	ours/axel	ours/lftp	ours/aria2
HEALTHY	16×1 MiB	0.345 [0.344, 0.347]	0.345 [0.344, 0.346]	0.357 [0.344, 0.371]
D2	16×4 MiB	0.375 [0.373, 0.378]	0.374 [0.369, 0.377]	0.375 [0.373, 0.377]
CAP8	16×32 MiB	1.003 [0.965, 1.026]	1.001 [0.963, 1.024]	0.996 [0.960, 1.017]
CAP0	16×32 MiB	1.025 [1.021, 1.029]	0.597 [0.594, 0.599]	0.995 [0.992, 0.998]

- axel and lftp split the file into 16 equal parts at the start, as aria2 does, so they gain nothing from a schedule that rises after the first second. **The D2 gap is not aria2-specific; it applies to tools that split once, up front.**
- Under CAP8 all four were level (about 53 s). Under CAP0, lftp took 114 s because it never re-splits a slow part (our engine without splitting behaved the same), and axel was 2.5 % faster than ours.

6.6 Delay and loss sweep (screen)

Frozen protocol `netem_sweep`: one-way delay $\in \{10, 40, 100\}$ ms $\times$ loss $\in \{0, 0.4, 2\}$ %; 3 blocks per cell; 108 of 108 valid.

- **D2** (16×4 MiB vs aria2): ours faster in every block of all 9 cells; cell means 0.362–0.457. The direction holds across the grid.
- **CAP8** (16×32 MiB vs aria2): the direction does *not* hold. 5 cells mixed; 2 had aria2 faster in every block: 10 ms with 2 % loss (1.073) and 40 ms with 0.4 % loss (1.035). At 100 ms ours was faster with 0 and 0.4 % loss (0.769, 0.955) and mixed with 2 %. Under the cap the two are near parity and the sign depends on the network.

6.7 Goodput with the fixed engine

Table 6.3: Goodput in MB/s (geometric mean of 10 runs; ours on the fixed engine)

Scenario	Our best fixed setting	Ours	aria2
CAP0	16×32 MiB, splitting	7.87	7.83
CAP8	16×32 MiB, splitting	9.87	10.03
HEALTHY	16×1 MiB	29.19	10.05
D2	16×4 MiB	27.02	9.99

What Chapter 6 shows

Much of our large wins comes from servers that set a response's rate when it starts: small ranges keep starting new requests after the server speeds up, while split-once tools stay at the slow starting rate. Part is unexplained (constant fast pacing S4, NGINX N1). At a constant slow rate the tools are level, and under the cap with splitting within 1.6 %; on the public mirrors aria2 took 1.00–1.26$\times$ each condition's best fixed cell. None of this involves adaptation.

7. Results II: Searching for an Adaptation Opportunity

After the D2 result, the natural question was: *is there any condition where changing N or S during a download beats the best fixed setting?* We searched systematically. Almost everything came back negative, and each negative result is informative.

7.1 Rule-based adaptation on D2 (Track 1)

Live, 20 blocks $\times$ 4 modes (fixed, N-only, S-only, joint heuristics), 80 of 80 valid. The joint heuristic took $\mathbf{2.316}\times$ as long as the best fixed setting ($R = 2.316\,[2.302, 2.329]$, 0 of 20 wins), though it was still faster than frozen aria2 (0.876). The heuristics adapt, but adapting costs more than it gains on D2. Outcome label: "beats aria2 only".

7.2 Mechanism screens

Table 7.1: Screens that looked for a state-dependent optimum (all closed before policy work)

Screen	Type	Result
Static rate + RTT (S-protocol v7, Stage 0S, scaling)	Live pilot + analysis	Range size has a real service-cost effect (Stage 0S GO, 32 of 32 runs) but every frozen switch had non-positive headroom (best exploratory +0.35 % vs 5 % bar)
S mechanism pivot (8 Mahimahi + 23 throughput traces, 4 service models)	Analytical	No robust causal opportunity; only a fragile clairvoyant gain
Adaptive-N mechanisms (eight candidate models, incl. direct, fair, fair+background, cap, degrade)	Analytical	≥ 5 % causal gain only in the `degrade` model (up to +6.66 %), which live grounding rejected
Joint N+S under phase pacing	Analytical (calibrated to D2, 40/40 configs within 3 %)	$N^* = 16$ everywhere; best causal bound +3.4 % (oracle-tuned ramp +3.7 %) vs 5 % bar; the primary one-switch causal controllers lost 16–28 %
Shared bottleneck + `iperf3` background traffic	Live, 45 runs	No interior N optimum: flat in the light states (margins < 5 %), and under heavy contention $N = 16$ won by a lot
Correlated loss + range retry (Gilbert–Elliott)	Analytical	$N^* = 16$ in 9 of 9 variants. An earlier "S fragile" reading was an artefact of a modelled 30 s request deadline: without it $S^* = 16$ MiB in all 9, no failures
Per-request server throttling (T1)	Live	No concurrency reversal at equal 1 MiB granularity
Adaptive range size across server phases (T2)	Live	Monotone toward the largest range; no reversal
Failure-based N-dependent throttling	Analytical	An abort cliff, not a tunable optimum

Key idea

Conclusion of the screens: within-download adaptation needs a mechanism whose optimum moves in N and S and is causally observable. None was found in the screened mechanisms other than the constructed soft cap of Chapter 8. The screens searched a space of server behaviours (72 cap candidates alone), so results in the capped chapters are *existence demonstrations in a selected environment*, not prevalence estimates.

7.3 Recorded cellular traces: the oracle gate (Confirmed; Failed gate)

Five recorded driving traces (Verizon EVDO and LTE, AT&T LTE, T-Mobile LTE and UMTS) were replayed through `mm-link`. Capacity in the Mahimahi traces swings about 5–27$\times$ (p90/p10). A per-trace

counterfactual oracle, with switching costs ("tau") measured in a separate injection campaign, estimates how much faster switching N at 120 s epoch boundaries (ladder $1 \leftrightarrow 2 \leftrightarrow 4 \leftrightarrow 8 \leftrightarrow 16$) could be than the best fixed N.

Table 7.2: Oracle headroom per trace (`testdata/final_oracle_analysis`)

Trace	Best fixed (s)	Best N	Headroom H
Verizon EVDO	543.98	8	1.641 %
Verizon LTE	68.52	16	0.019 %
AT&T LTE	53.18	4	0.016 %
T-Mobile LTE	12.17	4	0.061 %
T-Mobile UMTS	173.11	2	0.294 %
Mean			0.406 %
One-sided 95 % lower bound ($n = 5$, df $= 4$)			−0.26 %
Predeclared bar			10 %

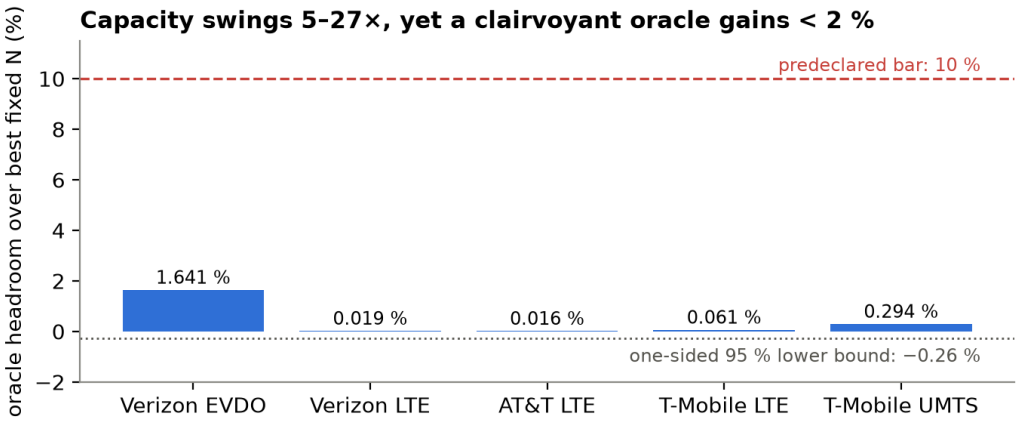


Figure 7.1: Oracle headroom per trace against the 10 % bar.

"Capacity changes 27× and there is no headroom? How?"

On one bottleneck path, the best N is roughly "enough connections to fill the pipe". When capacity drops, every setting slows down together; the *same* N stays about best. Adaptation pays only if the *best setting moves*, not if the speed moves. The oracle measures exactly that, and it found the best setting barely moves. Hence: **variability is not opportunity.**

Be careful how you say this

This is trace *replay* on a controlled host under a frozen counterfactual model, not measurement on live mobile networks. And an oracle is not a policy: the bound applies to any concurrency-switching policy *under that model*.

7.4 Simulated headroom that was not real (L2–L4)

Before retraining DQN ("v7"), three environment designs were tested for headroom in the simulator:

- L2: headroom $-0.3\,\%$; L3: $8.1\,\%$ ($< 10\,\%$ gate, stop).
- L4: **17.6 %** simulated headroom (best static $C16 = 174.0\,\text{s}$ vs an action-constrained oracle $143.4\,\text{s}$), passing the $10\,\%$ gate. But a small live sweep showed that high concurrency is *not* penalised live (degraded regime: C16 3.57 MiB/s, the fastest). The headroom came from a simulator-only mechanism. Also, the phases where C16 was worse carried $47.1\,\%$ of bytes, just under a predeclared $\geq 50\,\%$ rule (strictly, `NOT_READY_PHASE_WEIGHTING`). Outcome: `NOT_READY_LIVE_MISMATCH`; another training iteration was not justified.

Lesson: simulator headroom must be confirmed live before training on it. This is why every later simulator was checked blind against live runs.

8. Results III: The Capped Server, Adaptive Control and the Engine

Since no screened mechanism offered an opportunity, we *constructed* one: the soft concurrency cap (Equation 3.3). Under it, too many connections cost something, and if the cap switches on mid-download, the best setting changes during the download.

8.1 The live mechanism gate (Failed gate, 5 of 6 checks passed)

Family: healthy server vs capped server ($C = 8\,\text{MiB/s}$, $\gamma = 0.05$). Stationary optima: $(16, 1\,\text{MiB})$ healthy vs $(4, 4\,\text{MiB})$ capped. Six predeclared checks:

- N^* moved $16 \to 4$ and S^* moved $1 \to 4\,\text{MiB}$ (N-margin 5.75 %, just above its 5 % bar);
- cross-state penalties were material: +31.8 % using the healthy optimum under the cap; +254.4 % using the capped optimum when healthy;
- **the local S-margin was 3.74 %** (95 % bootstrap [3.09 %, 4.45 %], after four extra replicates) against 5 %: **failed**, and the criterion was not changed.

Three high-N/large-S capped cells aborted at the 30 s request timeout and are kept as invalid. The adaptive runs on this platform are therefore *exploratory* for the S-side mechanism.

8.2 Workload selection

8.2.1 Two workloads (P1)

Selector: probe at $(16, 1\,\text{MiB})$, classify after ≥ 20 samples, apply the predeclared (N, S), no further switching. Live, 10 blocks per workload:

- healthy: 19.1 s vs fixed $(16, 1\text{M})$ 19.0 s ($R = 1.006\,[0.931, 1.082]$); vs aria2 53.9 s ($R = 0.353$, 10/10);
- capped: 84.2 s vs fixed $(16, 1\text{M})$ 115.3 s ($R = 0.730$, 10/10); vs aria2 68.3 s ($R = 1.232$, 0/10: aria2 faster).

8.2.2 Six-workload family W1–W6 (Descriptive)

Six predeclared workloads (W1 healthy; W2–W6 cap/schedule/transition variants) were chosen by a predeclared rule from 72 analytic candidates (13 qualified). Two amendments were made after design output and before any live run; the first manifest was overwritten. Live validation: 5 of 6 model optima matched; **W5 mismatched** (live argmin $(16, 16\text{M})$ 85.4 s; the model's target $(4, 4\text{M})$ 95.9 s). 10 paired blocks per workload, 240 runs, 0 invalid.

Table 8.1: Selector across W1–W6

Selector vs …	Pooled ratio	Block CI	Workload-level CI	Blocks won
per-workload best fixed	0.999	[0.969, 1.029]	[0.873, 1.143]	24/60
family-best single fixed $(4, 4\text{M})$	0.770	[0.684, 0.859]	[0.455, 1.304]	40/60
aria2	1.660	[1.360, 1.990]	[0.707, 3.901]	10/60

The workload-level interval is primary (the unit of generalisation is the workload) and includes 1 for all three. The 0.770 is driven by W1 (0.286); over W2–W6 the geometric mean is 0.94 and the selector loses on W5 (1.058) and W6 (1.090). W4's 0.808 against its per-workload best is probe-phase timing (the selector runs the fast config until the cap appears), not better selection.

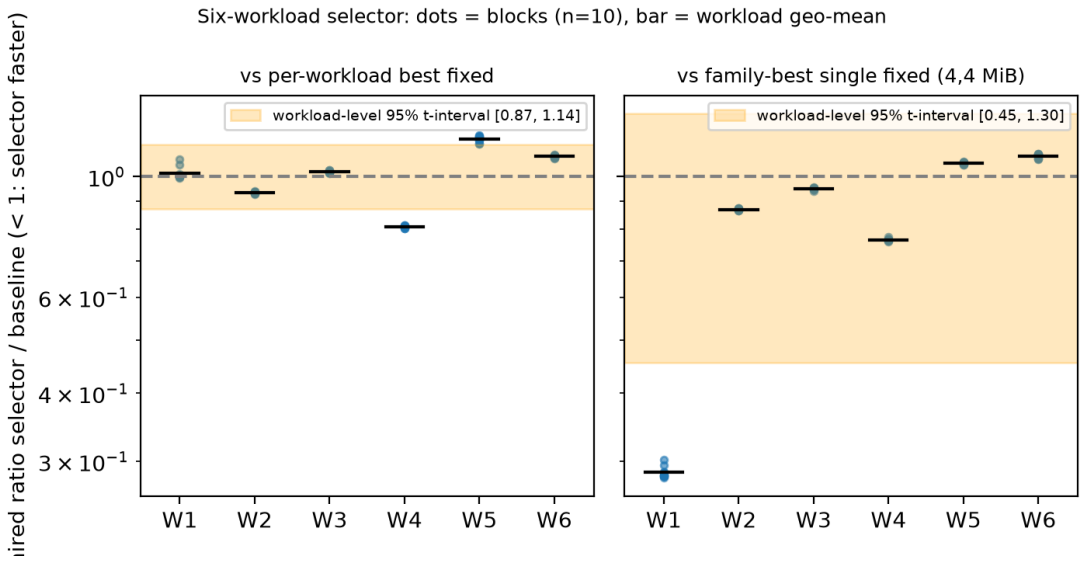


Figure 8.1: Selector ratios per workload; the shaded band is the workload-level 95 % interval.

8.3 Within-download adaptation (P2): 23.4 % less time (Confirmed on the original engine)

Schedule: no cap on $[0, 8\,\text{s})$, cap from 8 s. Controller: the causal drop rule, switching once to a predeclared target. 10 paired blocks, 0 invalid.

- **Causal joint** (target $(4, 4\text{M})$): 58.7 s vs the best of the three stationary configurations run, $(16, 1\text{M})$ 76.7 s: $R = 0.766\,[0.762, 0.769]$, 10/10 (23.4 % less time). The comparison set was $\{(16, 1\text{M}), (4, 1\text{M}), (4, 4\text{M})\}$; no full grid was run under this schedule.
- **Ablation:** N-only 62.0 s ($R = 0.808$); joint vs N-only 0.947; ***S*-only 82.1 s, worse than not switching** (1.070, 0/10). The N move carries the gain.
- **aria2 was faster:** 53.6 s vs 58.7 s ($R = 1.096$, 0/10).

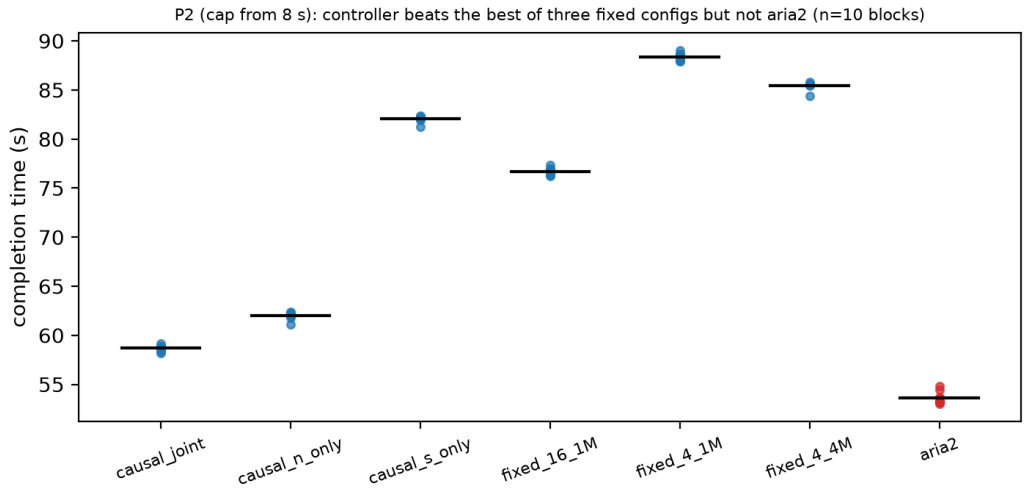


Figure 8.2: P2 completion times (10 blocks).

8.4 The engine ablation: why aria2 led under the cap (Confirmed)

An audit (24 Oct) found two engine differences between our downloader and aria2: (a) a 30 s whole-request deadline that aborts large ranges that are slow but progressing; (b) no straggler splitting. Both were made opt-in (`NoRequestTimeout`, `SplitStragglers`) and tested in a predeclared ablation: CAP0, CAP8, HEALTHY, D2 $\times$ 10 shuffled blocks, **230 runs, all valid**, zero failed attempts, retries or wasted bytes on the client side.

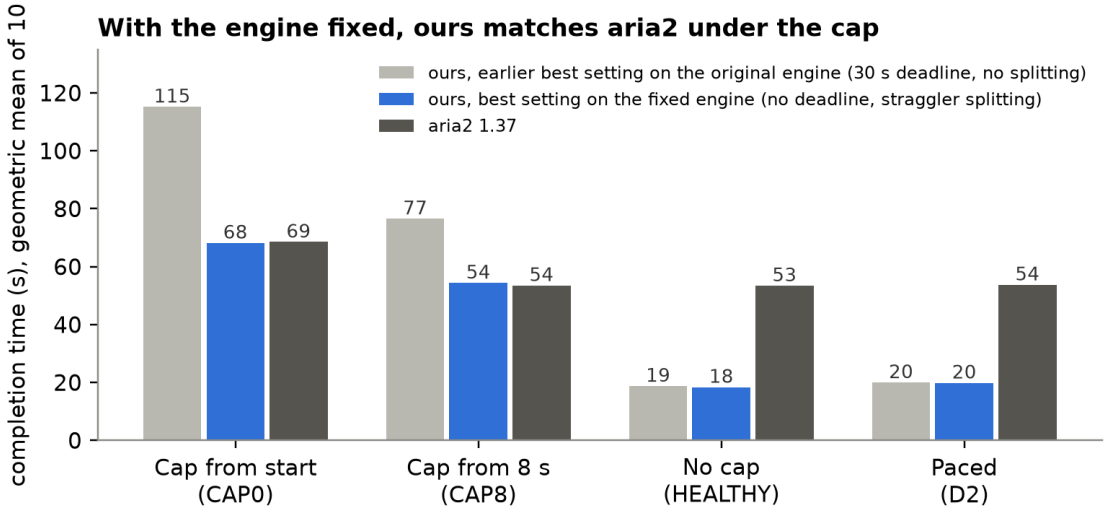


Figure 8.3: Completion time before and after the engine fix, and aria2.

- **Splitting was the main cause.** CAP0: 16×32 MiB with splitting took 40 % less time ($R = 0.599\,[0.598, 0.601]$) and matched aria2 (68.2 s vs 68.5 s, $R = 0.995\,[0.992, 0.999]$, detectable but negligible).
- **Large ranges need the deadline removed.** CAP8: 16×32 MiB without the deadline took $0.711\,[0.703, 0.719]$ of the earlier best fixed (16×1 MiB) and was within 1.6 % of aria2 (54.4 s vs 53.5 s, $R = 1.016\,[1.006, 1.029]$).

- **No harm elsewhere.** HEALTHY and D2: corrected vs original engine not distinguishable (0.981, 0.995); ours still far faster than aria2 there (0.344, 0.370).
- **Adaptation against the corrected baseline.** The causal controller replicated against its original set (0.767) but was **slower than the best fixed setting on the corrected engine**, 16×32 MiB: $R = 1.071\,[1.054, 1.085]$. Under CAP0 the selector was slower than 16×32 MiB: 1.229.

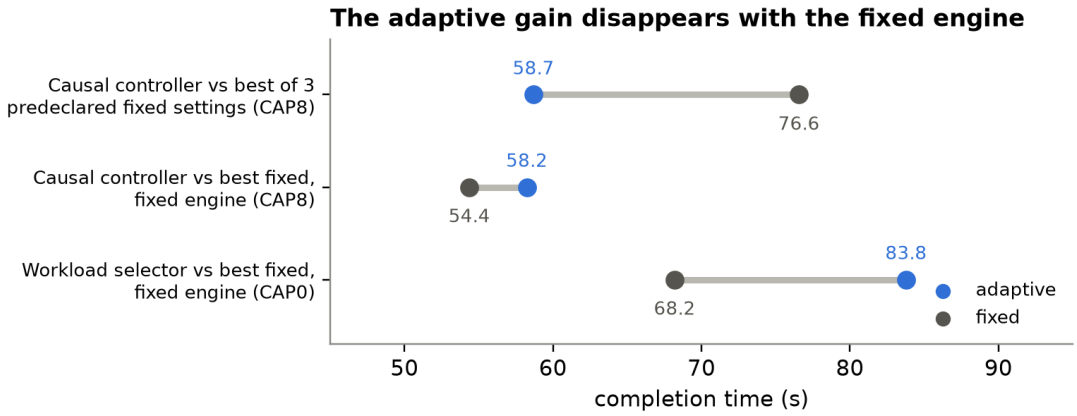


Figure 8.4: Adaptive controllers against the best fixed setting on the corrected engine.

Engine before policy

On our server models, two engine details (timeout semantics and straggler splitting) mattered more than any adaptive policy. The earlier +23.4 % holds only against small-range fixed settings on the original engine; we report both.

8.5 Online concurrency tuner (prior-work re-implementation)

FastBioDL-style tuner (Chapter 3); predeclared protocol; CAP8 and HEALTHY $\times$ 4 arms (tuner, fixed 16×1 MiB, fixed 4×1 MiB, aria2) $\times$ 8 blocks; 64 of 64 valid; corrected engine. The tuner keeps $S = 1$ MiB, so it is compared with fixed settings at the same range size; under CAP8, 16×32 MiB (54.4 s, engine ablation) was faster still.

Table 8.2: Online tuner (ratios tuner/other)

Scenario	Tuner (s)	Better fixed 1 MiB arm (s)	vs best fixed	vs aria2	Final N
CAP8	93.1	16×1 MiB, 76.8	1.212 [1.180, 1.248]	1.781	12.5
HEALTHY	22.5	16×1 MiB, 18.4	1.220 [1.203, 1.232]	0.432	16

It cut N 2.1 s after the cap (the causal controller 1.5 s), but needed 8.1 s to climb from $N = 4$ to 16 with no cap, and that ramp is where most of its loss occurs.

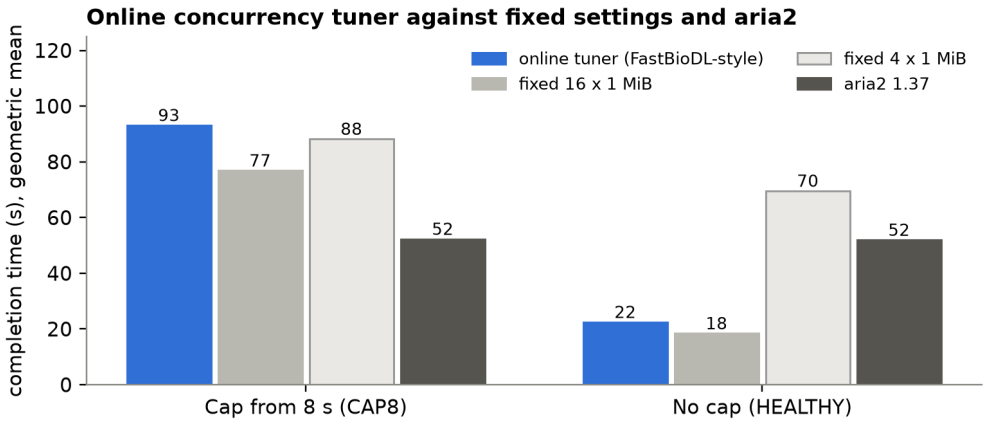


Figure 8.5: Online tuner against fixed settings and aria2.

8.6 Adaptation speed

Table 8.3: Adaptation speed (median seconds, range over runs)

Controller	Scenario	Measured	Seconds	Runs
Causal drop controller	CAP8	first N cut after the cap	1.52 [1.49, 1.74]	10
Online tuner	CAP8	first N cut after the cap	2.14 [2.06, 2.32]	8
Workload selector	CAP0	first move (classification)	3.99 [3.98, 4.25]	10
Online tuner	HEALTHY	climb from $N = 4$ to 16	8.08 [8.00, 8.25]	8

Every controller reacted within a few seconds, so *slow reaction is not why adaptation failed to pay*. The limit is what reacting can gain.

8.7 Why does 16×32 MiB win under the cap?

Under CAP8, 16×32 MiB starts all 16 ranges before the cap at 8 s, so every range keeps the pre-cap rate for its whole length (the request-start rule). Under CAP8 splitting adds almost nothing (54.43 s without, 54.37 s with); splitting was decisive under CAP0, where ranges start under the cap. A 1 MiB controller, however clever, starts almost every request after the cap and gets the capped rate (Figure 3.8). This is a property of our server model; the live-rate control server (Chapter 9) confirms it: there, 16×32 MiB slowed from 50.9 s to 67.4 s.

9. Results IV: Every Learned Policy

The learners act through the same policy interface as the rule-based controllers. Status words used here: **valid** (internally consistent, reproducible); **stale** (produced before a later simulator correction); **non-comparable** (different workload, file size or aria2 configuration, so ratios must not be pooled).

9.1 LinUCB

- Five- and six-mode 4 MiB benchmarks (benchmarks v1–v5, 256 KiB files at first, 4 MiB from v3; up to 300 runs; 5 scenarios: baseline, server throttle, latency, loss, combined).
- In the first formal benchmark LinUCB chose "no action" 100 % of the time (exploration lock-in). After an exploration fix, small files left only about 2 learned decisions, so the file was raised to 4 MiB (32–122 learned decisions). Standard LinUCB still locked onto one arm; periodic exploration changed coverage but not the dominant preference.
- Outcome: *descriptive*; no ranking was made. This motivated moving to Double DQN, which assigns credit over time.

9.2 DQN versions v1–v6

Table 9.1: DQN candidates (Double DQN, trained in simulation, run live in Go)

Version	Design	Outcome	Status
v1	Simulator train/validate/test + 25-run live transfer	Superseded; trained on an environment later corrected	stale
v2	Joint N, S; 240 pairs vs aria2 `-x4 -s4`, profiles U1–U8, 4 MiB	75.7 % slower ($R = 1.757\,[1.672, 1.847]$); faster in 31/240; fragmented ranges into 32–128 KiB	valid; non-comparable
v3	Simulator + live vs v2	Fixed fragmentation, but its simulator rewarded large ranges the server did not	valid (negative)
v4	N only, $S = 1$ MiB; 25-run live transfer	Live matched simulator; 4 MiB file gave only a four-request horizon	valid; no head-to-head
v5	$N \in 1..4$; 240 pairs vs aria2-F (16 conn), 16 MiB, U1–U8	$R = 2.322\,[2.230, 2.417]$; faster in 1/240; structural 4-vs-16 connection asymmetry	valid; model stale (later simulator fix)
v6	$N \in 1..16$; 150 pairs vs aria2-F, trajectories A2–E2, 16 MiB	pooled $R = 1.456\,[1.305, 1.624]$; faster in 37/150; E2 not slower (0.948 [0.897, 1.002]); D2 5.47× slower	valid
v6 long files	16/128/512 MiB, 5 pairs per cell	pooled 1.805 → 1.062 → 0.909 (512 MiB CI [0.750, 1.102] includes 1), driven by D2; A2 and E2 got worse with size	exploratory

Two handicaps worked against these agents regardless of learning: **short downloads** (4–16 MiB, a few seconds) and a **ramped start** (v6 started at 1 connection; aria2 opens all at once). The trajectories A2–E2 are 3-phase wall-clock scenarios: A bandwidth collapse/recovery, B latency spike, C burst loss, D server throttle/release, E combined.

9.3 Frozen v6 on the capped server (screen)

To remove both handicaps: 512 MiB, corrected engine, start $N = 1$ (as trained) or 16 (as aria2), not retrained; 5 blocks, 50 of 50 valid.

Table 9.2: Frozen DQN v6 on the capped and uncapped server

Scenario	Agent	Time (s)	vs best fixed	vs aria2
CAP8	start 16	71.3	1.356 [1.252, 1.472] (16×32)	1.367
CAP8	start 1	89.7	1.706 [1.612, 1.806]	1.720
HEALTHY	start 16	19.0	1.031 [1.010, 1.063] (16×1)	0.359
HEALTHY	start 1	22.0	1.194 [1.176, 1.219]	0.416

Against 16×1 MiB (its own range size) under the cap: not distinguishable, 0.931 [0.879, 1.016]. It changed N several hundred times per download (median 412 N-moving decisions under the cap), ending at $N = 2$.

9.4 Cap-trained DQN v1 (`rl_cap_train`)

What the v6 simulator got wrong: no cap fixed at request arrival; and its aggregate-throughput observable did not match the live runner's (live: bytes of the last `window` attempts over their span; with window 4 this barely changes above $N = 4$). The new simulator (`sim_env_cap.py`) fixes both and uses window 16; per-request server excess and client overhead are drawn from live request logs.

- **Blind live check (Confirmed for the simulator, 8 runs):** 4 fixed settings predicted before running; all within 1.4 % (largest -1.35 %).
- **In distribution (Confirmed, 10 blocks, 120 runs):** CAP8, start 16: $\mathbf{0.907}\,[\mathbf{0.893}, \mathbf{0.921}]$ of 16×1 MiB (simulator predicted 0.912). Slower than 16×32 MiB (1.298) and aria2 (1.310), as stated in advance. Not distinguishable from frozen v6 (0.949). HEALTHY: slower than 16×1 MiB (1.108). Start 1: slower in both.
- **Held out (Confirmed, 5 blocks each, 65 runs):** each scenario outside the training ranges on one axis. Cap $C = 16$ MiB/s (trained 4–12): 1.177, slower. 80 ms each way (trained 20–60): 1.016, n.d. (simulator predicted 0.930; jitter calibrated at 40 ms only). d2 schedule vs 4×1 MiB: 1.015, n.d. **Faster in 0 of 3.** (In G_C16 the old frozen v6 was not distinguishable from best fixed, 0.993, and v1 was slower than v6, 1.186.)
- **Reading:** it learned the server it was trained on; it did not generalize.

9.5 Joint concurrency and range-size agent (`rl_joint`)

5 actions (none, ±1 connection, ±4 MiB), range 1–33 MiB. Aim: can a learner find the large-range behaviour that wins under the cap?

- **Headroom existed in the simulator:** a hand-written rule (1 MiB ranges until the rate rises, then 33 MiB) scored **0.352** of the best fixed time under CAP8. That headroom comes from the arrival-time rule, so the protocol added a **live-rate control server**.
- **Simulator check (8 blind runs):** within +1.5 to +7.0 %; optimistic for long ranges.
- **Training:** first run undertrained (ε ended near 0.75; amendment A1); retrain 747,503–823,821 steps per seed. The selected agent still scored 1.513 of best fixed under CAP8 in simulation: it changed range size, but too late.
- **Live, arrival-rule server (Confirmed, 10 blocks, 150 runs):** slower than the best fixed setting in every scenario: CAP8 **1.471 [1.405, 1.546]** vs 16 × 32 MiB; HEALTHY 1.081 vs 16 × 1; CAP0 1.359 vs 16 × 32; D2 1.372 vs 16 × 5 MiB. Vs aria2: CAP8 1.422 slower, HEALTHY 0.388 faster. Live within about −1 to +5 % of predictions. At 8 s it was still on 1 MiB ranges in every scenario.
- **Live-rate control (Confirmed, 5 blocks, 40 runs):** fixed 16 × 32 MiB under the cap slowed from 50.9 s to 67.4 s; the agent was not distinguishable from it (1.029 [0.989, 1.081]); without a cap slower (1.128).
- **Reading:** the simulator was accurate; the learner's search failed. A simple rule found headroom the learner did not. Most of the large-range advantage under the cap comes from the arrival-time rule.

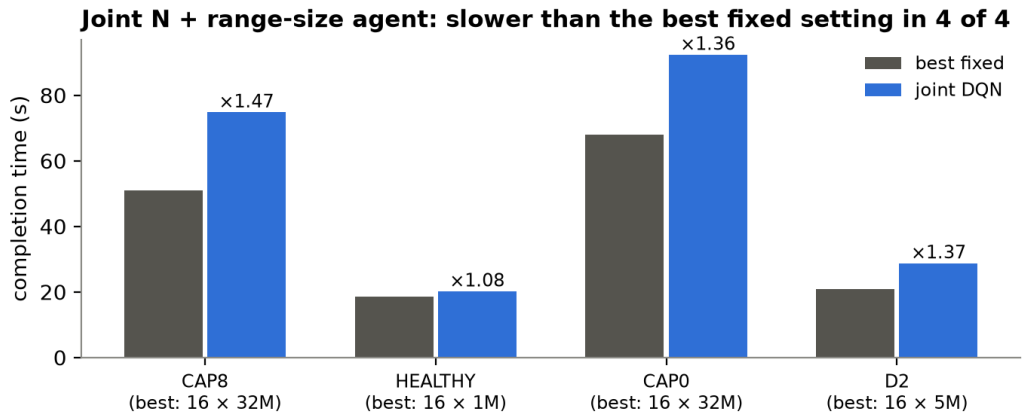


Figure 9.1: Joint N + range-size agent against the best fixed setting per scenario (arrival-rule server, 10 blocks).

9.6 DQN v2: broader training, held-out scenarios inside the ranges (`rl_cap_train_v2`)

Question: with a training distribution that covers the axes v1 failed on, and timing calibrated across delay, does the agent beat the best fixed setting on new scenarios it was not trained or selected on?

Training distribution: cap C 4–24 MiB/s; one-way delay 10–120 ms; early and late rate dips; per-request timing calibrated at four delays (20, 40, 80, 120 ms) with four rate classes. Learner, actions and state

unchanged from v1. 5 seeds of about 320,000 steps each; **seed 1 selected on validation alone**; disclosed before any live run that the other seeds were mostly at or above the best fixed time on held-out scenarios.

Held-out scenarios (inside the ranges, new combinations, each with 10–17 % simulator headroom): H_C12 ($C = 12$ from 6 s); H_RTT110 (110 ms each way, cap from 12 s); H_DIP (cap from 6 s + dip 30–40 s); H_C20 ($C = 20$ from 4 s, 20 ms each way).

Calibration and blind check: 24 calibration runs showed client overhead rising with delay; pooled rate classes mispredicted one in-sample cell, so four classes were adopted before the blind check; all four blind predictions at 80 and 120 ms within ±8 % (+1.4, −3.3, +5.6, +5.6 %).

Table 9.3: v2 study results (8 blocks; ratios against the better fixed 1 MiB arm)

Scenario	Best fixed	v2	v1	drop rule
CAP8 (in dist.)	16 × 1	0.845 [0.840, 0.850]	0.910	0.807
HEALTHY	16 × 1	1.047 [1.043, 1.052]	1.096	1.002
H_C12	9 × 1	0.882 [0.877, 0.886]	0.970	0.911
H_RTT110	14 × 1	0.819 [0.795, 0.848]	1.137	1.097
H_DIP	14 × 1	0.952 [0.945, 0.958]	0.966	0.916
H_C20	8 × 1	1.007 [1.000, 1.014]	1.125	1.355

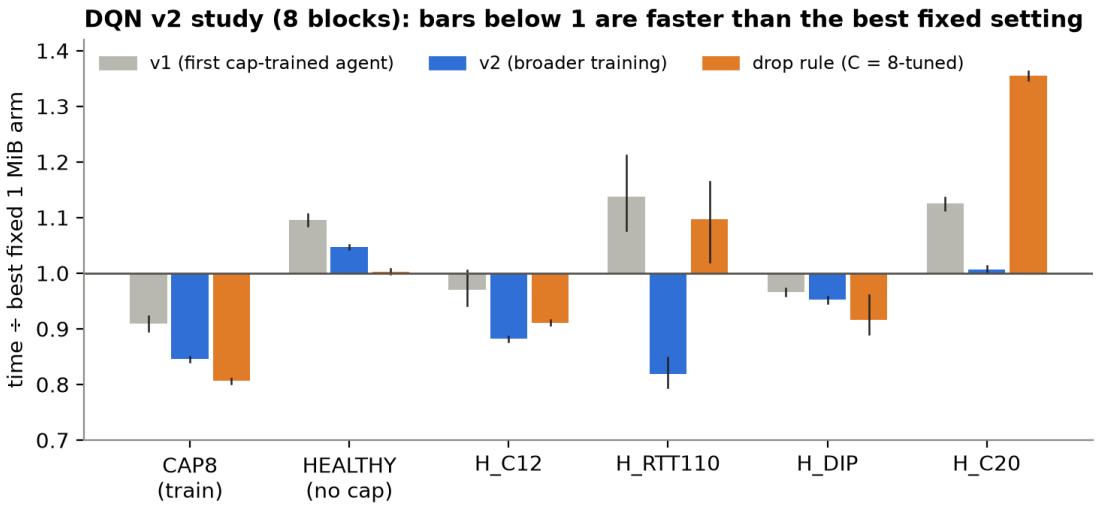


Figure 9.2: The v2 study: v1, v2 and the drop rule against the best fixed 1 MiB arm.

- **Held out:** v2 faster in 3 (H_C12, H_RTT110, H_DIP), slower in 1 (H_C20, 1.007; t-interval [0.998, 1.016] would say n.d., but the less favourable verdict counts). v1 on the same four: faster in 1, slower in 2. **v2 faster than v1 in all 6 live scenarios.**
- **In distribution:** CAP8 0.845 vs 16 × 1, 0.929 of v1. HEALTHY 1.047 (about 5 % cost).
- **Against the C=8-tuned drop rule:** v2 faster in 3 (H_C12 0.968, H_RTT110 0.746, H_C20 0.743), slower in 2 (CAP8 1.048, HEALTHY), n.d. in 1 (H_DIP). The rule's simulated time was already worse than fixed 16 × 1 in H_RTT110 and H_C20, so two of those wins reflect the rule's tuning more than learning.

- **Unexplained:** in H_RTT110, v2 ran 68.6 s live against a predicted 76.4 s (10 % faster than the simulator supports), while the fixed arms were 3–4 % slower than predicted.
- **Frozen verdict:** "v2 generalized partially: faster in 3, slower in 1 of 4 held-out scenarios, all inside the v2 training ranges (v1 on the same scenarios: faster in 1, slower in 2)". "Learned control adds something over a C=8-tuned drop rule: faster in 3, slower in 2 of 6 scenarios", with the caveat above.

Scope of the v2 claim

The comparison is with the best fixed setting at the same 1 MiB range size, under server-side changes, on one server model. Not 16 × 32 MiB, not aria2, not real servers. One selected seed from a procedure that is not reliably good across seeds. It costs about 5 % when nothing changes. "Generalizes" means new combinations inside the training envelope, not new kinds of conditions.

9.7 v2 against aria2 and large ranges (cross-study, not paired)

The v2 study did not run aria2 or 16 × 32 MiB (not among its Stage C/D arms; the protocol's claim scope excludes them). The joint study ran them on the same server and corrected engine, and its fixed 16 × 1 MiB time matched v2's (76.6 s vs 76.7 s), so the studies line up:

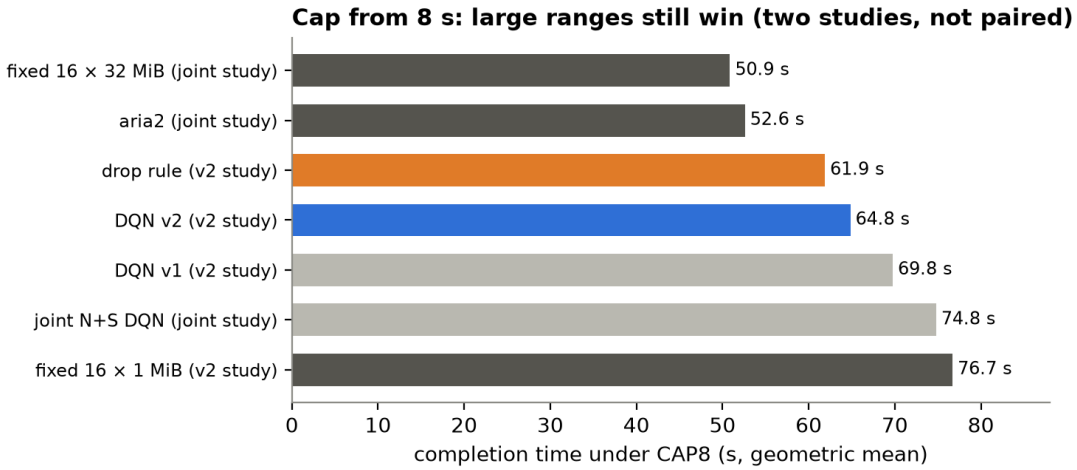


Figure 9.3: CAP8 completion times from the joint and v2 studies.

Under CAP8, v2 (64.8 s) was about 23–27 % slower than aria2 (52.6 s) and 16 × 32 MiB (50.9 s). With no cap, v2 (≈19.2 s) was far faster than aria2 (51.9 s).

9.8 Model-based controller attempt (Failed gate)

Online system identification followed by a single within-download joint (N, S) choice. The predeclared identifiability gate failed **0 of 6** criteria in simulation (560 rows): median regret vs grid oracle 0.54 (bar 0.05), median predicted-time error 0.46 (bar 0.10), total cost incl. probing 1.77× oracle (bar 1.25×), fallback to the healthy config in 64 % of rows (bar 30 %); an ablation with the *true* server concurrency also failed. No live evaluation was run; nothing was tuned afterwards.

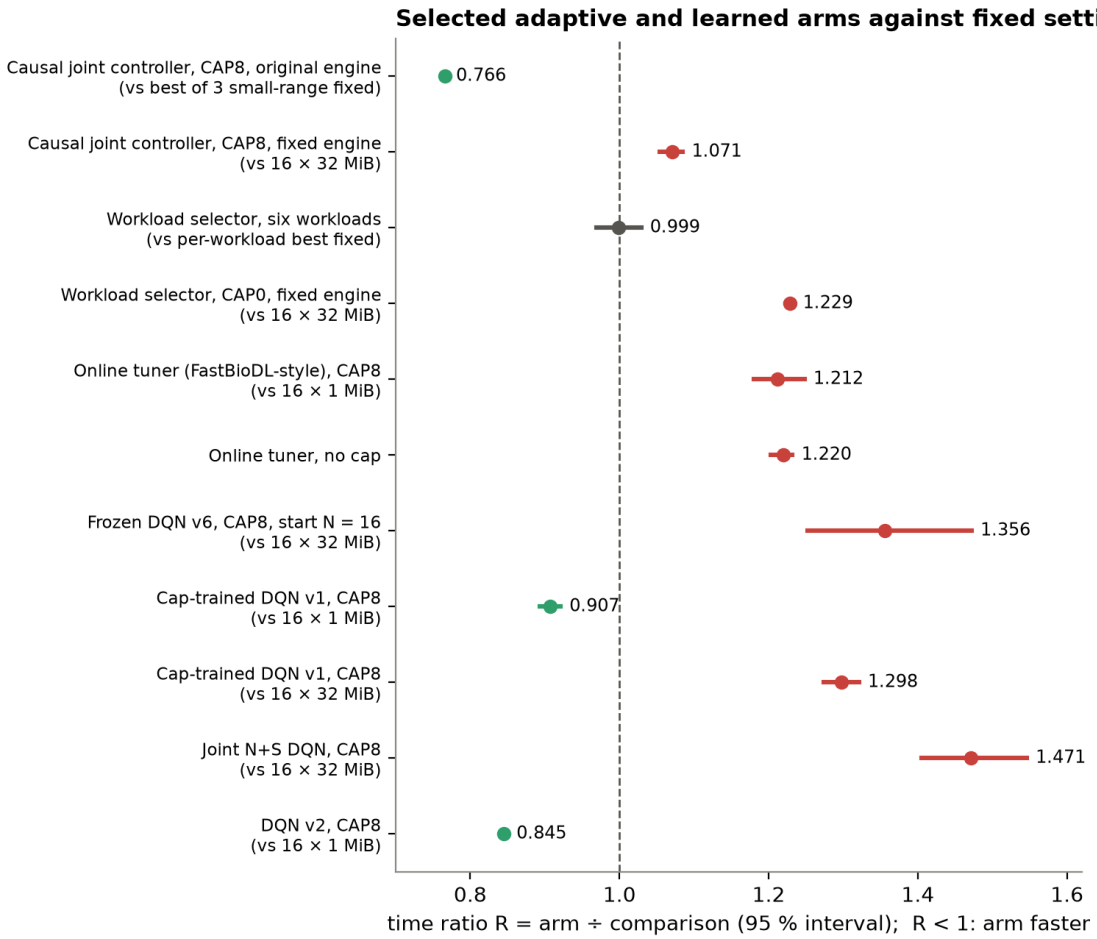


Figure 9.4: Selected adaptive and learned arms (primary CAP8-type comparisons) against their comparison fixed setting. Held-out and no-cap results are in the tables above.

10. Results V: Real Networks and Real Servers

10.1 Live-internet pilot (`live_pilot_l1`, Descriptive)

Question: on real paths, does the best fixed (N,S) change across servers and times of day, and how much would a clairvoyant per-condition choice gain over one global setting?

Design. One client on residential broadband in India (Tata Teleservices) downloaded `linux-3.0.tar.xz` (63,798,008 bytes, checked against kernel.org's published SHA-256) from four public mirrors: **kcdn** (cdn.kernel.org, CDN), **kedge** (mirrors.edge.kernel.org, origin), **riken** (ftp.riken.jp, Japan), **tuna** (mirrors.tuna.tsinghua.edu.cn, China). Arms: 8 fixed cells ($N \in \{1,4,8,16\} \times S \in \{1,4\}$ MiB), aria2 and a single-stream curl reference; 2 reps per cell per server; four time-of-day slots on the primary network (Sunday midday, Sunday evening, Monday evening, Tuesday morning). A *condition* is one (server, slot) pair: 16 conditions. Tsinghua's midday condition comes from a supplementary re-run slot, S1T (amendment A2), because the first slot's Tsinghua runs were refused. Protocol frozen 18 Oct before the first slot; amendments logged.

Results.

- **Validity:** 328 of 350 runs valid. Invalid: 16 runs the Tsinghua mirror refused with HTTP 403 in the first slot, and 6 during an internet drop (re-run in full, amendment A4). The fastest valid run reached 2.43 MB/s, the access link's ceiling.
- **Best cell per condition:** no single cell was top everywhere, but the top cell always had $N \geq 8$, except 4×1 MiB on the CDN in two slots and 4×4 MiB on Tsinghua in one. The top cell changed across slots on 4 of 4 servers. On the origin mirror the better $N = 4$ cell was 1.37–1.71$\times$ slower than the best in every slot.
- **Selection headroom:** one global cell, **8×1 MiB**, was on average **3.7 %** slower than each condition's clairvoyant best (bootstrap over conditions **[1.0, 4.8]**; 16 conditions). Within 5 % in 11 of 16 conditions; at most 15.3 % slower in any one. The upper bound is below the predeclared 5 % line.
- No condition was access-link-bound by the protocol's definition ($N = 1$ was always > 5 % slower than the best).
- Within-run variability: median coefficient of variation of 1 s throughput bins 0.11–0.29 across servers.
- aria2's time over the condition's best fixed cell: 1.00–1.26.

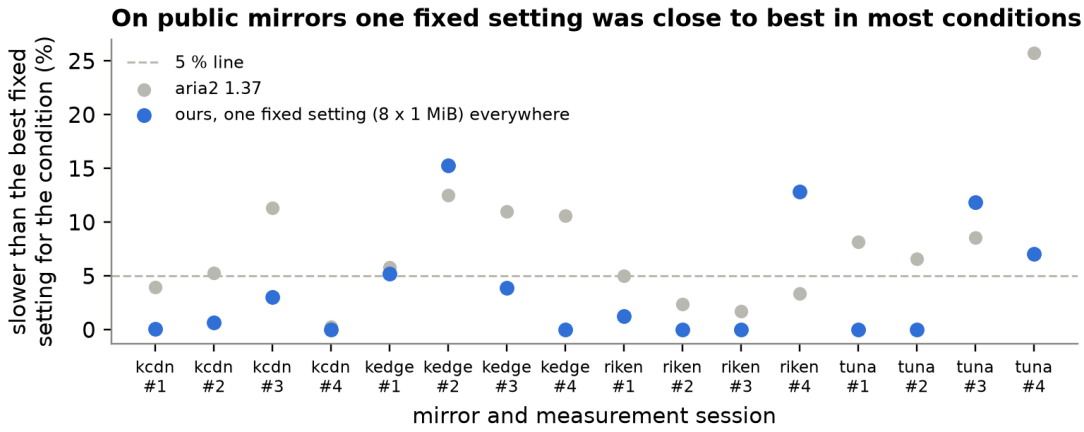


Figure 10.1: Live pilot: how much slower one global fixed setting and aria2 were than the per-condition best.

The accidental second network. One morning slot ran on a different, unidentified network (amendment A3), reported separately and never pooled: 68 of 80 valid (DNS and retry failures in about half an hour of instability); peak 12.3 MB/s; much higher variability (median CV 1.16–1.77); 8×1 MiB headroom **7.2 % [1.2, 12.2]** over 4 conditions. Inconclusive, but the one hint in the pilot that the best setting may move by more than 5 %.

Side finding. The Tsinghua mirror rejects the Go standard library's default User-Agent with HTTP 403 for every range request, while accepting curl, aria2 and an identifying User-Agent. A stock Go downloader is locked out. From the second slot on, our runs used an explicit identifying User-Agent (amendment A2); in the first slot the other mirrors saw Go's default, an uncontrolled change the protocol records.

Be careful how you say this

One client, one primary access network, about two days, two reps per cell, fixed cells only. It is screening evidence that a fixed setting with enough connections was near-best on the observed paths. It is not a validation across networks.

10.2 Adaptive arms on real mirrors (`live_mirror_check`, Descriptive)

Question: do the v2 agent and the drop rule *cost* anything on public mirrors whose behaviour we did not design? With 3.7 % headroom, this is mainly a **no-harm check**, not a search for wins.

Design. kcdn, kedge, riken; same file and etiquette as the pilot; corrected engine (no deadline, splitting) in every arm; 1 MiB ranges. Arms: `v2` (start 16, window 16), `drop` (start 16, $k = 3$, ratio 0.5, capped $N = 4$), fixed 16×1 MiB and 8×1 MiB. 5 blocks per server, shuffled (seed 20261101): 60 runs per slot.

Rule fixed in advance: "no harm on server X" if the arm's ratio against the better fixed arm is not "slower" *and* its point estimate is at most 1.05. Summary sentence: "On k of 3 public mirrors, the v2 agent showed no harm against the best fixed 1 MiB setting (and the drop rule on j of 3)."

10.2.1 What happened

- **Amendment A1:** range-probe attempts fixed at up to 3 of 20 s; kedge timed out once, then answered `206` in 9.8 s.

- **Slot M1 lost to a client-side outage.** From 22:10 UTC every attempt to all three servers (including the CDN) failed with no HTTP response; only 20 of 60 runs valid. Reported in full, not used (amendment A2).
- **Link gate (A2):** slot M2 starts only if all three probes and a google.com fetch respond within 3 s and an 8 MB single-stream kcdn fetch reaches 1.5 MB/s. It passed at 23:05 UTC.
- **Slot M2: 60 of 60 valid.**

Table 10.1: Mirror check slot M2 (ratios against the better fixed arm, 16 × 1 MiB on all three; 95 % bootstrap)

Mirror	v2	drop rule	v2 median final N	drop switched
kcdn	1.401 [1.102, 1.773] slower	1.125 [0.868, 1.456]	6	1 of 5
kedge	1.654 [0.873, 2.849]	1.063 [0.824, 1.553]	7	4 of 5
riken	1.215 [0.719, 1.899]	1.332 [0.917, 2.017]	14	4 of 5

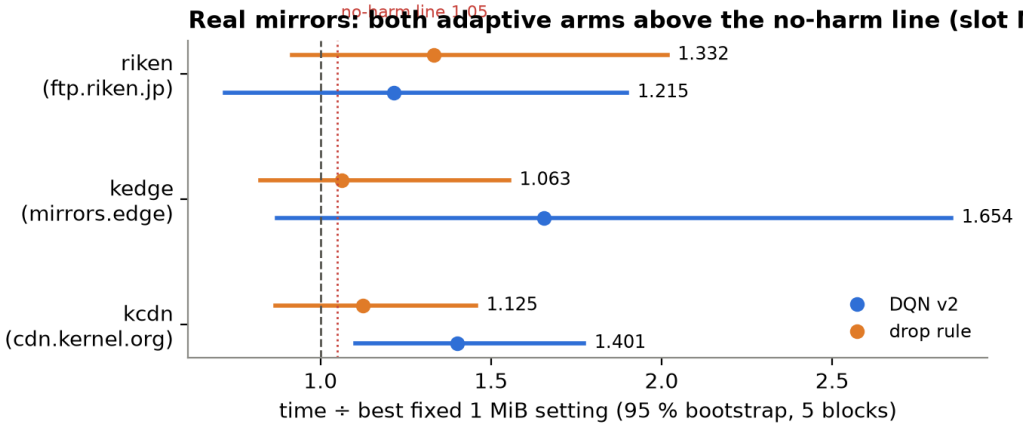


Figure 10.2: Real mirrors: both adaptive arms above the 1.05 no-harm line on all three.

Frozen sentence: "On **0 of 3** public mirrors, the v2 agent showed no harm against the best fixed 1 MiB setting (and the drop rule on **0 of 3**)." Only the kcdn v2 bootstrap interval excludes 1 (its t-interval is [0.950, 2.067]; "slower" follows the predeclared less-favourable rule); the others fail the 1.05 point-estimate rule without being clearly slower.

10.2.2 Why

The mirrors have no cap, but their per-request throughput is noisy. v2 ended at a median N of 6–7 on kcdn and kedge; the drop rule switched to $N = 4$ in 4 of 5 runs on kedge and riken. The agents adapt to signals that, on these servers, do not mean what they meant on the server model they were built for. Adaptation carried a cost here, with little headroom to pay for it (3.7 % selection headroom in the pilot).

"They mistook noise for a cap"

This is our interpretation. What was measured is that both arms cut connections on uncapped servers and lost time. We did not analyse the throughput traces to show that noise triggered each cut. One client, one evening, 5 blocks: descriptive only.

“Is v2’s loss on the mirrors because it was trained badly?”

Partly, but not mainly. (1) The drop rule involves *no* training and lost the same way. (2) The pilot found little selection headroom (3.7 %), so even a perfect agent had little to gain. (3) Where training does share the blame: v2 trained on uncapped episodes too, but only with timing noise calibrated on our loopback server, not the far noisier per-request throughput of public mirrors, so the mirrors were out of distribution. Better training (noisy uncapped scenarios, a “hold unless the drop persists” rule) could plausibly remove the harm, but it cannot create a win where there is no headroom.

11. Synthesis: What It All Means

11.1 Why reinforcement learning did not win

Key idea

The learner was not the bottleneck; the opportunity was.

1. **The ceiling was low.** On one bottleneck path the best N is "enough connections to fill the pipe". When capacity changes you go faster or slower, but the same setting stays best. The cellular oracle (0.41 % mean, lower bound -0.26 %) bounds any concurrency-switching policy, learned or not, under the frozen trace model; the live pilot bounds per-condition selection among fixed cells (3.7 %).
2. **Simulator gains did not always transfer.** L4 showed 17.6 % headroom in simulation that the live server did not reproduce. Every later simulator was checked blind; where it was accurate (joint study), the learner still failed to find the headroom a simple rule found.
3. **Where real headroom existed, a one-threshold rule captured it.** The causal joint controller (23.4 % less time than small-range fixed settings on the original engine; its N-only version 0.808) and, in v2's study, the C=8-tuned drop rule (0.807 in CAP8) did at least as well as learners in their home scenarios.
4. **Engine details came first.** Removing the 30 s deadline and adding straggler splitting changed the ranking more than any controller did.
5. **Learned agents were narrow.** v1 learned its training server and did not generalize; v2 generalized partially within its envelope; on real mirrors both learned and rule-based adaptation cost time.

11.2 What we claim and what we do not

Table 11.1: Claim map (from the paper; "label" is the evidence strength)

Claim	Label
Fixed 16×4 MiB beats aria2 by 62.4 % on D2 (30/30, 0.3756)	Confirmed (one workload)
The D2 gap is partly granularity $\times$ request-start timing $\times$ pacing	Confirmed for 0.862; residual unexplained
Ours faster than aria2 across six schedules except constant slow (parity)	Descriptive screen (28/30)
Ours faster than aria2 against stock NGINX per-response limits	Descriptive screen (10/10); N1 parity expectation not met
Our 1/4 MiB settings beat axel and lftp too (no cap, D2)	Screen (0.345–0.375)
D2 advantage holds across delay and loss	Screen (9 of 9 cells)
. . . and the capped-server parity does	**Not established**

Table 11.1: Claim map (from the paper; "label" is the evidence strength)

Claim	Label
No exploitable concurrency headroom on replayed cellular traces	Confirmed under frozen model; Failed gate
Soft cap creates a state-dependent joint optimum (live)	Descriptive; **Failed gate** (S-margin)
Causal controller beats best stationary config within a download	Confirmed (best of 3, original engine)
. . . and beats aria2	**Not supported** (1.096)
Selector matches per-workload best fixed on average	Descriptive (0.999; not equivalence)
Selector improves on one family-wide config	**Not established**
aria2's lead under the cap came from the deadline and no splitting	Confirmed (230 runs)
Adaptive controllers beat best fixed on the corrected engine	**Not supported** (1.071; 1.229)
Online tuner beats best fixed	**Not supported** (1.212; 1.220)
Frozen v6 beats best fixed on the capped server	**Not supported** (1.356)
Cap-trained DQN beats best fixed 1 MiB where trained	Confirmed (0.907)
. . . generalizes to held-out conditions	**Not supported** (0 of 3)
. . . beats best fixed overall or aria2	**Not supported** (1.298; 1.310)
Joint N + range-size agent beats best fixed	**Not supported** (1.471; 4 of 4 slower)
Broadly trained v2 beats best fixed 1 MiB on held-out scenarios inside its ranges	**Partial** (3 of 4; one seed)
. . . and beats a simple drop rule	**Mixed** (3 vs 2 of 6)
One fixed cell (8×1 MiB) near-best on live mirrors	Descriptive pilot; criterion met ($4.8\,\% < 5\,\%$)
. . . on a second access network	Descriptive, inconclusive ($7.2\,\%$)
v2 and the drop rule do no harm on public mirrors	**Not supported** (0 of 3)
Learned policies beat aria2	**Not supported**

11.3 What is novel in this project

Start with what is *not* new

Adaptive concurrency and segment sizing are established (FastBioDL, Falcon, direct search, Liu et al.'s deep RL, aria2/IDM segment splitting). We do **not** claim a new algorithm, and we do not claim adaptive segmented downloading is our invention. Review 1 positioned the project "as an engineering and experimental contribution rather than a claim of inventing adaptive segmented downloading", and

Review 3 says the same.

Our novelty is in the **question, the method and the findings**:

1. **A different question.** Prior work asks "how do we tune N online?". We ask "*when is there anything for any controller to exploit?*" and answer it with oracle bounds, so within the action space and model each oracle covers, the answer holds for every policy, not only ours.
2. **Headroom gates before learners.** In the trace study and the later learner studies we computed clairvoyant headroom against a bar fixed in advance, *before* training, and blind-checked the simulators' fixed-setting predictions against live runs (L4 showed why: its simulated headroom did not exist live). This separates "the environment offers nothing" from "the learner failed". The cellular result ($-0.26\,\%$ lower bound with 5–27$\times$ capacity swings) is a learner-independent negative result.
3. **A partly mechanism-explained comparison with aria2.** Much of the 62.4 % gap is traced, with per-request server logs and controls, to an interaction between range granularity, request-start timing and per-response pacing (matched-granularity control 0.862; a residual is unexplained). The direction also held against axel and lftp (screen, 5 blocks) and across a 3×3 delay–loss grid (screen, 3 blocks per cell). We show it is *not* adaptation.
4. **"Engine before policy".** A 230-run ablation shows that two engine details (a whole-request timeout and missing straggler splitting) explained aria2's lead under a concurrency cap, and that once fixed, the best fixed setting beat the adaptive controllers tested in the ablation (causal 1.071, selector 1.229), while a re-implemented FastBioDL-style tuner lost even to fixed 1 MiB settings (1.212, 1.220). We are not aware of prior download-tuning work that isolates engine details from policy this way; say "to our knowledge" in the viva.
5. **The request-start rate rule as a hidden confounder.** Both headline effects (D2, and large ranges under the cap) depend on a server fixing a response's rate on arrival. We built a live-rate control server to test it: fixed 16×32 MiB under the cap slowed from 50.9 s to 67.4 s. Anyone benchmarking downloaders against rate-limited servers should know this.
6. **A complete, honest inventory of learned controllers.** The early ones (LinUCB, DQN v1–v6) are reported as negative or descriptive; the later ones (cap-trained v1, joint, v2) were each tested against the best fixed setting with frozen protocols, blind simulator checks, validation-only seed selection and held-out scenarios with headroom, and v2 also against a simple rule. Failed gates are reported as failed. Positive results (v1 0.907, v2 3 of 4) are reported with their exact scope.
7. **Real-server no-harm testing of adaptive arms**, showing the cost side of adaptation: on three uncapped mirrors both adaptive arms failed the predeclared no-harm rule (point estimates 1.06–1.65 against fixed 16×1 MiB; only kcdn v2 clearly slower; one client, one evening, 5 blocks).
8. **A reusable platform and a verifiable evidence chain**: an instrumented downloader with one policy interface, a server with explicit rate rules, and a claims checker that recomputes all 220 paper numbers from raw artifacts and rejects wrong numbers in a self-test.

The 20-second novelty answer

"We don't propose a new tuning algorithm. Our contribution is to measure *when* adaptation can help at all, with oracle bounds computed before training, and to explain the results mechanistically. We found the opportunity is usually near zero; that a large gap against aria2 is a request-timing effect, not adaptation; that engine details mattered more than any controller; and that learned agents win only narrowly, inside their training envelope."

11.4 Practical lessons

- **For download-manager builders:** pick enough connections for the server (in the pilot 8×1 MiB was near-best on the mirrors we measured, though in the later mirror check 16×1 MiB was faster on all three); use straggler splitting; use a stall-based timeout, not a whole-request deadline; small-enough ranges keep you responsive to rate changes. Adaptive control is unlikely to pay unless the server penalises concurrency in a way that changes over time.
- **For researchers:** measure headroom first; compare against the best fixed setting and a simple rule, not a weak default; check simulators blind; make server rate semantics explicit.

11.5 Future work

- Action spaces with delayed, changing costs: multi-mirror scheduling, restarting slow requests, straggler hedging. Each must pass a headroom gate first and beat both the best static setting and a strong myopic heuristic.
- Live measurements across many clients, networks and mobile paths.
- Training on noisy, uncapped server behaviour (from the mirror check), and a "hold unless the drop persists" guard.
- Make straggler splitting the default and add a stall-based timeout, after validation on real servers.
- HTTP/2 and multi-source downloads; a PPO hybrid; IDM on Windows.

11.6 Publication status

The paper (*Request granularity and concurrency adaptation for HTTP range downloads under controlled server-side pacing: a mechanism-first evaluation*) targets *IEEE Access*. The content is complete in `reports/paper_draft.md` and all 220 numbers are verified. Still open: conversion to the IEEE Access LaTeX template; author block, affiliations, ORCID, funding and conflict-of-interest statements; confirming the AI-use disclosure wording against the publisher's policy; a data DOI; and the redistribution terms for the Mahimahi-bundled traces. (The Mahimahi, Sprout, LinUCB, DQN, Double DQN and netem citations were verified on 18 Oct; the IEEE package README is out of date on this.) Nothing has been submitted.

12. Limitations and Threats to Validity

Know these by heart: an examiner who finds a limitation you already state loses the point.

1. **Constructed environment.** The positive adaptive results hold under a server model we designed and selected (72-candidate search). They show conditions under which adaptation can help, not that real servers behave this way.
2. **Request-start rate semantics** drive both the D2 mechanism and the large-range advantage under the cap. The live-rate control confirmed the cap case (50.9 s $\rightarrow$ 67.4 s).
3. **One machine, loopback + netem, one aria2 version (1.37.0)** for every controlled result. Client and server share a laptop.
4. **Real paths only in descriptive studies:** the live pilot (one client, one primary network, plus one accidental slot on a second; fixed cells only) and the mirror check (three mirrors, one evening, 5 blocks). Not a validation across networks.
5. **Stock servers:** only the NGINX per-response screen ($n = 5$), which exercises neither a shared cap nor a connection limit. The adaptive results have no stock-server counterpart.
6. **Traces are replayed**, not live mobile networks; the oracle bound holds under a frozen counterfactual model.
7. **Learned-policy generalization is narrow:** v2 holds inside its training ranges, for one seed selected on validation from a procedure not reliably good across seeds, and only against fixed settings at the same range size.
8. **Engine before policy:** earlier adaptive results (P1, P2, W1–W6) were measured on the original engine. Removing the deadline means a fully stalled connection would wait for the run limit; a stall-based timeout is future work.
9. **Statistics:** block-level intervals reflect emulator noise; workload-level intervals are primary for family claims; $n = 5$ screens are screening only; no multiplicity correction was applied across the many comparisons.
10. **Multiplicity and amendments:** W1–W6 was selected from 72 analytic candidates and amended twice before live runs; protocols are predeclared and hashed but not independently timestamped.
11. **One configuration per controller**, fixed before its runs; other parameters might do better, though the small headroom limits how much.
12. **Fairness:** 16 connections take more of a shared link and load public mirrors more; we measured speed, not fairness.
13. **Two learner families** (LinUCB, Double DQN). No PPO; no model-based control beyond the gated system-identification attempt (which was not RL).
14. **Unexplained residuals:** ours is 14 % faster than aria2 even with matched granularity, and faster at constant fast pacing; v2 beat its simulator prediction by 10 % in H_RTT110 (fixed arms 3–4 % slower than predicted). We say these are unexplained.

12.1 Incidents we disclose

- **Mirror slot M1** lost to a client-side outage; reported, not used; a link gate added for M2.
- **Live pilot**: an internet drop (re-run in full), the Tsinghua 403, and the accidental second network.
- **Joint study**: the first training run was undertrained (ended at $\varepsilon \approx 0.75$); retrained by amendment, not evaluated.
- **v1 study Stage D**: the protocol said 80 runs; 65 were run (an arithmetic slip, amendment A1).
- **Checker incident**: during the final paper update the claims checker briefly truncated four committed analysis files on disk (they were committed empty, then restored byte-identical from the previous commit, and the checker was fixed). No results changed.
- **Frozen analysis scripts** used a fixed $t = 2.045$ for every n and once a normal 1.96; the paper reports corrected intervals; the frozen artifacts were not modified.
- **Early wording**: "40 ms RTT" was wrong (it is 40 ms each way, 80 ms RTT); corrected.

13. Doubt Clearing: Viva Questions and Model Answers

Answers are short first, then detail. Practise saying the first sentence of each answer out loud. Numbers are from the paper and study folders.

13.1 The basics

Q1. In one minute, what is your project?

Answer. We built an instrumented segmented HTTP downloader in Go whose connection count N and range size S a policy can change mid-download, plus a controlled test platform, and asked *when* adapting N and S actually beats a well-chosen fixed setting. Answer: rarely. A fixed setting beat aria2 by 62.4 % on a paced server for a request-timing reason, not adaptation; recorded cellular traces showed almost no headroom even for an oracle; adaptation helped only on a server we built where concurrency has a time-varying cost, and even there engine fixes mattered more; learned agents won only narrowly against same-range-size fixed settings on the server they were trained for, and cost time on real mirrors.

Q2. What are N and S, and why these two?

Answer. N is the number of parallel range requests; S is the bytes per request. They are *the* dials of segmented downloaders (aria2's `-x`, `-s`, `-k`; IDM's connection count). Narrow scope is what lets a controlled study attribute effects.

Q3. What is a range request? What does your downloader check?

Answer. `Range: bytes=a-b`; the server must reply `206` with `Content-Range`. We reject `200`, wrong length, or mismatched `Content-Range`; we probe with `bytes=0-0` first; we verify the final SHA-256.

Q4. Why half-open ranges internally?

Answer. Length is $e - s$, adjacency is $e_1 = s_2$, and there is one conversion point to HTTP's inclusive form, which removes off-by-one errors.

Q5. What happens when a request fails?

Answer. The same range is retried with the same bounds up to `MaxRetries`; ranges are never resized in flight except by an opt-in straggler split.

Q6. What happens if the policy lowers N mid-download?

Answer. Nothing is cancelled: extra workers retire after their current range finishes.

13.2 Method and statistics

Q7. Why the geometric mean of ratios?

Answer. Ratios are scale-free, and the geometric mean treats "half the time" and "twice the time" symmetrically; the arithmetic mean of 0.5 and 2 is a misleading 1.25.

Q8. What is a block? Why paired?

Answer. One run of every arm in a seeded random order under the same conditions. Pairing within blocks cancels slow drift of the machine and network.

Q9. How are intervals computed?

Answer. 95 % percentile bootstrap over blocks, 10,000 resamples with a seed fixed per study, on the log ratios. (The D2 headline interval is a t-interval.) In later studies also a paired t-interval, and the less favourable verdict is used.

Q10. Your intervals are extremely narrow. Isn't that suspicious?

Answer. The emulated server is nearly deterministic, so the intervals measure testbed noise, not real-world variability. We say so. On the real mirrors the intervals are wide, e.g. [0.873, 2.849].

Q11. "Not distinguishable" means equal, right?

Answer. No. It means the interval contains 1. It is not an equivalence test.

Q12. Did you pre-register?

Answer. We predeclared every study in a logged protocol with recorded hashes before its runs, with dated amendments. They were not deposited with an independent timestamping service, and one manifest (W1–W6) was amended twice after design output was seen. So we say "predeclared", not formally "pre-registered".

Q13. How do we know your numbers are right?

Answer. `tools/claims_check.py` recomputes all 220 numbers in the paper from raw run files, each tied to its sentence, and a self-test shows it rejects wrong numbers. It caught real errors in our frozen scripts (a fixed t value for every n), which the paper corrects.

Q14. No multiple-comparison correction?

Answer. Not in the later studies (the v5 head-to-head used Holm across 8 conditions), and we state it as a limitation. Confirmatory claims are few and predeclared; most others are labelled screens or descriptive.

13.3 The aria2 result

Q15. Is the 62.4 % win real?

Answer. Yes on that workload: 30 of 30 pairs, $R = 0.3756\,[0.3726, 0.3787]$. It comes mostly from range granularity and the server's request-start rate rule: aria2's 32 MiB pieces all start in the slow phase and stay slow; 112 of our 128 4 MiB ranges start after the speed-up. With matched granularity the ratio is 0.862.

Q16. So your downloader is better than aria2?

Answer. No. It is a fixed-configuration result on one server mechanism. Where the timing effect is absent (constant slow rate, the cap with splitting, public mirrors) the tools are level or close.

Q17. Why not configure aria2 with smaller pieces?

Answer. With `-x16`, `-split` of 16, 64 or 256 gave the same range-size distribution in our probe. aria2's flags were chosen once from a discovery sweep and kept fixed.

Q18. Is this specific to aria2?

Answer. No: axel and lftp split the file once up front too, and ours took 0.345–0.375 of their time without a cap and on D2. Under CAP8 all four were level.

Q19. Why not IDM or wget2?

Answer. IDM is closed source and Windows-only, and cannot be scripted for hundreds of randomised runs in our Linux testbed. wget2 fetches one file over a single connection however it is configured (its source, and upstream issue #195).

Q20. Did you check a server you did not write?

Answer. Yes, stock NGINX `limit_rate`: ours faster in 10 of 10 pairs (0.863, 0.597). Our predeclared parity expectation for constant per-response limits was *not* met, and we did not find why.

Q21. What part of the gap is unexplained?

Answer. With aria2-like pieces ours is still 14 % faster, and at constant fast pacing 15.4 s vs 27.1 s. A candidate is the finishing tail of 32 MiB pieces, untested.

13.4 The search for an opportunity

Q22. Networks change all the time. Why don't fixed settings lose?

Answer. On one bottleneck path the best N is "enough to fill the pipe". When capacity drops, everything slows, but the same setting is still about right. Adaptation pays only when *too much concurrency has a cost that changes over time*.

Q23. Explain the cellular-trace result.

Answer. The recorded traces swing about 5–27$\times$ in capacity (p90/p10 across the Mahimahi set). A clairvoyant oracle switching N at epoch boundaries, with measured switching costs, gains 0.41 % on average over the best fixed N; the one-sided 95 % lower bound is -0.26 % against a predeclared 10 % bar. Since the oracle is the best any policy could do, this bounds every concurrency-switching policy under that model.

Q24. Isn't it replay, not a real network?

Answer. Yes, replay of real recorded traces through Mahimahi, which makes it reproducible. We say it is not a live mobile measurement.

Q25. What other mechanisms did you screen?

Answer. Static rate + RTT, shared-bottleneck queueing with background traffic, correlated loss, per-request throttling, failure-based throttling, range size across phases, joint phase pacing. None produced a state-dependent optimum that survived live grounding, except the soft cap we constructed.

Q26. What was L4?

Answer. An environment design with 17.6 % simulated headroom that passed its gate; a live sweep showed high concurrency is not penalised live, so the headroom was a simulator artefact. We stopped instead of training on it.

13.5 The capped server and the engine

Q27. You built the server model yourselves. Isn't the positive result rigged?

Answer. It is constructed, and we say so: it shows the conditions under which adaptation *can* help, not that real servers behave this way. The family was selected from 72 candidates, the amendments are disclosed, the gate's S-margin failed and stayed failed, and the cross-family claim is not established.

Q28. Explain the cap formula.

Answer. A new request gets $\min(\text{base}, C/(k(1+\gamma(k-1))))$, where k counts the active requests including itself, $C = 8$ MiB/s, $\gamma = 0.05$. The implied total $C/(1+\gamma(k-1))$ falls with k (about 7.3 MiB/s at $k = 3$, 4.6 at 16), so extra connections cost. The rate is fixed at arrival.

Q29. What did the causal controller achieve?

Answer. With the cap from 8 s, 58.7 s vs 76.7 s for the best of three fixed settings ($R = 0.766$, 10/10). The N move carried it (N-only 0.808; S-only 1.070, worse than not switching). aria2 was faster (53.6 s).

Q30. aria2 beat you under the cap. Doesn't that undermine the result?

Answer. We investigated and it changed the conclusion. A 230-run ablation traced aria2's lead to two engine details in our downloader: no straggler splitting (with splitting, 16×32 MiB took 0.599 of the time and matched aria2, 0.995) and the 30 s whole-request timeout (without it, within 1.6 % of aria2 with the cap from 8 s).

Q31. You fixed the engine. Did adaptation still help?

Answer. Not against the best fixed setting on the fixed engine: the causal controller was 7 % slower than 16×32 MiB (1.071), the selector 23 % slower under CAP0 (1.229). The +23.4 % holds only against small-range fixed settings on the original engine. "Engine before policy."

Q32. Why does 16×32 MiB win under the cap?

Answer. It starts all 16 ranges before the cap at 8 s, and each keeps the pre-cap rate for its whole length; splitting then hands the tail to idle connections. With 1 MiB ranges almost every request starts after the cap.

Q33. Isn't that a quirk of your server?

Answer. Yes, and we tested it: on a live-rate server that re-checks rates during each response, 16×32 MiB under the cap slowed from 50.9 s to 67.4 s.

Q34. Why did the online tuner lose?

Answer. It reacted to the cap in 2.1 s, but needed 8.1 s to climb from $N = 4$ to 16 without a cap; that ramp is most of its loss (1.212 and 1.220 of the better fixed 1 MiB setting).

Q35. Was slow reaction the problem?

Answer. No: the rule controllers we timed reacted within seconds (causal 1.5 s and tuner 2.1 s after the cap; the selector classified at 4.0 s). The limit is what reacting can gain.

13.6 Reinforcement learning

Q36. So RL failed. Why should we care?

Answer. The question was never "can RL win?" but "when is there anything for any controller to exploit?". Our strongest negative results are learner-independent oracle bounds. That is useful for anyone designing adaptive downloaders.

Q37. Why did RL lose?

Answer. Low ceiling (oracle bounds), simulator gains that did not transfer (L4), and a one-threshold rule capturing the real headroom where it existed. A better learner cannot beat a near-zero ceiling.

Q38. Describe your DQN.

Answer. (Cap-trained agents.) Double DQN, MLP $13 \to 64 \to 64 \to 3$ (ReLU), 13 download features, actions {none, ± 1 connection}, decision after each completed request, reward $-\Delta t$ (return $\approx$ minus completion time), $\gamma = 0.99$, lr 5×10^{-4}, replay 100k, batch 64, ε 1.0→0.05 over 150k steps, target sync 1,000, Huber loss with gradient clipping. The joint agent had 5 actions and $\gamma = 1$. Trained in PyTorch on a simulator; weights exported to JSON; inference in Go; a parity test checks Go matches Python.

Q39. Why Double DQN and not PPO?

Answer. After LinUCB locked onto one action, we needed credit assignment over time; Double DQN reduces overestimation and was carried through to live evaluation. PPO is future work; we tested two learner families, not all.

Q40. Why did LinUCB lock onto one arm?

Answer. With small files there were very few decisions after warm-up, and with a myopic reward one action quickly dominated; periodic exploration changed coverage but not the preference. It was descriptive only.

Q41. Why did DQN v2/v5/v6 lose to aria2?

Answer. Short files (4–16 MiB) made start-up dominate, and v6 started from 1 connection while aria2 opened 16 at once. v5 was limited to 4 connections against aria2's 16 (v2 was compared with aria2 at 4 connections). In v6's long-file diagnostic the pooled gap shrank (1.805 → 0.909) but A2 and E2 got worse.

Q42. Why not let the agent choose range size too?

Answer. We did, under a frozen protocol (`rl_joint`). A hand rule took 0.352 of the best fixed time under CAP8 in simulation; the agent did not find it, and was slower than the best fixed setting in all four live scenarios (CAP8 1.471). The simulator predicted live times within about 5 %, so the learner, not the model, failed.

Q43. You trained the DQN for the capped server. Isn't that tuning to the test?

Answer. That is why each study has in-distribution and held-out stages, predeclared. v1 won in distribution (0.907) but in 0 of 3 held-out scenarios outside its training ranges. v2 was then trained more broadly and tested on four new held-out scenarios inside its ranges, each with 10–17 % headroom, selected on validation only.

Q44. Does the learned agent generalize?

Answer. Partly, within its training envelope: v2 was faster than the best fixed 1 MiB setting in 3 of 4 held-out scenarios (0.882, 0.819, 0.952), slower in 1 (1.007); faster than v1 in all 6; about 5 % slower with no cap. Caveats: one seed, the drop rule beat it in its home scenarios, and "generalizes" means new combinations inside the envelope.

Q45. Can the agent beat 16×32 MiB or aria2?

Answer. We said in advance it cannot: a 1 MiB N-only agent cannot keep pre-cap rates. Even the perfect switch in our simulator (64 s) is slower than the 52.6 s reference for 16×32 MiB written in that protocol (53.7 s live in that study). Cross-study, v2 under CAP8 (64.8 s) was about 23–27 % slower than aria2 and 16×32 MiB.

Q46. Did v2 beat aria2 and the fixed settings on your synthetic server?

Answer. It beat the best fixed setting at its own range size in its training scenario and in 3 of 4 held-out ones. It does not beat aria2 or large ranges under the cap; with no cap it is far faster than aria2 (19.2 s vs 51.9 s), as every 1 MiB setting is.

Q47. Why compare against the best fixed setting, which no one knows in advance?

Answer. It is the strongest honest baseline: if adaptation cannot beat the setting a sensible engineer would pick, it is not worth its complexity. On the mirrors, one global setting was on average 3.7 % slower than the per-condition best, so knowing conditions in advance was worth little anyway.

13.7 Real networks

Q48. What about real networks? Everything is loopback.

Answer. Two real-network studies. The live pilot (four public mirrors, four time slots): one global setting, 8×1 MiB, was on average 3.7 % [1.0, 4.8] slower than each condition's best (at most 15.3 %). The mirror check ran v2 and the drop rule on three mirrors: both showed no harm on 0 of 3 (point estimates 1.06–1.65); only v2 on kcdn was clearly slower (1.401 [1.102, 1.773]). Both are descriptive (one client, one primary network).

Q49. Did you run the adaptive agents on real servers?

Answer. Yes, a frozen no-harm check. The first slot was lost to an outage on our own link and is reported. On the clean rerun (60 of 60 valid), no harm on 0 of 3 mirrors for both: v2 1.2–1.65$\times$ (only kcdn clearly), drop rule 1.06–1.33$\times$. The agents cut connections on uncapped servers whose throughput is noisy.

Q50. Is that because v2 was not trained properly?

Answer. Partly, not mainly. The untrained drop rule failed the same way; with about 3.7 % selection headroom even a perfect agent had little to gain. v2 trained on uncapped episodes, but only with our loopback server's timing noise, so the mirrors were out of distribution; better training could remove the harm, not create a win.

Q51. What surprised you most?

Answer. The cellular traces (massive capacity swings, almost no headroom) and that a major mirror (Tsinghua) rejects Go's default User-Agent with HTTP 403, so a stock Go downloader is locked out.

13.8 Novelty and value

Q52. What is novel here?

Answer. Not a new algorithm. The novelty is measuring *when* adaptation can help at all with oracle bounds computed before training; a mechanism-explained aria2 comparison; showing engine details mattered more than any controller; exposing the request-start rate rule as the driver of large-range wins (with a live-rate control); and an honest, predeclared inventory of learned controllers with held-out tests and real-server no-harm checks. See Section 11.3.

Q53. Isn't this obvious?

Answer. The framework is intuitive, but the findings were not: about 5–27$\times$ capacity swings with under 1 % mean oracle headroom; a 62.4 % gap from request timing; aria2's cap lead from two engine details; simulator headroom vanishing live; a one-rule controller capturing the real headroom.

Q54. Is a negative result publishable?

Answer. Yes, with this rigour: predeclared protocols, failed gates reported as failed, reproducible numbers. It suits venues that judge soundness, such as IEEE Access. The framing is "when does adaptive downloading help?", not "RL for downloading".

Q55. Is the paper submitted?

Answer. Not yet. The content is complete and number-checked; the IEEE Access template conversion and the author-provided items (author block, ORCID, funding, COI, AI-use wording, data DOI) are open.

Q56. What would you do with more time?

Answer. Action spaces with delayed, changing costs (multi-mirror scheduling, restarting slow requests, hedging), live measurements at scale, noisy-server training. Each must pass a headroom gate first and beat both the best static setting and a strong myopic heuristic.

Q57. Did you use AI tools?

Answer. Yes, an AI coding assistant was used to write and modify code and analysis scripts, run experiments and tests, audit results against raw data and draft text; it generated no data except by running the specified programs, and every number is recomputed from raw runs. The disclosure is in the paper's declarations; answer honestly.

13.9 Design choices examiners like to probe

Q58. Client and server run on the same laptop. Doesn't CPU contention bias the results?

Answer. It can add noise, which is why arms are interleaved in seeded random order within blocks (both arms in a block see the same machine state), heavy jobs were kept off the machine during timing-sensitive runs, and one exception (v2 Stage A under external load) is disclosed. It is listed as a limitation: one machine, loopback.

Q59. Review 1 promised resume support and a fixed control interval T. Why were they dropped?

Answer. Resume was outside the evaluated scope (every experiment needs a fresh download). Decisions happen after each completed request instead of every T seconds because the live runner decides per attempt; the DQN simulator was aligned to that cadence after a mismatch in v5. Both are listed as deviations.

Q60. Why HTTP/1.1 and not HTTP/2?

Answer. Scope fixed in Review 1: one request per connection makes N a clean, controllable variable. HTTP/2 multiplexing and multi-source downloading are future work.

Q61. Why 8–10 blocks? How did you choose sample sizes?

Answer. The testbed is nearly deterministic, so 8–10 paired blocks give intervals far narrower than the effects of interest; screens used 3–5 blocks and are labelled screens. The v2 study used 8 blocks plus a t-interval sensitivity check. No formal power analysis was done; say so if asked.

Q62. Why reward $-\Delta t$ and not throughput?

Answer. Completion time is the objective, and summing $-\Delta t$ over an episode gives minus the completion time (exactly with $\gamma = 1$). Throughput rewards can be gamed by short bursts; Review 1 also proposed penalty terms, which LinUCB's reward kept.

Q63. How was the oracle's switching cost measured?

Answer. With a separate switch-injection campaign on the replayed traces ("tau"): the extra time caused by changing N at an epoch boundary, aggregated per trace and floored at zero (an earlier signed version was rejected because it let the cost-aware oracle beat the cost-free one). The oracle pays tau at every switch.

Q64. What about reliability, not just speed?

Answer. On the corrected engine all 230 ablation runs had zero failed attempts, retries or wasted bytes, and transfer efficiency was 100 %. On the original engine, the 30 s deadline re-downloaded up to 112 % of the file in one aborted capped run.

Q65. Why a 512 MiB file and a maximum of 16 connections?

Answer. 512 MiB makes downloads long enough (tens of seconds) for a controller to act many times and for a mid-download cap at 8 s to matter; earlier 4–16 MiB files left the learners almost no decisions. 16 matches aria2's per-server maximum and our etiquette limit for public mirrors.

Q66. Is opening 16 connections fair to other users?

Answer. Not necessarily: N TCP flows take roughly N shares of a shared bottleneck. We measured speed, not fairness, and list it as a limitation.

13.10 Hostile or tricky questions

Q67. You changed your title from RL to "when does it help". Isn't that moving the goalposts?

Answer. Review 1 already framed the project as an investigation ("whether jointly selecting (N, S) provides a measurable advantage") and said evaluation "does not depend exclusively on successful RL convergence". The system, scope and policies stayed as planned; the evidence changed the emphasis.

Q68. Isn't comparing to the best fixed setting chosen after the fact unfair?

Answer. It is generous to fixed settings, yes, which is why we also report one *global* setting (live pilot: 3.7 % from per-condition best) and a selector that chooses online (0.999 vs per-workload best).

Q69. Your v2 win is one seed. Couldn't it be luck?

Answer. It could be partly. We selected seed 1 on validation only, disclosed before the live runs that other seeds were worse, and report the result as "one selected seed from a procedure that is not reliably good across seeds".

Q70. Why did v2 do better live than its simulator predicted in H_RTT110?

Answer. We do not know: 68.6 s live vs 76.4 s predicted, while the fixed arms ran 4 % slower than predicted. We report it as unexplained.

Q71. If the drop rule beats v2 at home, why learn at all?

Answer. In our data a simple rule is the bar a learner has to clear. v2 beat it in 3 scenarios (two where the rule's tuning was wrong), lost in 2. Learning added robustness across cap strengths but not a clear win over a well-tuned rule.

Q72. Removing the request timeout is dangerous. Would you ship that?

Answer. Not as is. A fully stalled connection would wait forever; the CLI keeps 30 s. The right fix is a stall-based timeout (no progress for x seconds), which is future work.

Q73. 16 connections to a public mirror is impolite.

Answer. We capped at 16 per server, left 5 s between runs, sent an identifying User-Agent, and would abort a server on any 429/503. We note fairness as unmeasured.

14. Presentation and Demo Guide

14.1 Suggested slide flow (about 15 minutes)

Table 14.1: Suggested slides, figures and presenter (by area of the work division)

#	Slide	Content and figure	Presenter
1	Title and question	"When is there anything for adaptation to exploit?"	any
2	Background	N, S, range requests; why adaptation is assumed to help	Sourajyoti Basak
3	System	Figure 3.1; invariants; policy interface	Sourajyoti Basak
4	Test platform	Paced server rules, the cap (Figure 3.7), netem, Mahimahi	Sourajyoti Basak
5	Method	Blocks, geometric ratio, bootstrap, headroom gates, predeclared protocols, claims checker	Shwetank Shrijan
6	Result 1: fixed vs aria2	62.4 %; mechanism (Figure 6.2); axel/lftp, sweep	Shwetank Shrijan
7	Result 2: no opportunity	Screens; cellular oracle (Figure 7.1)	Ayush Konar
8	Result 3: constructed cap	Causal −23.4 %; engine ablation (Figure 8.3); "engine before policy"	Sourajyoti Basak/ Shwetank Shrijan
9	Result 4: learned agents	LinUCB, DQN v2–v6; v1 0.907; joint 1.471; v2 3 of 4 (Figure 9.2)	Ayush Konar
10	Result 5: real networks	Live pilot 3.7 %; mirror check 0 of 3 (Figure 10.2)	Shwetank Shrijan
11	Summary	Forest plot (Figure 9.4); why RL did not win	Ayush Konar
12	Novelty and limits	Section 11.3; top 5 limitations	any
13	Demo	Three steps below	Sourajyoti Basak/ Shwetank Shrijan

The presenter column is a suggestion that follows the Review 2 work division; adjust as you agree.

14.2 Demo script (about 2.5 minutes plus talking)

Build once beforehand so no compile time is spent in the viva:

```
go build -o /tmp/dl ./cmd/downloader
go build -o /tmp/rl_bench ./testdata/rl_capped/tools/
sudo -n tc qdisc show dev lo     # must show "noqueue": no shaping left
   over
```

Step 1: live download from a public mirror (about 30 s).

```
/tmp/dl -url https://cdn.kernel.org/pub/linux/kernel/v3.x/linux-3.0.tar.xz
    \
  -out /tmp/linux-3.0.tar.xz -concurrency 8 -chunk 1048576 \
  -sha256 ef5e8c2dedb20a6eab01bb379e9982cd7d47cf95bd87c9ee3e872272e85c24b6
```

Point out: `-size` omitted, so the CLI probes with `Range: bytes=0-0` and requires 206; 8×1 MiB was on average 3.7 % slower than the per-condition best in the pilot; the checksum is kernel.org's published value. Fallback without internet: skip to step 2.

Live download from cdn.kernel.org with checksum verification

```
$ ./downloader -url https://cdn.kernel.org/pub/linux/kernel/v3.x/linux-3.0.tar.xz -out
    linux-3.0.tar.xz -concurrency 8 -chunk 1048576 -sha256
    ef5e8c2dedb20a6eab01bb379e9982cd7d47cf95bd87c9ee3e872272e85c24b6
server supports range requests; size 63798008 bytes
downloaded 63798008 bytes to linux-3.0.tar.xz in 26.7 s (2.39 MB/s)
sha256 OK ef5e8c2dedb20a6eab01bb379e9982cd7d47cf95bd87c9ee3e872272e85c24b6
$ curl -s https://cdn.kernel.org/pub/linux/kernel/v3.x/sha256sums.asc | grep linux-3.0.tar.xz
ef5e8c2dedb20a6eab01bb379e9982cd7d47cf95bd87c9ee3e872272e85c24b6  linux-3.0.tar.xz
$ sha256sum linux-3.0.tar.xz
ef5e8c2dedb20a6eab01bb379e9982cd7d47cf95bd87c9ee3e872272e85c24b6  linux-3.0.tar.xz
```

Figure 14.1: Terminal capture of step 1.

Step 2: the capped server with the retrained agent (about 70 s).

```
/tmp/rl_bench -system ours -n 16 -chunk 1048576 -no-request-timeout -split
    \
  -policy dqn6 -dqn-model testdata/rl_cap_train/model/dqn_cap_candidate.
    json -window 16 \
  -schedule t1 -cap-c 8388608 -cap-gamma 0.05 -cap-at 8 \
  -netem-latency-ms 40 -netem-loss 0.4 -run-id demo -out /tmp/demo.json
```

Point out: the cap switches on at 8 s; the agent sees request times jump and lowers N (measured on 25 Oct: N 14–16 before the cap, 6 by 15.8 s; 69.2 s, content verified). Over 10 blocks this agent took 0.907 of fixed 16×1 MiB, but is slower than 16×32 MiB and aria2, because large ranges started before the cap keep the faster rate. Afterwards show `sudo -n tc qdisc show dev lo`. Shorter fallback (about 18 s): drop the policy and cap flags to run fixed 16×1 MiB uncapped.

Step 3: tests and the number checker (about 2 min).

```
go test -count=1 ./internal/... 2>&1 | tail -15
python3 tools/claims_check.py | tail -2    # 220 numbers recomputed from
    raw runs
```

Point out: no test needs internet; the checker recomputes every number and fails on any it cannot reproduce; its self-test (`python3 tools/test_claims_check.py`) shows it rejects wrong numbers.

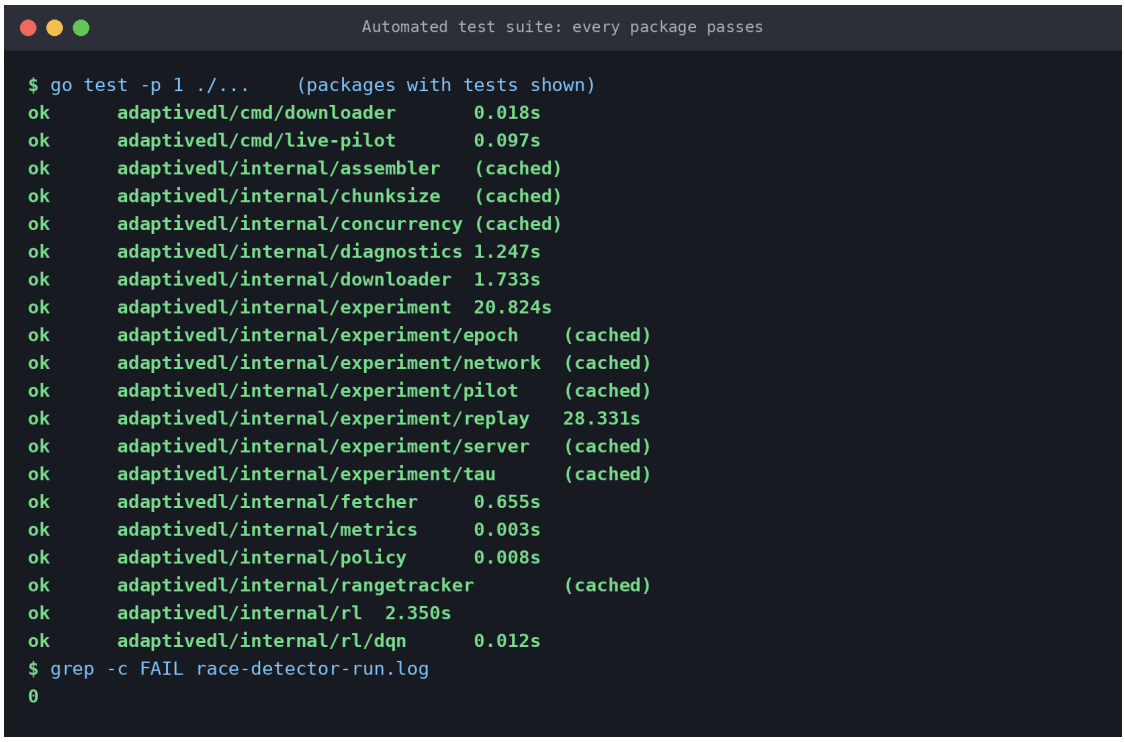


Figure 14.2: Terminal capture: the test suite passing.

Demo hygiene

Do not run anything heavy on the laptop during the demo (timing-sensitive). If the internet is down, say so and skip step 1. If something fails, say what failed; never hide it. The whole project's value is honesty.

14.3 Five sentences everyone should be able to say

1. "We asked when adapting connections and range size during a download actually beats a well-chosen fixed setting."
2. "A fixed setting beat aria2 by 62 % on a paced server, but that is request timing, not adaptation."
3. "Even an oracle that knows the future gains under half a percent on average on replayed cellular traces."
4. "Adaptation helped only on a server we built where concurrency has a time-varying cost, and fixing two engine details mattered more than any controller."
5. "Learned agents won only against same-range-size fixed settings on servers like the one they were trained on, and on real mirrors adapting cost time."

A. Glossary of Terms and Internal Labels

Term	Meaning
General	
N	Number of parallel connections (range requests in flight).
S	Range size (chunk size): bytes asked for in one request.
Range request, 206	`Range: bytes=a-b`; a supporting server replies `206 Partial Content` with `Content-Range`.
Half-open $[s,e)$	Bytes s to $e-1$; length $e-s$.
RTT	Round-trip time. Our netem link: 40 ms each way = 80 ms RTT.
KiB, MiB, MB/s	1 KiB = 1,024 B; 1 MiB = 1,048,576 B; MB/s = 10^6 B/s.
Arm	One option in an experiment (fixed setting, controller, agent, tool).
Block	One run of every arm in seeded random order under the same conditions.
Time ratio R	Geometric mean of per-block time ratios; $R < 1$: first option faster.
Bootstrap interval	95 % percentile interval from 10,000 resamples of blocks.
Screen	A small experiment (3–5 blocks) to check a direction, not confirm.
Protocol, amendment	The plan fixed before the first run; dated additions at its end.
Headroom H	Oracle's relative gain over the best fixed setting; an upper bound for any policy.
Gate	A criterion fixed in advance that a stage must pass to continue.
Oracle	Hypothetical controller that knows the future; not runnable.
Goodput	File size / completion time.
Transfer efficiency	Useful bytes / all bytes received.
Straggler splitting	Idle worker takes half the unreceived bytes of the in-flight range with the most bytes left (≥ 2 MiB remaining).
Request deadline (T30)	The original 30 s whole-request timeout. `_NT`: no deadline. `_split`: splitting on.
Original / fixed engine	With T30 and no splitting / without the deadline and with splitting.
Best fixed setting	The fastest fixed (N,S) in a scenario, chosen after the runs.
Server scenarios	
D2	512 MiB; 640 KiB/s, 160 from 2 s, 2,500 from 4 s; per response; netem 40 ms + 0.4 %.
HEALTHY / t1	640 KiB/s rising to 2,500 from 1 s; no cap.
CAP0, CAP8	Soft cap from the start; cap from 8 s.
Soft cap	$\min(\text{base}, C/(k(1+\gamma(k-1))))$, k = active requests including the new one, $C = 8$ MiB/s, $\gamma = 0.05$, fixed at arrival.
Request-start (arrival-time) rule	A response keeps the rate in force when its request arrived.
Live-rate mode	Server mode that re-evaluates a response's rate during the response (control in `rl_joint`).
S0–S5	The six pacing-sweep schedules (S0 = D2).

Term	Meaning
N1, N2	NGINX `limit_rate 640k`; `limit_rate_after 1m` + `limit_rate 640k`.
H_C12, H_RTT110, H_DIP, H_C20	v2 held-out scenarios (Table 3.4).
G_C16, G_RTT, G_D2	v1 held-out scenarios: $C = 16$ MiB/s; 80 ms each way; the d2 schedule.
CAP8L, HEALTHYL	`rl_joint` Stage L scenarios on the live-rate server.
Controllers and agents	
Causal drop controller / drop rule	Switch once to a preset target when windowed median per-request throughput falls below half its running max.
C=8-tuned drop rule	N-only drop rule, $k = 3$, ratio 0.5, capped $N = 4$ (4 suits $C = 8$ MiB/s).
Workload selector	Probe at 16×1 MiB, classify after 20 samples, move once to the class target.
Online tuner	FastBioDL-style: 2 s windows, $U = T/K^N$ ($K = 1$), reverse and halve on a ≥ 5 % drop.
LinUCB	Linear contextual bandit with an upper-confidence exploration bonus.
DQN v1–v6	Successive Double DQN candidates; v2–v3 joint N, S; v4 onward N only.
aria2-F	aria2 `-x16 -s16 -k1M`, selected by calibration v2.
Cap-trained v1 (`dqnc`)	DQN retrained on `sim_env_cap` (`rl_cap_train`).
`dqnj`	Joint N + range-size agent (`rl_joint`).
v2 (cap)	Broader-trained cap agent (`rl_cap_train_v2`); not the same as early DQN v2.
Internal study labels	
U1–U8	Eight wall-clock unstable profiles for DQN v2/v5 vs aria2.
A2–E2	v6 trajectories: A bandwidth collapse/recovery, B latency spike, C burst loss, D server throttle/release, E combined.
L1–L4	Environment designs for a "v7" DQN: L2 -0.3 %, L3 8.1 %, L4 17.6 % (simulator only).
L-P1 / `live_pilot_l1`	The live-internet pilot (unrelated to L1 above).
T1, T2	Live probes: per-request throttle (no concurrency reversal); adaptive range size (no reversal).
M1–M3 (traces)	Mobile-trace chain: audit, acquisition, envelope replay. (In the mirror check, M1/M2 are time slots.)
Stage 0, Stage 0S	Mahimahi replay pilot for the oracle; S-protocol feasibility pilot (GO).
tau	Measured transition cost of switching N at an epoch boundary.
S-protocol v7 ("salvage S")	Frozen protocol for range-size adaptation on a static server (not DQN v7).
Track 1	Heuristic joint N+S on D2 (2.32× slower than fixed).
Track 2 / 2-B	N-dependent server throttle: analytical pre-screen stopped / the soft cap built live.
P1, P2	Track 2-B experiments: cross-workload selector; within-download causal controller.
W1–W6	The six-workload family for the selector.
Stages K, A, B, C, D	In the cap studies: calibration, blind parity, training, live in-distribution, live held-out.
kcdn, kedge, riken, tuna	cdn.kernel.org, mirrors.edge.kernel.org, ftp.riken.jp, mirrors.tuna.tsinghua.edu.cn.

B. Numbers Cheat Sheet

What	Number	Study
D2: fixed 16×4 MiB vs aria2	0.3756 [0.3726, 0.3787]; 30/30; 20.12 s vs 53.56 s	aria2_competitive_ benchmark
Granularity-matched control	0.862 [0.834, 0.890]	same
Ranges after the speed-up	112 of 128 (3,360 of 3,840 over 30 runs)	same
Pacing sweep	28/30 pairs; 0.378–0.998	d2_pacing_sweep
NGINX	0.863, 0.597; 10/10	nginx_limit_rate
axel/lftp (HEALTHY, D2)	0.345–0.375	baselines_ext
CAP0 lftp	114 s (never re-splits)	baselines_ext
Delay×loss, D2	9 of 9 cells; 0.362–0.457	netem_sweep
Track 1 joint heuristic vs fixed (D2)	2.316	joint_ns_d2
Cellular oracle	mean 0.406 %; LCB −0.26 %; bar 10 %	final_oracle_analysis
L4 simulated headroom	17.6 % (not live)	diagnostics/l4_ environment_design
Cap gate S-margin	3.74 % [3.09, 4.45] vs 5 %	n_dependent_server_ joint_ns
P2 causal joint vs best of 3	0.766; 58.7 s vs 76.7 s; aria2 53.6 s	same
N-only / *S*-only	0.808 / 1.070	same
W1–W6 selector vs per-workload best	0.999 [0.969, 1.029]; workload [0.873, 1.143]	same
Selector vs family-best	0.770; workload [0.455, 1.304]	same
Splitting, CAP0	0.599; then vs aria2 0.995	split_ablation
CAP8 16×32 no deadline vs aria2	1.016; 54.4 s vs 53.5 s	split_ablation
Causal vs best fixed (fixed engine)	1.071	split_ablation
Selector vs 16×32 (CAP0)	1.229	split_ablation
Online tuner	1.212 (CAP8), 1.220 (no cap); 2.1 s cut; 8.1 s climb	online_tuner
DQN v2 / v5 / v6 vs aria2	1.757 / 2.322 / 1.456	final_evaluation*
v6 long files	$1.805 \rightarrow 1.062 \rightarrow 0.909$	diagnostics
Frozen v6 on CAP8 (start 16)	1.356 vs 16×32; 0.931 vs 16×1	rl_capped
Model-based attempt	0 of 6 criteria	model_based_ns (branch)
Cap-trained v1, CAP8	0.907 [0.893, 0.921]; sim 0.912	rl_cap_train

What	Number	Study
v1 vs 16×32 / aria2	1.298 / 1.310	rl_cap_train
v1 held out	1.177, 1.016, 1.015 (0 of 3)	rl_cap_train
Joint sim headroom (hand rule)	0.352	rl_joint
Joint agent CAP8	1.471 [1.405, 1.546]; 4 of 4 slower	rl_joint
Live-rate control	50.9 s $\rightarrow$ 67.4 s; agent 1.029	rl_joint
v2 CAP8	0.845 [0.840, 0.850]; vs v1 0.929; vs drop 1.048	rl_cap_train_v2
v2 HEALTHY	1.047	rl_cap_train_v2
v2 held out	0.882, 0.819, 0.952, 1.007 (3 of 4)	rl_cap_train_v2
v2 blind check	+1.4, -3.3, +5.6, +5.6 %	rl_cap_train_v2
Live pilot headroom	3.7 % [1.0, 4.8]; 8×1 MiB; 11/16 within 5 %; max 15.3 %	live_pilot_l1
Second network	7.2 % [1.2, 12.2]	live_pilot_l1
Mirror check v2	1.401 (kcdn, slower), 1.654, 1.215	live_mirror_check
Mirror check drop	1.125, 1.063, 1.332; switched 4/5 on kedge and riken	live_mirror_check
Claims checker	220/220; self-test 9/9	tools
Test functions (Review 2)	549 in 29 packages	—

C. Inventory of Every Study Folder

Every folder under `testdata/`, in roughly chronological order. "PC" is the section of `docs/PROJECT_CONTEXT.md` that documents it. Ratios are ours/reference time unless stated.

Folder	Question and design	Outcome	Status (PC)
Early LinUCB and benchmark era			
`baseline`	Reference 4-mode matrix: 5 scenarios × 4 modes × 10 reps = 200 runs	Baseline measurements; no mode claimed superior	Descriptive (18)
`calibration`	Is the LinUCB reward sensible? 5 scenarios (stable, throttle, latency, failure, combined)	Diagnostic only	Descriptive (22.1–22.3)
`benchmark`	First 5-mode benchmark, 250 runs, 256 KiB file	LinUCB chose "none" 100 %: exploration lock-in	Superseded (22.6)
`benchmark_v2`	Same after an exploration fix, 250 runs	Only ∼2 learned decisions: horizon too short	Descriptive (22.9–22.10)
`benchmark_v3`	Same with a 4 MiB file, 250 runs	32–122 learned decisions; lock-in persists	Descriptive (22.11)
`benchmark_v4`	Standard vs periodic-exploration LinUCB, 100 runs	Periodic exploration gave effective actions, scenario-dependent	Descriptive (22.14)
`benchmark_v5`	Six-mode 4 MiB benchmark, 300 runs	No winner claimed	Descriptive (22.15)
`benchmark_aria2`	aria2 (4 conn) on the v5 scenarios, 50 runs	Confounded by request granularity	Descriptive (22.16–22.17)
`benchmark_v6`	1 s wall-clock throttle, 6 modes, 60 runs	Concurrency-adaptive 1324.5 ms vs fixed 2057.9 ms; LinUCB-periodic 1323.4	Descriptive (22.18)
`benchmark_aria2_v2`	aria2 under the same throttle, 10 runs	With a granularity-matched control	Descriptive (22.19–22.20)
`final_research_audit`	Deterministic audit of the frozen chain	All counts re-derived; no discrepancy	Synthesis (22.21)
DQN era			
`diagnostics`	About 40 sub-studies: LinUCB diagnostics, DQN v1–v6 training/validation/live, aria2 calibration (configs C and F), granularity, L2–L4, T1/T2, M2/M3, long-duration	See Chapters 7 and 9	Mixed (22.7–22.51)
`final_evaluation`	DQN v2 vs aria2 v1 (4 conn), U1–U8 × 30 pairs, 4 MiB	1.757; faster in 31/240	Negative (22.26)
`final_evaluation_v5`	v5 vs aria2-F, 240 pairs, 16 MiB	2.322; 1/240	Negative; model stale (22.32, 22.102)
`final_evaluation_v6`	v6 vs aria2-F, A2–E2 × 30 pairs	1.456; 37/150; E2 0.948	Negative (22.35)
`*_smoke`	Harness smoke tests for v5 and v6	Not data	Smoke

Folder	Question and design	Outcome	Status (PC)
external	Real mobile traces (UGent/imec, NYU-METS, Mahimahi set)	Data only	Data (22.49)
Cellular oracle chain (Sep 26–28)			
oracle_preregistration_v2	Oracle protocol 1 (120 s epochs, ladder 1–16, signed tau)	Signed tau broke the oracle; superseded	Superseded (22.71–22.72)
oracle_preregistration_v3	Oracle protocol 2 (non-negative tau)	EVDO 1.61 %, LTE 0.03 % (2 traces)	Descriptive (22.73–22.74)
oracle_preregistration_v4	Final 5-trace gate protocol	Executed as Stages A (15 replays) and B (88 taus)	Executed (22.75–22.79)
final_oracle_analysis	Does headroom reach 10 %? $n = 5$ traces	Mean 0.41 %; LCB -0.26 %	**Failed gate** (22.80)
Range-size and mechanism screens (Sep 28–29)			
salvage_s_protocol_v7	Frozen protocol: S adaptation, static server, 512 MiB, $N = 4$	Ready for Stage 0S	Protocol (22.81–22.83)
salvage_s_stage0s	Feasibility pilot, 32 runs, 4 fixed arms	4/4 criteria pass: GO	Feasibility (22.84)
salvage_s_scaling_analysis	Does a larger file raise S headroom? Analytical	All switches ≤ 0; best +0.35 % vs 5 %	Failed screen (22.85)
salvage_s_mechanism_pivot	Any mechanism moving preferred S? 8 + 23 traces	No robust opportunity	Failed screen (22.86)
adaptive_n_mechanism_screen	Any mechanism for adaptive N?	Only degrade (+6.66 %), rejected live	Failed screen (22.87)
final_project_synthesis	What can the paper claim (as of Sep 28)?	Later overtaken by the aria2 result	Synthesis (22.88)
aria2_competitive_benchmark	Fixed 16×4 MiB vs strongest aria2 on D2; 30 final pairs	0.3756; control 0.862	**Confirmed** (22.89)
joint_ns_phase_pacing_screen	Causal joint N+S under phase pacing (analytical)	+3.7 % vs 5 %	Failed screen (22.90)
live_joint_ns_queueing	Shared bottleneck + iperf3, 45 runs	No interior N optimum	Failed gate (22.91)
joint_ns_loss_mechanism	Correlated loss + retry (analytical); no-deadline re-check	$N^* = 16$ in 9/9; S shift was a deadline artefact	Failed screen (22.92)
joint_ns_d2	Track 1: heuristic joint N+S on D2, 80 runs	2.316 vs best fixed	Negative (22.93)
n_dependent_server_joint_ns	Track 2/2-B: pre-screen; soft cap; gate; P1, P2; S-only; timeout screen; W1–W6 (240 runs)	Gate failed S-margin; P2 0.766; selector 0.999 / 0.770	Mixed; capped claims superseded by split_ablation (22.94–22.101)
Publication readiness and live internet (Oct 11–24)			
d2_pacing_sweep	Six schedules S0–S5, 60 runs	28/30 pairs ours faster	Screen (22.104)
nginx_limit_rate	Stock NGINX, 20 runs	0.863, 0.597; parity expectation not met	Screen (22.104)
live_pilot_l1	4 mirrors $\times$ slots, 8 cells + aria2 + curl	3.7 % [1.0, 4.8]; second network 7.2 %	Descriptive (22.106)

Folder	Question and design	Outcome	Status (PC)
`split_ablation`	Engine ablation, 4 scenarios × 10 blocks, 230 runs	Splitting 0.599; vs aria2 0.995 / 1.016; causal 1.071	**Confirmed** (22.107)
`online_tuner`	FastBioDL-style tuner, 64 runs	1.212, 1.220	Negative (22.108)
`rl_capped`	Frozen v6 on the capped server, 50 runs	1.356 (start 16)	Negative screen (22.108)
After the reviews (Oct 25)			
`rl_cap_train`	Cap-trained DQN v1: parity, training, live 120, held out 65	0.907; 0 of 3 held out	Confirmed in-dist.; generalization failed (22.109)
`baselines_ext`	axel, lftp vs ours and aria2, 80 runs	0.345–0.375; level under CAP8	Screen (22.109)
`netem_sweep`	3×3 delay–loss grid, 108 runs	D2 9/9; CAP8 sign varies	Screen (22.109)
`rl_joint`	Joint N + range-size agent: parity 8, live 150, live-rate 40	1.471; 4 of 4 slower; 50.9 s → 67.4 s	Negative (22.109)
`rl_cap_train_v2`	Broader training + drop rule: K 24, A 8, C 64, D 160	0.845; 3 of 4 held out	Confirmed, partial (22.109)
`live_mirror_check`	v2 and drop rule on 3 mirrors; M1 aborted, M2 60/60	No harm on 0 of 3 for both	Descriptive (22.109)

D. Repository Map and How to Reproduce

D.1 Top-level layout

```
AGENTS.md, CLAUDE.md      working rules for contributors (human or AI)
queue.md                  running work queue
go.mod                    module adaptivedl, Go 1.27.1 (only dependency:
    Gonum)
cmd/                      command-line programs (below)
internal/                 the engine, controllers, policies, RL, experiment
     platform
python/                   training-only code (PyTorch): simulators, DQN,
    analysis
testdata/<study>/         one folder per study (later ones: protocol.md,
    manifest, raw/, analysis/, tools/)
tools/claims_check.py     recomputes every paper number from raw artifacts
reports/                  paper draft, reviews, viva material, this
    handbook
docs/                     PROJECT_CONTEXT.md (living log), REPRODUCING.md,
    notes
```

D.2 `internal/` packages

`rangetracker`, `fetcher`, `assembler`, `metrics`, `concurrency`, `chunksize`, `downloader` (config, worker pool, policy runner), `policy`, `rl` (+ `rl/dqn`), `experiment` (+ `server`, `network`), `diagnostics`.

D.3 `cmd/` programs

Program	Purpose
`downloader`	User-facing CLI: probe with `bytes=0-0`, download, `-sha256` verification.
`experiment`	Controlled 4-mode comparison and the formal matrix (incl. LinUCB modes).
`calibration`	LinUCB calibration and reward diagnostics.
`linucb-diagnostic`, `linucb-horizon`	Forced-action exploration diagnostic; post-warm-up decision count vs file size.
`aria2-benchmark`, `aria2-wallclock`	aria2 on the controlled server; under the 1 s wall-clock throttle.
`aria2-calibration`, `-v2`	Choose one global aria2 config on a validation set (v2 selected config F).
`granularity-study`	Our engine fixed at 4×1 MiB to isolate granularity effects.

Program	Purpose
`wallclock-throttle`	Six-mode 1 s wall-clock throttle scenario.
`dqn-transfer`, `dqn-live-v5`, `dqn-live-v6`	Live transfer validation of DQN candidates.
`final-evaluation`, `-v5`, `-v6`	Frozen DQN head-to-heads against aria2.
`l2-aria2-probe`, `l4-live-probe`, `t1-live-probe`	Environment-design live probes.
`epoch-pilot`, `tau-injection`, `oracle`, `mm-replay`, `m3-replay`	Cellular trace replay, switching cost and oracle analysis.
`live-pilot`	Live-internet runner (fixed cells, aria2, curl, policy arms).

Each later study also has its own harness under `testdata/<study>/tools/` (e.g. `split_bench.go`, `tuner_bench.go`, `rl_bench.go`, `joint_bench.go`, `v2_bench.go`).

D.4 Reproducing

```
go build ./... && go vet ./... && gofmt -l internal cmd
go test ./...                        # add -race; -p 1 for replay tests
pytest python/tests                  # Python unit tests
python3 tools/claims_check.py        # 220 numbers from raw artifacts
python3 tools/test_claims_check.py # checker self-test
python3 testdata/<study>/tools/analyze.py    # re-run one study's analysis
python3 reports/handbook/make_figs.py        # this handbook's new figures
```

Live runs need root for `tc netem`, aria2 1.37.0 and (for traces) Mahimahi; see `docs/REPRODUCING.md`. Never run timing-sensitive experiments while the machine is loaded.

E. References

[1] R. Fielding, M. Nottingham and J. Reschke, "HTTP Semantics," RFC 9110, IETF, 2022 (Range requests, Section 14).

[2] aria2 project, "aria2: The ultra fast download utility," version 1.37. `https://aria2.github.io/`

[3] Z. Liu, R. Kettimuthu, I. Foster and P. H. Beckman, "Toward a smart data transfer node," *Future Generation Computer Systems*, vol. 89, pp. 10–18, 2018.

[4] R. M. Swargo, E. Arslan and M. Arifuzzaman, "Adaptive parallel downloader for large genomic datasets" (FastBioDL), arXiv:2508.05511, 2025.

[5] R. M. Swargo and M. Arifuzzaman, "Elastic data transfer optimization with hybrid reinforcement learning," arXiv:2511.06159, 2025.

[6] M. Arifuzzaman, B. Bockelman, J. Basney and E. Arslan, "Falcon: Fair and efficient online file transfer optimization," *IEEE TPDS*, vol. 34, no. 8, pp. 2265–2278, 2023.

[7] P. Balaprakash, V. Morozov, R. Kettimuthu, K. Kumaran and I. Foster, "Improving data transfer throughput with direct search optimization," *Proc. ICPP*, 2016, pp. 248–257.

[8] S. Abdollah, C. Partridge and S. Shannigrahi, "MDTP: An adaptive multi-source data transfer protocol," arXiv:2505.09597, 2025.

[9] K. Spiteri, R. Urgaonkar and R. K. Sitaraman, "BOLA: Near-optimal bitrate adaptation for online videos," arXiv:1601.06748, 2016.

[10] T. Liu et al., "DeLoad: Demand-driven short-video preloading with scalable watch-time estimation," arXiv:2510.18459, 2025.

[11] R. Netravali et al., "Mahimahi: Accurate record-and-replay for HTTP," USENIX ATC, 2015.

[12] K. Winstein, A. Sivaraman and H. Balakrishnan, "Stochastic forecasts achieve high throughput and low delay over cellular networks" (Sprout), USENIX NSDI, 2013.

[13] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, 2015.

[14] H. van Hasselt, A. Guez and D. Silver, "Deep reinforcement learning with double Q-learning," AAAI, 2016.

[15] L. Li, W. Chu, J. Langford and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation" (LinUCB), WWW, 2010.

[16] M. Mathis, J. Semke, J. Mahdavi and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," *ACM SIGCOMM CCR*, vol. 27, no. 3, 1997.

[17] Linux `tc-netem`(8) manual page.

The paper cites only references checked against an authoritative record (`reports/references.md`). In that log, [11]–[15] and [17] are verified (18 Oct); [16] (Mathis) is a background reference for this handbook only. [3] and [5] appear in Review 1 and Review 3 but are flagged *unresolved* in the log and are *not* cited in the paper: do not rely on them in the viva.